

AppForge — Conversational + Visual App Creator

Complete Implementation Blueprint

Table of Contents

- 1. Platform Overview
- 2. Tech Stack
- 3. Project Structure
- 4. Platform Database Schema
- 5. Agent System
- 6. Backend API
- 7. Frontend Pages & Components
- 8. Visual Editors
- 9. Generated App Structure
- 10. Preview System
- 11. Module System
- 12. AppModel Index
- 13. Binding System
- 14. Rules Engine
- 15. Workflow Engine
- 16. Database Management
- 17. Authentication & Authorization
- 18. Real-time & Collaboration
- 19. Deployment & Export
- 20. AI Agent Builder
- 21. AI-Powered Application Features
- 22. Organization & Multi-Tenancy
- 23. Discovery & Templates
- 24. Implementation Phases

1. Platform Overview

1.1 What It Is

A platform where users describe applications in natural language, get a fully working app with live preview, then visually edit every aspect — UI, data models, workflows, rules, and navigation — through specialized visual editors. Every visual edit produces a natural language instruction that the same LLM agent processes, keeping the system simple.

1.2 Core Loop

User Input (chat or visual editor)

→ Orchestrator classifies intent

→ Routes to appropriate agent

→ Agent reads AppModel index for context

→ Agent reads/edits source code

→ Validator confirms build passes

→ Indexer updates AppModel

→ Preview hot-reloads

→ All editors refresh

1.3 Key Principles

- Code is the source of truth. The AppModel index is a navigation aid, not a code generator.
- Every visual editor action produces a text instruction. One LLM pipeline for everything.
- Generated apps use PostgreSQL from day one. Docker Compose handles local setup.
- Apps are built module by module, not all at once. Each module is a manageable generation task.
- The LLM is a very fast developer, not a magic generator. Humans guide, validate, and refine.
- Generated apps can be intelligent. LLM capabilities are embedded into data processing, workflows, and UI — not just chatbots.
- Organizations are the root entity. Users, roles, departments exist at the org level and are shared across all apps.
- RBAC is three layers deep: org-level roles, app-level roles, and field/record-level access — enforced everywhere.
- Generated apps follow Clean Architecture: domain logic has zero framework dependencies, services orchestrate use cases, repositories abstract the database, API routes are thin HTTP handlers.

2. Tech Stack

2.1 Platform (the builder tool itself)

```
Backend:
  Runtime:      Python 3.12+
  Framework:    FastAPI
  LLM SDK:      Claude Agent SDK (claude-agent-sdk)
  LLM Models:   Claude Sonnet 4 (primary), Claude Haiku 4.5 (utility)
  Database:     PostgreSQL 16 (platform state – projects, conversations, etc.)
  ORM:          SQLAlchemy 2.0 + asyncpg
  Migrations:   Alembic
  Task Queue:   None initially (async endpoints suffice), later: Celery + Redis
  WebSocket:    FastAPI WebSocket for real-time preview updates
  SSE:          sse-starlette for streaming agent progress
  File Storage: Local filesystem (output/{project_id}/)
  Container Mgmt: Docker SDK for Python (docker-py)

Frontend:
  Framework:    Next.js 15 (App Router)
  Language:     TypeScript 5.7+
  Styling:      Tailwind CSS 4
  State:        Zustand (global state) + React Query (server state)
  Visual Editors:
    Flow/Graph: React Flow (workflows, navigation, ERD)
    UI Canvas:  GrapesJS + @grapesjs/react (drag-and-drop UI editor, white-labeled)
    Code Editor: Monaco Editor (embedded VS Code editor)
    Icons:      Lucide React
  Drag & Drop:  dnd-kit (for non-canvas drag interactions)
  HTTP:         ky (fetch wrapper)
  WebSocket:    native WebSocket API
  Forms:        React Hook Form + Zod validation
```

2.2 Generated Apps (what the platform produces)

```
Framework:    Next.js 15 (App Router)
Language:     TypeScript 5.7+
Styling:      Tailwind CSS 4
Database:     PostgreSQL 16 via Docker Compose
ORM:          Drizzle ORM + pg driver
Auth:         NextAuth.js v5 (when auth is needed)
Email:        Resend (when email is needed)
File Upload:  UploadThing or local (when file upload is needed)
Containerized: docker-compose.yml with Postgres service
```

3. Project Structure

3.1 Platform Monorepo

```

appforge/
├── backend/
│   ├── main.py                # FastAPI app, CORS, startup/shutdown
│   ├── config.py             # Environment config, constants
│   ├── database.py           # Platform DB connection (SQLAlchemy)
│   └── models/               # SQLAlchemy models (platform DB)
│       ├── __init__.py
│       ├── project.py        # Project model
│       ├── conversation.py    # Chat history model
│       ├── module.py         # Module model
│       └── user.py           # Platform user model
│   └── routers/              # FastAPI routers
│       ├── __init__.py
│       ├── projects.py       # Project CRUD + file operations
│       ├── generate.py       # App generation (SSE streaming)
│       ├── refine.py         # Chat refinement (SSE streaming)
│       ├── preview.py        # Dev server lifecycle
│       ├── modules.py        # Module management
│       ├── app_model.py      # AppModel index read/write
│       ├── data_browser.py   # Browse/edit generated app's DB
│       ├── visual_editors.py # Endpoints for visual editor actions
│       └── auth.py           # Platform authentication
│   └── agents/               # LLM agent definitions
│       ├── __init__.py
│       ├── orchestrator.py   # Agent #0: Intent router
│       ├── planner.py        # Agent #1: Multi-turn planning
│       ├── refiner.py        # Agent #2: Change request handler
│       ├── explainer.py      # Agent #3: Q&A about the app
│       ├── code_generator.py  # Agent #4: Initial code generation
│       ├── code_editor.py     # Agent #5: Targeted file edits
│       ├── scaffolder.py     # Agent #6: Add features to existing app
│       ├── indexer.py        # Agent #7: Update AppModel index
│       ├── validator.py      # Agent #8: Build verification
│       ├── reviewer.py       # Agent #9: Visual QA
│       └── workflow_generator.py # Agent #10: Workflow action generator
│   └── services/             # Business logic
│       ├── __init__.py
│       ├── preview_manager.py # Docker + dev server management
│       ├── project_manager.py # Project lifecycle
│       ├── db_proxy.py       # Proxy queries to generated app's DB
│       ├── app_model_service.py # AppModel index operations
│       ├── git_service.py    # Git operations for version history
│       └── export_service.py  # ZIP download, deployment prep
│   └── schemas/              # Pydantic schemas (request/response)
│       ├── __init__.py
│       ├── project.py
│       ├── generate.py
│       ├── refine.py
│       ├── app_model.py
│       └── visual_editor.py
│   └── alembic/              # Platform DB migrations
│       ├── versions/
│       └── env.py

```

```

├── alembic.ini
├── requirements.txt
├── Dockerfile
├── .env.example

├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── # Next.js App Router pages
│   │   │   ├── layout.tsx # Root layout with providers
│   │   │   ├── page.tsx # Landing / project list
│   │   │   ├── projects/
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── layout.tsx # Project workspace layout
│   │   │   │   │   ├── page.tsx # Project overview / chat
│   │   │   │   │   ├── preview/
│   │   │   │   │   │   ├── page.tsx # Live preview tab
│   │   │   │   │   ├── code/
│   │   │   │   │   │   ├── page.tsx # Code editor tab
│   │   │   │   │   ├── ui-editor/
│   │   │   │   │   │   ├── page.tsx # Visual UI editor tab
│   │   │   │   │   ├── data/
│   │   │   │   │   │   ├── page.tsx # Data model editor tab
│   │   │   │   │   ├── workflows/
│   │   │   │   │   │   ├── page.tsx # Workflow editor tab
│   │   │   │   │   ├── rules/
│   │   │   │   │   │   ├── page.tsx # Rules editor tab
│   │   │   │   │   ├── navigation/
│   │   │   │   │   │   ├── page.tsx # Navigation editor tab
│   │   │   │   │   ├── database/
│   │   │   │   │   │   ├── page.tsx # Data browser + SQL console
│   │   │   │   │   ├── settings/
│   │   │   │   │   │   ├── page.tsx # Project settings
│   │   │   │   └── globals.css
│   │   ├── components/
│   │   │   ├── layout/
│   │   │   │   ├── # Shell components
│   │   │   │   ├── Sidebar.tsx # Main navigation sidebar
│   │   │   │   ├── Header.tsx # Top bar with project info
│   │   │   │   ├── TabBar.tsx # Editor tab switcher
│   │   │   │   └── ProjectShell.tsx # Workspace wrapper
│   │   │   ├── chat/
│   │   │   │   ├── # Chat panel components
│   │   │   │   ├── ChatPanel.tsx # Main chat container
│   │   │   │   ├── ChatInput.tsx # Message input with send
│   │   │   │   ├── ChatMessage.tsx # Individual message bubble
│   │   │   │   ├── ChatHistory.tsx # Scrollable message list
│   │   │   │   └── PlanApproval.tsx # Plan review + approve/reject
│   │   │   ├── preview/
│   │   │   │   ├── # Live preview components
│   │   │   │   ├── PreviewFrame.tsx # Device frame + iframe
│   │   │   │   ├── DeviceSelector.tsx
│   │   │   │   ├── RefineBar.tsx # Chat input below preview
│   │   │   │   └── PreviewLog.tsx # Collapsible agent log
│   │   │   ├── code/
│   │   │   │   ├── # Code editor components
│   │   │   │   ├── FileTree.tsx # File tree sidebar
│   │   │   │   ├── CodeEditor.tsx # Monaco editor wrapper
│   │   │   │   └── FileTab.tsx # Open file tabs
│   │   │   ├── ui-editor/
│   │   │   │   ├── # Visual UI editor (GrapesJS, white-labeled)
│   │   │   │   ├── EditorShell.tsx # GjsEditor + Canvas wrapper, disables default UI
│   │   │   │   ├── ComponentPalette.tsx # Custom blocks sidebar (shadcn/ui)
│   │   │   │   └── LayerPanel.tsx # Custom layer/component tree (shadcn/ui)

```

- └─ PropertiesPanel.tsx # Custom trait manager panel (shadcn/ui)
- └─ StylePanel.tsx # Custom style manager panel (shadcn/ui)
- └─ DeviceToolbar.tsx # Custom responsive device switcher
- └─ EditorToolbar.tsx # Custom top toolbar (undo, redo, preview, save)
- └─ FieldMapper.tsx # Bind model fields to UI components
- └─ grapes-plugins/ # Custom GrapesJS plugins
 - └─ appforge-blocks.ts # Register app component blocks
 - └─ appforge-components.ts # Custom component types + views
 - └─ tailwind-integration.ts # Tailwind CSS in canvas
 - └─ form-components.ts # Form field component types
- ─ data-editor/ # Data model editor
 - └─ ERDCanvas.tsx # Entity-relationship diagram
 - └─ EntityCard.tsx # Model card in ERD
 - └─ FieldEditor.tsx # Add/edit/delete fields
 - └─ RelationLine.tsx # Relationship arrows
 - └─ EnumEditor.tsx # Enum value manager
 - └─ IndexEditor.tsx # Database index manager
 - └─ SeedEditor.tsx # Seed data table
 - └─ ImpactAnalysis.tsx # Shows affected bindings
- ─ workflow-editor/ # Workflow editor
 - └─ WorkflowCanvas.tsx # React Flow canvas
 - └─ TriggerNode.tsx # Trigger node component
 - └─ ActionNode.tsx # Action node component
 - └─ ConditionNode.tsx # Branch/condition node
 - └─ NodePalette.tsx # Draggable node types
 - └─ NodeProperties.tsx # Selected node config
 - └─ VariablePicker.tsx # {{variable}} autocomplete
 - └─ WorkflowTester.tsx # Test run with sample data
 - └─ ExecutionLog.tsx # Step-by-step run result
- ─ rules-editor/ # Rules editor
 - └─ RulesTable.tsx # Main rules list
 - └─ ValidationRuleForm.tsx
 - └─ AccessRuleForm.tsx
 - └─ BusinessRuleForm.tsx
 - └─ ComputedRuleForm.tsx
 - └─ StateMachineEditor.tsx # Visual state diagram
 - └─ TriggerRuleForm.tsx
 - └─ ConditionBuilder.tsx # Visual condition builder
- ─ navigation-editor/ # Navigation editor
 - └─ NavCanvas.tsx # Screen flow diagram
 - └─ ScreenNode.tsx # Page/screen node
 - └─ NavProperties.tsx # Navigation config
 - └─ MenuEditor.tsx # Sidebar/navbar config
- ─ database/ # Data browser
 - └─ DataBrowser.tsx # Table data grid
 - └─ SQLConsole.tsx # Raw SQL editor
 - └─ TableSelector.tsx # Pick table to browse
 - └─ CellEditor.tsx # Inline cell editing
- ─ modules/ # Module management
 - └─ ModuleList.tsx # Module cards/list
 - └─ ModuleCard.tsx # Individual module
 - └─ ConnectionsMap.tsx # Cross-module dep graph
 - └─ ModuleWizard.tsx # New module creation
- ─ progress/ # Progress/streaming
 - └─ ProgressStream.tsx # SSE event display
 - └─ AgentSpinner.tsx # Loading states
 - └─ EventItem.tsx # Individual event render

```

├── shared/                                # Reusable components
│   ├── Button.tsx
│   ├── Input.tsx
│   ├── Select.tsx
│   ├── Modal.tsx
│   ├── Toast.tsx
│   ├── Tabs.tsx
│   ├── Badge.tsx
│   ├── Table.tsx
│   ├── ConfirmDialog.tsx
│   └── EmptyState.tsx
├── stores/                                # Zustand stores
│   ├── project-store.ts                  # Current project state
│   ├── chat-store.ts                    # Conversation state
│   ├── preview-store.ts                 # Preview server state
│   ├── app-model-store.ts               # Cached AppModel index
│   ├── editor-store.ts                  # Active editor state
│   └── ui-store.ts                      # UI state (panels, modals)
├── hooks/                                # Custom React hooks
│   ├── useSSE.ts                        # SSE stream consumer
│   ├── useProject.ts                    # Project data fetching
│   ├── useAppModel.ts                   # AppModel index queries
│   ├── usePreview.ts                    # Preview lifecycle
│   ├── useRefine.ts                     # Refinement submission
│   ├── useAgentAction.ts                # Generic agent action hook
│   └── useDataBrowser.ts                # Data browser queries
├── lib/                                  # Utilities
│   ├── api.ts                           # API client (ky instance)
│   ├── sse.ts                           # SSE parser utilities
│   ├── instruction.ts                   # Visual editor → text instruction builder
│   └── constants.ts                     # Shared constants
├── types/                                # TypeScript types
│   ├── project.ts
│   ├── app-model.ts                     # AppModel index types
│   ├── events.ts                        # SSE event types
│   ├── visual-editor.ts                 # Visual editor action types
│   └── workflow.ts                      # Workflow model types
├── package.json
├── next.config.ts
├── tsconfig.json
├── postcss.config.mjs
├── Dockerfile
├── output/                               # Generated projects live here
│   ├── {project_id}/
│   │   ├── src/
│   │   ├── docker-compose.yml
│   │   ├── app-model.json               # AppModel index for this project
│   │   └── ...
│   └── ...
├── docker-compose.yml                    # Platform services (platform DB, Redis)
├── Makefile                             # Dev commands
└── README.md

```

4. Platform Database Schema

The platform itself needs a database to track organizations, users, projects, conversations, and state.

4.1 Tables

```
-- =====
-- ORGANIZATION & IDENTITY TABLES
-- =====

-- Organizations (tenants)
CREATE TABLE organizations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(100) NOT NULL UNIQUE,          -- acme-corp → used in URLs
  logo_url    TEXT,
  plan        VARCHAR(50) NOT NULL DEFAULT 'free',   -- free, pro, enterprise
  settings    JSONB DEFAULT '{}',                  -- org-wide settings
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Platform users (who's using the builder)
CREATE TABLE platform_users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) NOT NULL UNIQUE,
  name        VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255),
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Org membership (many-to-many: users belong to orgs)
CREATE TABLE org_members (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES platform_users(id) ON DELETE CASCADE,
  platform_role VARCHAR(50) NOT NULL DEFAULT 'member', -- owner, admin, builder, member
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(org_id, user_id)
);

-- Departments
CREATE TABLE departments (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  parent_id   UUID REFERENCES departments(id) ON DELETE SET NULL, -- hierarchy
  head_user_id UUID REFERENCES org_people(id),                  -- department head
  display_order INTEGER NOT NULL DEFAULT 0,
  metadata    JSONB DEFAULT '{}',                              -- cost center, location, etc.
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(org_id, name)
);

-- Teams (sub-groups within departments)
CREATE TABLE teams (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  department_id UUID REFERENCES departments(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  lead_user_id UUID REFERENCES org_people(id),
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(org_id, name)
);
```

```

-- Org people (end-users of the generated apps – NOT platform_users)
CREATE TABLE org_people (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id            UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  email             VARCHAR(255) NOT NULL,
  name              VARCHAR(255) NOT NULL,
  title             VARCHAR(255), -- job title
  department_id     UUID REFERENCES departments(id) ON DELETE SET NULL,
  team_id           UUID REFERENCES teams(id) ON DELETE SET NULL,
  manager_id        UUID REFERENCES org_people(id) ON DELETE SET NULL, -- reporting line
  status            VARCHAR(50) NOT NULL DEFAULT 'active', -- active, inactive, invited
  avatar_url        TEXT,
  metadata          JSONB DEFAULT '{}', -- phone, location, hire_date,
etc.
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(org_id, email)
);

-- Org roles (custom roles defined per org)
CREATE TABLE org_roles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id            UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  name              VARCHAR(100) NOT NULL, -- admin, manager, staff, viewer
  description       TEXT,
  level             INTEGER NOT NULL DEFAULT 0, -- 0=lowest, higher=more
authority
  is_system         BOOLEAN NOT NULL DEFAULT FALSE, -- system roles can't be deleted
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(org_id, name)
);

-- Person-role assignments
CREATE TABLE org_person_roles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  person_id         UUID NOT NULL REFERENCES org_people(id) ON DELETE CASCADE,
  role_id           UUID NOT NULL REFERENCES org_roles(id) ON DELETE CASCADE,
  UNIQUE(person_id, role_id)
);

-- Groups (cross-cutting: audit committee, safety team, etc.)
CREATE TABLE org_groups (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id            UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  name              VARCHAR(255) NOT NULL,
  description       TEXT,
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(org_id, name)
);

-- Group membership
CREATE TABLE org_group_members (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id          UUID NOT NULL REFERENCES org_groups(id) ON DELETE CASCADE,
  person_id         UUID NOT NULL REFERENCES org_people(id) ON DELETE CASCADE,
  UNIQUE(group_id, person_id)
);

-- =====
-- RBAC POLICY TABLES
-- =====

-- App access policies (which roles/groups/people can access which apps)

```



```

CREATE TABLE app_access_policies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  target_type VARCHAR(20) NOT NULL,          -- role, group, department,
person
  target_id   UUID NOT NULL,                  -- references
role/group/dept/person
  access_level VARCHAR(50) NOT NULL DEFAULT 'user', -- user, admin, none
  created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(project_id, target_type, target_id)
);

-- Field-level access policies
CREATE TABLE field_access_policies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  model_name  VARCHAR(255) NOT NULL,          -- "Product", "SupportTicket"
  field_name  VARCHAR(255) NOT NULL,          -- "unitCost", "salary"
  target_type VARCHAR(20) NOT NULL,          -- role, group, department,
person
  target_id   UUID NOT NULL,
  can_view    BOOLEAN NOT NULL DEFAULT TRUE,
  can_edit    BOOLEAN NOT NULL DEFAULT TRUE,
  condition   TEXT,                          -- optional condition expression
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_field_access ON field_access_policies(project_id, model_name, field_name);

-- Workflow assignment policies (who handles which workflow tasks)
CREATE TABLE workflow_assignment_policies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  workflow_id VARCHAR(255) NOT NULL,          -- "expense-approval"
  node_id     VARCHAR(255) NOT NULL,          -- specific step in the workflow
  assign_type VARCHAR(50) NOT NULL,          -- role, group, department,
person,
-- manager_of_requester,
department_head
  assign_target UUID,                        -- NULL for dynamic types like
manager_of
  sla_hours     INTEGER,                    -- auto-escalate after N hours
  escalate_to   VARCHAR(50),               -- role name to escalate to
  created_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

-- =====
-- PROJECT & APP TABLES (updated)
-- =====

-- Projects (now belong to an org)
CREATE TABLE projects (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  short_id    VARCHAR(8) NOT NULL UNIQUE,    -- human-readable ID
  org_id      UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  description  TEXT,
  owner_id    UUID NOT NULL REFERENCES platform_users(id),
  status      VARCHAR(50) NOT NULL DEFAULT 'active', -- active, archived, deleted
  output_dir  TEXT NOT NULL,                -- filesystem path
  preview_port INTEGER,                     -- current preview port (null if
stopped)
  db_port     INTEGER,                      -- Postgres port for this project
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()

```

```

);

-- Modules within a project
CREATE TABLE modules (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  description TEXT,
  status       VARCHAR(50) NOT NULL DEFAULT 'active', -- active, generating, error
  display_order INTEGER NOT NULL DEFAULT 0,
  created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(project_id, name)
);

-- Module dependencies
CREATE TABLE module_dependencies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  module_id   UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
  depends_on_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
  UNIQUE(module_id, depends_on_id)
);

-- Conversation messages
CREATE TABLE conversations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  role        VARCHAR(20) NOT NULL, -- user, assistant, system
  content     TEXT NOT NULL,
  message_type VARCHAR(50) NOT NULL DEFAULT 'chat', -- chat, plan, approval,
  refinement, error
  metadata    JSONB DEFAULT '{}', -- agent name, tool calls, costs,
  etc.
  created_at   TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_conversations_project ON conversations(project_id, created_at);

-- Generation/refinement jobs (tracks async agent runs)
CREATE TABLE agent_jobs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  agent_type  VARCHAR(50) NOT NULL, -- planner, code_generator,
  refiner, etc.
  status      VARCHAR(50) NOT NULL DEFAULT 'running', -- running, completed, failed,
  cancelled
  instruction TEXT,
  result      JSONB, -- {num_turns, cost_usd,
  duration_ms, files_changed}
  started_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  completed_at TIMESTAMP,
  error_message TEXT
);

-- Version history (git commits in the project)
CREATE TABLE versions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  commit_hash VARCHAR(40) NOT NULL,
  message     TEXT NOT NULL,
  agent_job_id UUID REFERENCES agent_jobs(id), -- which agent run created this
  version
  files_changed JSONB DEFAULT '[]', -- list of changed file paths
  created_at   TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

-- =====
-- TEMPLATE & DISCOVERY TABLES
-- =====

-- App templates (pre-built plans for common app types)
CREATE TABLE app_templates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        VARCHAR(100) NOT NULL UNIQUE,          -- "crm-basic", "hr-onboarding"
  name        VARCHAR(255) NOT NULL,
  description  TEXT NOT NULL,
  category    VARCHAR(100) NOT NULL,                  -- Operations, Sales, HR,
  subcategory VARCHAR(100),                            -- optional finer grouping
  icon        VARCHAR(50),                               -- lucide icon name
  tags       JSONB DEFAULT '[]',                       -- ["crm", "sales", "pipeline"]
  plan       JSONB NOT NULL,                           -- full Planner-format plan
  (entities, pages, workflows, etc.)
  preview_image TEXT,                                -- URL or path to screenshot
  complexity   VARCHAR(20) NOT NULL DEFAULT 'medium', -- simple, medium, complex
  estimated_modules INTEGER NOT NULL DEFAULT 1,
  org_departments JSONB DEFAULT '[]',                 -- departments this template is
  relevant to
  is_published BOOLEAN NOT NULL DEFAULT TRUE,
  created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_templates_category ON app_templates(category);

-- Discovery sessions (tracks multi-turn discovery conversations)
CREATE TABLE discovery_sessions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES platform_users(id),
  status      VARCHAR(50) NOT NULL DEFAULT 'active', -- active, brief_ready,
  converted, abandoned
  initial_input TEXT NOT NULL,                        -- user's first message
  discovery_type VARCHAR(50) NOT NULL,                 -- problem_first,
  reference_based, department_need, vague_idea
  brief        JSONB,                                  -- structured brief when ready
  converted_to  UUID REFERENCES projects(id),           -- project created from this
  session
  messages     JSONB DEFAULT '[]',                    -- conversation history
  created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_discovery_org ON discovery_sessions(org_id, status);

```

4.2 SQLAlchemy Models

```

# backend/models/project.py

from sqlalchemy import Column, String, Text, Integer, DateTime, ForeignKey
from sqlalchemy.dialects.postgresql import UUID, JSONB
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
import uuid

from database import Base

class Project(Base):

```

```
__tablename__ = "projects"

id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
short_id = Column(String(8), unique=True, nullable=False)
org_id = Column(UUID(as_uuid=True), ForeignKey("organizations.id"), nullable=False)
name = Column(String(255), nullable=False)
description = Column(Text)
owner_id = Column(UUID(as_uuid=True), ForeignKey("platform_users.id"), nullable=False)
status = Column(String(50), nullable=False, default="active")
output_dir = Column(Text, nullable=False)
preview_port = Column(Integer)
db_port = Column(Integer)
created_at = Column(DateTime, nullable=False, server_default=func.now())
updated_at = Column(DateTime, nullable=False, server_default=func.now(),
onupdate=func.now())

organization = relationship("Organization", back_populates="projects")
modules = relationship("Module", back_populates="project", cascade="all, delete-orphan")
conversations = relationship("Conversation", back_populates="project", cascade="all, delete-orphan")
versions = relationship("Version", back_populates="project", cascade="all, delete-orphan")
```

5. Agent System

5.1 Agent Overview

Agent #	Name	Model	Avg Turns	When Called
0	Orchestrator	Haiku 4.5	1-2	Every user input
1	Planner	Sonnet 4	5-15	New app/module
2	Refiner	Sonnet 4	3-8	Change requests
3	Explainer	Haiku 4.5	2-5	Questions about app
4	Code Generator	Sonnet 4	30-80	Initial generation
5	Code Editor	Sonnet 4	5-15	Targeted file edits
6	Scaffolder	Sonnet 4	15-40	New feature addition
7	Indexer	Haiku 4.5	3-8	After every code change
8	Validator	Haiku 4.5	2-5	After every code change
9	Reviewer	Sonnet 4	5-10	After generation (optional)
10	Workflow Generator	Sonnet 4	5-15	Workflow action functions
11	Agent Builder	Sonnet 4	10-30	AI agent creation/editing
12	Discovery	Sonnet 4	5-20	Users without clear requirements

5.2 Agent #0: Orchestrator

```
# backend/agents/orchestrator.py

ORCHESTRATOR_SYSTEM_PROMPT = """You are a routing agent for an application builder platform.
Your job is to classify the user's intent and route to the correct specialist agent.

You have access to the current project's AppModel index which tells you what exists.

## Classification Rules

Respond with EXACTLY ONE of these categories:
```

PLAN – User wants to create a new app or add a major new module/feature
 Examples: "Build me a task app", "Add an inventory module",
 "I need user authentication", "Create a reporting dashboard"

REFINE – User wants to change something that already exists
 Examples: "Make the button red", "Add a search bar to the task list",
 "Change the card layout to a table", "Add validation to the email field"

EXPLAIN – User is asking a question, not requesting a change
 Examples: "How does the auth work?", "What tables exist?",
 "Why is the sidebar component structured this way?", "Show me the task flow"

SCAFFOLD – User wants to add a specific new feature to an existing module
 Examples: "Add a notifications system", "Add CSV export to the task list",
 "Add a dark mode toggle", "Add pagination to the API"
 (Distinguish from PLAN: scaffold is a focused feature, plan is a whole module)

AGENT – User wants to create, modify, or manage an AI agent within the app
 Examples: "Add a customer support chatbot", "Create an AI assistant that can query orders",
 "Add a tool to the support agent", "Change the agent's personality",
 "Add a second agent for inventory questions"

DISCOVER – User has a vague idea, problem, or reference but no clear requirements
 Examples: "I need something for my HR team", "We have a problem with tracking expenses",
 "Build something like Trello", "I'm not sure what I need but our onboarding is broken",
 "What apps would help my sales department?"

NAVIGATE – User wants to switch views or see something specific
 Examples: "Show me the data model", "Open the workflow editor",
 "Go to the preview", "Open the agent builder"

UNDO – User wants to revert a change
 Examples: "Undo that", "Revert the last change", "Go back to before"

AMBIGUOUS – You can't determine intent. Ask a clarifying question.

Response Format

```
{
  "intent": "PLAN|REFINE|EXPLAIN|SCAFFOLD|AGENT|DISCOVER|NAVIGATE|UNDO|AMBIGUOUS",
  "reasoning": "brief explanation",
  "clarification": "question to ask if AMBIGUOUS, null otherwise",
  "context_needed": ["list of AppModel sections the downstream agent will need"]
}
```

```
ORCHESTRATOR_OPTIONS = ClaudeAgentOptions(
    system_prompt=ORCHESTRATOR_SYSTEM_PROMPT,
    allowed_tools=["Read"], # Can read app-model.json
    permission_mode="bypassPermissions",
    max_turns=2,
    model="claude-haiku-4-5-20251001",
)
```

5.3 Agent #1: Planner

```
# backend/agents/planner.py
```

```
PLANNER_SYSTEM_PROMPT = """You are an application architect planning the structure of a new application or module. You have a multi-turn conversation with the user to clarify requirements,
```

then produce a detailed structured plan.

IMPORTANT: You have access to the organization's structure (departments, teams, roles, people, reporting lines). Use this to automatically infer access control, workflow assignments, and role-based permissions. Do NOT ask the user "what roles do you need?" – infer them from the org.

Your Process

1. **UNDERSTAND:** Read the user's request. If building onto an existing app, read the AppModel index to understand what already exists.
2. **CLARIFY:** Ask 2–4 focused questions about:
 - Core entities and their relationships
 - Key user workflows (what does the user DO in this app?)
 - Authentication/authorization needs
 - Any integrations (email, payments, file upload, external APIs)
 - Mobile vs web vs both
 - AI/intelligence requirements (auto-categorization, search, content generation)
 Do NOT ask more than 4 questions. Make reasonable assumptions for anything else.

IMPORTANT: If the user's description implies AI capabilities (e.g., "auto-categorize", "smart search", "generate descriptions", "detect sentiment", "find similar"), automatically include the appropriate smart fields, semantic search, or AI workflow nodes in the plan. You don't need to ask – just include them.

3. **PLAN:** Once requirements are clear, produce a structured plan:

Plan Output Format (JSON)

```
{
  "module_name": "Inventory Management",
  "description": "Track products, warehouses, stock levels, and purchase orders",

  "data_models": [
    {
      "name": "Product",
      "fields": [
        {"name": "id", "type": "uuid", "constraints": ["pk", "auto"]},
        {"name": "sku", "type": "varchar(100)", "constraints": ["unique", "not_null"]},
        {"name": "name", "type": "varchar(255)", "constraints": ["not_null"]},
        {"name": "description", "type": "text", "constraints": []},
        {"name": "category", "type": "varchar(100)", "constraints": []},
        {"name": "unitCost", "type": "decimal(10,2)", "constraints": ["not_null"]},
        {"name": "unitPrice", "type": "decimal(10,2)", "constraints": ["not_null"]},
        {"name": "reorderPoint", "type": "integer", "constraints": ["default:10"]},
        {"name": "isActive", "type": "boolean", "constraints": ["default:true"]},
        {"name": "createdAt", "type": "timestamp", "constraints": ["not_null", "auto"]},
        {"name": "updatedAt", "type": "timestamp", "constraints": ["not_null", "auto"]}
      ],
      "indexes": ["idx_product_sku (sku)", "idx_product_category (category)"]
    }
    // ... more models
  ],

  "relations": [
    {"from": "StockLevel", "to": "Product", "type": "many-to-one", "field": "productId"},
    {"from": "StockLevel", "to": "Warehouse", "type": "many-to-one", "field":
"warehouseId"},
    {"from": "PurchaseOrderLine", "to": "PurchaseOrder", "type": "many-to-one", "field":
"orderId"},
    {"from": "PurchaseOrderLine", "to": "Product", "type": "many-to-one", "field":
"productId"}
  ]
}
```

```

],

"pages": [
  {
    "route": "/inventory/products",
    "name": "Product List",
    "description": "Searchable, filterable table of all products with stock indicators",
    "components": ["ProductTable", "ProductFilters", "AddProductButton"]
  },
  {
    "route": "/inventory/products/[id]",
    "name": "Product Detail",
    "description": "Product info, stock levels across warehouses, order history",
    "components": ["ProductInfo", "StockLevelTable", "OrderHistory"]
  }
  // ... more pages
],

"api_routes": [
  {"method": "GET", "path": "/api/products", "description": "List products with search/filter/pagination"},
  {"method": "POST", "path": "/api/products", "description": "Create product"},
  {"method": "GET", "path": "/api/products/:id", "description": "Get product detail with stock levels"},
  {"method": "PUT", "path": "/api/products/:id", "description": "Update product"},
  {"method": "DELETE", "path": "/api/products/:id", "description": "Soft delete product (set isActive=false)"}
  // ... more routes
],

"workflows": [
  {
    "name": "on-low-stock",
    "trigger": "StockLevel.available drops below Product.reorderPoint",
    "steps": ["Create draft PurchaseOrder", "Notify purchasing team"]
  },
  {
    "name": "on-goods-received",
    "trigger": "GoodsReceipt is completed",
    "steps": ["Update StockLevel quantities", "Update PO status to received"]
  }
],

"rules": [
  {"type": "validation", "model": "Product", "field": "unitPrice", "condition": "value > 0", "message": "Price must be positive"},
  {"type": "validation", "model": "Product", "field": "sku", "condition": "unique", "message": "SKU must be unique"},
  {"type": "computed", "model": "StockLevel", "field": "available", "expression": "quantity - reserved"},
  {"type": "state_machine", "model": "PurchaseOrder", "field": "status", "states": ["draft", "submitted", "approved", "received", "cancelled"]}
],

"seed_data": {
  "description": "Generate 10 sample products, 3 warehouses, stock levels, and 2 sample POs",
  "notes": "Use realistic product names and SKUs for a general supplies company"
},

"access_control": {
  "app_roles": [
    {"name": "inventory_admin", "maps_from_org_role": "manager", "description": "Full CRUD on all inventory data"},

```

```

    {"name": "warehouse_staff", "maps_from_org_role": "staff", "maps_from_dept":
"warehouse", "description": "Can update stock levels"},
    {"name": "inventory_viewer", "maps_from_org_role": "staff", "description": "Read-only
access"}
  ],
  "field_access": [
    {"model": "Product", "field": "unitCost", "view": ["finance_dept", "manager+"],
"edit": ["inventory_admin"]},
    {"model": "Product", "field": "margin", "view": ["finance_dept", "director+"],
"edit": []},
    {"model": "Product", "field": "supplierNotes", "view": ["procurement_team", "admin"],
"edit": ["procurement_team"]}
  ],
  "record_scope": [
    {"model": "Product", "scope": "department", "column": "department_id",
"rules": "staff sees own dept, manager sees own + sub-depts, finance/admin sees
all"}
  ],
  "workflow_assignments": [
    {"workflow": "on-low-stock", "node": "approve-po", "assign_to": "requester's
manager",
"sla_hours": 24, "escalate_to": "director"},
    {"workflow": "on-goods-received", "node": "verify-receipt", "assign_to":
"warehouse_staff",
"assign_mode": "round_robin"}
  ],
  "ai_features": {
    "smart_fields": [
      // Auto-populated when user requirements imply AI
      {"model": "Product", "field": "description", "type": "ai_generate",
"source": ["name", "specs", "features"], "trigger": "manual",
"prompt": "Generate marketing product description from specs"}
    ],
    "semantic_search": [
      {"model": "Product", "fields": ["name", "description"], "trigger":
"on_create_update"}
    ],
    "ai_workflow_nodes": [],
    "ai_rules": [],
    "scheduled_ai": [],
    "ai_components": [
      {"type": "SmartFilterBar", "page": "/inventory/products"},
      {"type": "DataInsightsPanel", "page": "/dashboard"}
    ]
  },
  "cross_module_dependencies": [
    // Only if this module depends on existing modules
    {"module": "finance", "reason": "Goods receipt creates AP journal entry"}
  ]
}

```

4. CONFIRM: Present the plan to the user for approval. If they want changes, adjust and re-present.

Important Rules

- Keep plans scoped. A single module should have 3–8 data models, 4–12 pages, and 2–6 workflows.
- If the user's request is too large for one module, split it into multiple modules and explain why.
- Always include seed data so the app works immediately on preview.
- Consider the existing app's data models when planning relations and cross-module


```

integrations.
- Use PostgreSQL-native types (uuid, varchar, timestamp, jsonb, etc.).
"""

PLANNER_OPTIONS = ClaudeAgentOptions(
    system_prompt=PLANNER_SYSTEM_PROMPT,
    allowed_tools=["Read", "Glob"],
    permission_mode="bypassPermissions",
    max_turns=15,
    model="claude-sonnet-4-20250514",
)

```

5.4 Agent #2: Refiner

```

# backend/agents/refiner.py

REFINER_SYSTEM_PROMPT = """You are a developer applying changes to an existing Next.js +
Tailwind CSS + PostgreSQL (Drizzle ORM) application based on user instructions.

## Your Process

1. Read the AppModel index (app-model.json) to understand what exists and where files are.
2. Identify which files need to change based on the instruction.
3. Read only the affected files.
4. Make targeted edits using the Edit tool (NOT Write – preserve existing code).
5. If adding new files, use Write.
6. If the change affects the database schema, also update src/infrastructure/db/schema.ts
and run migrations.
7. After all edits, verify the build: npm run build
8. If the build fails, fix the error.

## Rules
- Make ONLY the changes described in the instruction.
- Use Edit for modifications, Write only for new files.
- Preserve existing code structure and patterns.
- Preserve the Clean Architecture layer separation. When adding new logic, place it in the
correct layer:
  - Domain types/rules → `src/domain/{module}/`
  - Service logic → `src/application/{module}/`
  - DB queries → `src/infrastructure/db/repositories/`
  - Auth/RBAC → `src/infrastructure/auth/`
  - AI features → `src/infrastructure/ai/`
  - Route handlers → `src/app/api/` (keep thin – delegate to services)
  - Components → `src/components/` (presentation only)
- Never move business logic into route handlers or components.
- Never add direct DB queries to route handlers – always go through services and
repositories.
- If the instruction came from a visual editor, it will be very specific – follow it
exactly.
- If the instruction came from chat, it may be vague – use your judgment for the best
implementation.
- When changing data models, update: schema.ts, repository, service, API routes,
components, and seed data.
- Use Tailwind CSS classes matching the project's existing style.
- Do NOT add new dependencies without explicit instruction.
- Do NOT refactor or reorganize code beyond what's requested.

## Database Changes
When modifying src/infrastructure/db/schema.ts:
- Use Drizzle ORM pgTable, pgEnum syntax
- Add proper constraints (notNull, unique, default, references)
- Add indexes for frequently queried columns

```

```
- After schema changes, the preview system will run drizzle-kit push automatically
""""
```

```
REFINER_OPTIONS = ClaudeAgentOptions(
    system_prompt=REFINER_SYSTEM_PROMPT,
    allowed_tools=["Write", "Edit", "Read", "Bash", "Glob"],
    permission_mode="bypassPermissions",
    max_turns=30,
    model="claude-sonnet-4-20250514",
)
```

5.5 Agent #4: Code Generator

```
# backend/agents/code_generator.py
```

```
CODE_GENERATOR_SYSTEM_PROMPT = """You are a senior full-stack developer generating a
complete
Next.js + Tailwind CSS + PostgreSQL application from a structured plan.
```

```
## Tech Stack (ALWAYS use these exactly)
```

```
- Next.js 15 (App Router)
- TypeScript 5.7+
- Tailwind CSS 4 (with @import "tailwindcss" in globals.css)
- PostgreSQL 16 via Docker Compose
- Drizzle ORM with pg driver
- @tailwindcss/postcss for PostCSS config
```

```
## Clean Architecture – EVERY generated app follows this structure
```

The app is organized into strict layers. Each layer has a clear responsibility.
 Import direction: domain ← application ← infrastructure, api → application, components → types.

```
### Directory Placement Rules
```

Code type	Goes in	Example
TypeScript types, domain constants	<code>`src/domain/{module}/entities.ts`</code>	<code>`Product`, `OrderStatus`</code>
Pure business rule functions	<code>`src/domain/{module}/rules.ts`</code>	<code>`canApproveExpense(user, expense)`</code>
Domain error classes	<code>`src/domain/{module}/errors.ts`</code>	<code>`InsufficientStockError`</code>
Zod input/output schemas	<code>`src/application/{module}/{resource}.schema.ts`</code>	<code>`createProductSchema`</code>
Use case / service methods	<code>`src/application/{module}/{resource}.service.ts`</code>	<code>`ProductService.create()`</code>
Drizzle table definitions	<code>`src/infrastructure/db/schema.ts`</code>	<code>`pgTable('products', ...)`</code>
DB connection pool	<code>`src/infrastructure/db/connection.ts`</code>	<code>`export const db = drizzle(pool)`</code>
DB queries (repository)	<code>`src/infrastructure/db/repositories/{resource}.repository.ts`</code>	<code>`ProductRepository.findById()`</code>
Auth middleware	<code>`src/infrastructure/auth/middleware.ts`</code>	<code>`requireAuth()`</code>
RBAC enforcement	<code>`src/infrastructure/auth/rbac.ts`</code>	<code>`applyScopeFilter()`</code>
Org context resolver	<code>`src/infrastructure/auth/org-context.ts`</code>	<code>`resolveOrgContext()`</code>
AI config, smart fields, search	<code>`src/infrastructure/ai/`</code>	<code>`config.ts`, `smart-fields.ts`, `semantic-search.ts`, `usage.ts`</code>
Real-time socket server	<code>`src/infrastructure/realtime/socket-server.ts`</code>	<code>`initSocketServer()`</code>
Real-time React hook	<code>`src/infrastructure/realtime/use-realtime.ts`</code>	<code>`useRealtime(channel, event, cb)`</code>
Email sending	<code>`src/infrastructure/email/email.service.ts`</code>	<code>`sendEmail()`</code>

```
| External API clients | `src/infrastructure/external/{api}.client.ts` | `StripeClient` |
| Seed data | `src/infrastructure/db/seed.ts` | Idempotent seed script |
| API route handlers | `src/app/api/{resource}/route.ts` | Thin – max 15–20 lines |
| Page components | `src/app/{module}/{resource}/page.tsx` | Server Components |
| React UI components | `src/components/{module}/{Resource}Form.tsx` | Presentation only |
| Shared types | `src/types/index.ts` | Re-exports from domain entities |
```

Import Direction Rules (NEVER violate)

- `domain/` imports NOTHING (pure TypeScript, no npm packages except type-only)
- `application/` imports from `domain/` and `infrastructure/`
- `infrastructure/` imports from `domain/` only
- `app/` (routes) imports from `application/` only (never touch DB directly)
- `components/` imports from `types/` only (no business logic)

Anti-Patterns to AVOID

- NO DB queries in route handlers (use services)
- NO business logic in components (just render props)
- NO framework imports in domain layer (no Drizzle, no Next.js, no Zod)
- NO direct DB calls from app/ routes (always go through service → repository)
- NO `src/lib/` grab-bag folder (everything has a proper layer)

Required Files (generate ALL of these)

Project Config

1. package.json – dependencies: next, react, react-dom, drizzle-orm, pg, zod, tailwindcss, @tailwindcss/postcss
optional deps (when real-time needed): socket.io, socket.io-client
devDeps: drizzle-kit, tsx, @types/pg, typescript, @types/react, @types/react-dom
scripts: dev, build, start, db:up, db:push, db:seed, db:reset, db:studio
2. docker-compose.yml – PostgreSQL 16 service with health check, using DB_PORT env var
3. .env – DATABASE_URL=postgresql://app:app@localhost:\${DB_PORT:-5432}/app
4. .env.example – same without real values
5. .gitignore – node_modules, .next, .env, data/
6. tsconfig.json – standard Next.js config with path aliases (@/ → src/)
7. next.config.ts – standard Next.js config
8. postcss.config.mjs – {"plugins": {"@tailwindcss/postcss": {}}}
9. drizzle.config.ts – PostgreSQL dialect, schema path, DATABASE_URL

Domain Layer

10. src/domain/{module}/entities.ts – TypeScript types + domain constants per module
11. src/domain/{module}/rules.ts – Pure business rule functions (no imports from infra)
12. src/domain/{module}/errors.ts – Domain-specific error classes

Application Layer

13. src/application/{module}/{resource}.service.ts – Service with CRUD + business operations
14. src/application/{module}/{resource}.schema.ts – Zod schemas for input validation

Infrastructure Layer

15. src/infrastructure/db/schema.ts – ALL tables using pgTable, pgEnum, with proper types:
 - uuid for IDs (defaultRandom)
 - varchar for short strings (with length)
 - text for long strings
 - timestamp for dates (defaultNow where appropriate)
 - pgEnum for enums
 - Foreign keys with references() and onDelete behavior
 - Indexes for searchable/filterable columns
16. src/infrastructure/db/connection.ts – Pool connection, drizzle instance, export db
17. src/infrastructure/db/seed.ts – Realistic sample data, check before inserting (idempotent)
Run with: npx tsx src/infrastructure/db/seed.ts
18. src/infrastructure/db/repositories/{resource}.repository.ts – DB queries per resource
 - Repositories return domain types, not Drizzle row types
 - One repository per entity

```

19. src/infrastructure/auth/middleware.ts – Auth middleware
20. src/infrastructure/auth/rbac.ts – Field/record access enforcement
21. src/infrastructure/auth/org-context.ts – Org structure resolver

### API Layer (thin handlers)
22. src/app/globals.css – @import "tailwindcss"; + any custom styles
23. src/app/layout.tsx – Root layout with Google Fonts, metadata
24. src/app/page.tsx – Dashboard/home page
25. src/app/{module}/{resource}/page.tsx – List pages (Server Components)
26. src/app/api/{resource}/route.ts – CRUD API routes per resource
    Each handler: parse request → validate with Zod → call service → return response

### Components (presentation only)
27. src/components/layout/ – Sidebar, Header, PageWrapper
28. src/components/shared/ – DataTable, FormField, Badge, etc.
29. src/components/{module}/ – Module-specific components ({Resource}Form, {Resource}Table,
etc.)

### Shared Types
30. src/types/index.ts – Re-exports domain entity types for use by components and routes

## Server-Side Rendering Strategy
– Server Components by default for ALL pages – data fetched at the server, zero client JS
– List pages, detail pages, dashboards, layouts, navigation: ALWAYS Server Components
– Server Components can call services directly (no API roundtrip needed for reads)
– Use Client Components ("use client") ONLY for: forms with state, real-time UI, DataTable
  with client-side sorting/filtering, components using useState/useEffect/event handlers
– API routes for ALL mutations (create, update, delete) – not Server Actions
– Server Actions may wrap service calls as thin pass-throughs when convenient, but API
  routes remain the primary mutation pattern for portability and testability

## Real-Time (when needed)
– Only add Socket.IO when the app has features that need it (agents, workflows with
  assignments, smart fields, collaborative editing, live notifications)
– Socket server: `src/infrastructure/realtime/socket-server.ts` – attaches to Next.js
  custom server, handles room-based broadcasting
– Event types: `src/infrastructure/realtime/events.ts` – typed event definitions
– React hook: `src/infrastructure/realtime/use-realtime.ts` – `useRealtime(channel, event,
cb)`
– Room scoping: `org:{orgId}`, `user:{userId}`, `record:{model}:{id}`
– Keep SSE for agent chat streaming (simpler, one-directional)
– Use Socket.IO for: smart field completion push, workflow task assignments,
  live notifications, data refresh after mutations by other users

## Code Quality Rules
– Every API route must validate input with Zod schemas before calling services
– Every API route must handle errors with try/catch and return proper HTTP status codes
– Every API route must be thin: max 15–20 lines (parse → validate → call service → respond)
– Every list endpoint must support basic pagination (page, limit query params)
– Every component must have TypeScript props interface
– Use "use client" directive only on components with interactivity (state, effects,
handlers)
– Server Components by default, Client Components only when needed
– Use exact Tailwind arbitrary values for pixel-perfect styling
– Format all files consistently (2-space indent, single quotes, trailing commas)
– Services are the ONLY place that coordinates DB calls + business rules
– Each service method = one use case (e.g., createProduct, approveExpense)

## CRITICAL: After generating all files, run:
1. npm install
2. npm run db:up (starts Docker Postgres)
3. npx drizzle-kit push (applies schema)
4. npx tsx src/infrastructure/db/seed.ts (seeds data)
5. npm run build (verifies compilation)

```

If any step fails, fix the error and retry.

Navigation & Layout

- Generate a sidebar or navbar with links to all pages
- Use a consistent layout with header, sidebar, and content area
- Active page should be highlighted in navigation
- Use Next.js Link component for client-side navigation

"""

```
CODE_GENERATOR_OPTIONS = ClaudeAgentOptions(
    system_prompt=CODE_GENERATOR_SYSTEM_PROMPT,
    allowed_tools=["Write", "Edit", "Read", "Bash", "Glob"],
    permission_mode="bypassPermissions",
    max_turns=80,
    model="claude-sonnet-4-20250514",
)
```

5.6 Agent #5: Code Editor

backend/agents/code_editor.py

```
CODE_EDITOR_SYSTEM_PROMPT = """You are a focused code editor. You receive a specific file
and a specific instruction. Make the minimum change needed.
```

Rules:

- Use Edit tool, not Write (preserve existing code)
- Change ONLY what the instruction says
- Do not refactor, reorganize, or "improve" surrounding code
- Do not add comments unless the instruction says to
- Preserve existing formatting and style
- After editing, verify the change makes sense in context

If the file doesn't exist yet, use Write to create it.

If the change requires imports, add them.

If the change would break TypeScript types, fix the type errors too.

"""

```
CODE_EDITOR_OPTIONS = ClaudeAgentOptions(
    system_prompt=CODE_EDITOR_SYSTEM_PROMPT,
    allowed_tools=["Write", "Edit", "Read", "Bash"],
    permission_mode="bypassPermissions",
    max_turns=15,
    model="claude-sonnet-4-20250514",
)
```

5.7 Agent #7: Indexer

backend/agents/indexer.py

```
INDEXER_SYSTEM_PROMPT = """You maintain the AppModel index (app-model.json) for a generated
application. After code changes, you scan the affected files and update the index.
```

What You Do

1. Read the current app-model.json
2. Read the list of changed files provided in the instruction
3. Scan each changed file to extract:
 - Components: name, file path, props, description
 - Pages: route, file path, description, components used
 - Data models: name, fields with types, file path

```

- API routes: method, path, file path, description
- Workflows: name, trigger, file path
- Rules: type, attached model/field, condition
- Bindings: which components are bound to which models
4. Update app-model.json with the new information
5. Remove entries for deleted files

## Output
Write the updated app-model.json using the Write tool.
The format must match the AppModel schema exactly (see below).

## Important
- Do NOT hallucinate entries. Only index what you can verify by reading the files.
- If a file was deleted, remove its entries from the index.
- If a file was modified, re-read it and update its entries.
- Preserve entries for files that weren't changed.
"""

INDEXER_OPTIONS = ClaudeAgentOptions(
    system_prompt=INDEXER_SYSTEM_PROMPT,
    allowed_tools=["Read", "Write", "Glob"],
    permission_mode="bypassPermissions",
    max_turns=10,
    model="claude-haiku-4-5-20251001",
)

```

5.8 Agent #8: Validator

```

# backend/agents/validator.py

VALIDATOR_SYSTEM_PROMPT = """You verify that a generated/modified Next.js application
compiles
and runs correctly.

## Steps
1. Run: npm run build
2. If it fails:
    a. Read the error output
    b. Identify the root cause
    c. Make a targeted fix using Edit
    d. Re-run: npm run build
    e. Repeat up to 3 times
3. If it passes, return success

## Common Fixes
- Missing imports: add the import statement
- Type errors: fix the type annotation or add type assertion
- Undefined variables: check if the variable was renamed or removed
- Module not found: check if the file path is correct

## Rules
- Only fix build errors. Do not refactor or improve code.
- If you can't fix the error after 3 attempts, report the error details.
- Do not install new packages.
"""

VALIDATOR_OPTIONS = ClaudeAgentOptions(
    system_prompt=VALIDATOR_SYSTEM_PROMPT,
    allowed_tools=["Edit", "Read", "Bash"],
    permission_mode="bypassPermissions",
    max_turns=10,
)

```

```

    model="claude-haiku-4-5-20251001",
)

```

5.9 Agent Orchestration Flow

```

# backend/agents/__init__.py - main orchestration logic

async def handle_user_input(
    project_id: str,
    user_message: str,
    output_dir: str,
    source: str = "chat", # "chat" | "visual_editor"
) -> AsyncIterator[dict]:
    """Main entry point for all user inputs - chat messages and visual editor actions."""

    # Step 1: Classify intent (skip for visual editor - always REFINE)
    if source == "visual_editor":
        intent = "REFINE"
    else:
        intent = await classify_intent(user_message, output_dir)

    # Step 2: Route to appropriate agent
    if intent == "PLAN":
        async for event in run_planner(output_dir, user_message):
            yield event
        # After plan approval, run code generator
        # ... (see generation flow below)

    elif intent == "REFINE":
        yield sse_event("refine_start", {"message": f"Applying: {user_message}"})

        # Run refiner
        files_changed = []
        async for event in run_refiner(output_dir, user_message):
            yield event
            if event.get("event") == "file_created":
                files_changed.append(event["data"]["path"])

        # Validate build
        yield sse_event("status", {"message": "Verifying build..."})
        build_ok = await run_validator(output_dir)
        if not build_ok:
            yield sse_event("error", {"message": "Build failed after changes"})
            return

        # Update index
        yield sse_event("status", {"message": "Updating project index..."})
        await run_indexer(output_dir, files_changed)

        # Push schema changes if any db files changed
        if any("schema" in f or "db/" in f for f in files_changed):
            yield sse_event("status", {"message": "Applying database changes..."})
            await push_schema(output_dir)

        yield sse_event("refine_complete", {"message": "Done"})

    elif intent == "EXPLAIN":
        async for event in run_explainer(output_dir, user_message):
            yield event

    elif intent == "SCAFFOLD":
        async for event in run_scaffolder(output_dir, user_message):

```

```

        yield event

    elif intent == "AGENT":
        yield sse_event("agent_build_start", {"message": f"Building agent:
{user_message}"})
        files_changed = []
        async for event in run_agent_builder(output_dir, user_message):
            yield event
            if event.get("event") == "file_created":
                files_changed.append(event["data"]["path"])
        # Validate + Index
        build_ok = await run_validator(output_dir)
        await run_indexer(output_dir, files_changed)
        yield sse_event("agent_build_complete", {"message": "Agent ready"})

    elif intent == "DISCOVER":
        yield sse_event("discover_start", {"message": "Let me help you figure out what you
need..."})
        async for event in run_discovery_agent(output_dir, user_message, org_id):
            yield event
            # When discovery produces a brief, hand off to Planner
            if event.get("event") == "discover_brief_ready":
                brief = event["data"]["brief"]
                async for plan_event in run_planner(output_dir, brief):
                    yield plan_event

    elif intent == "UNDO":
        await git_revert_last(output_dir)
        yield sse_event("status", {"message": "Reverted last change"})

```

6. Backend API

6.1 All Endpoints

Organizations

POST	/api/orgs	Create organization
GET	/api/orgs	List user's organizations
GET	/api/orgs/:orgId	Get org details
PUT	/api/orgs/:orgId	Update org settings
POST	/api/orgs/:orgId/invite	Invite platform user to org

Org Structure

GET	/api/orgs/:orgId/people	List all people in org
POST	/api/orgs/:orgId/people	Add person manually
PUT	/api/orgs/:orgId/people/:id	Update person details
DELETE	/api/orgs/:orgId/people/:id	Deactivate person
POST	/api/orgs/:orgId/people/import	Bulk import (CSV/JSON upload)
GET	/api/orgs/:orgId/departments	List departments
POST	/api/orgs/:orgId/departments	Create department
PUT	/api/orgs/:orgId/departments/:id	Update department (rename, move)
DELETE	/api/orgs/:orgId/departments/:id	Delete department (reassign people)
GET	/api/orgs/:orgId/teams	List teams
POST	/api/orgs/:orgId/teams	Create team
GET	/api/orgs/:orgId/org-chart	Get full org tree (for visual editor)
PUT	/api/orgs/:orgId/org-chart	Update org tree (from visual editor drag)
GET	/api/orgs/:orgId/roles	List org roles
POST	/api/orgs/:orgId/roles	Create custom role
PUT	/api/orgs/:orgId/roles/:id	Update role
DELETE	/api/orgs/:orgId/roles/:id	Delete role (reassign people)
GET	/api/orgs/:orgId/groups	List groups
POST	/api/orgs/:orgId/groups	Create group

PUT /api/orgs/:orgId/groups/:id/members Add/remove group members

RBAC Policies

GET /api/projects/:id/access-policies List app access policies
 POST /api/projects/:id/access-policies Add app access policy
 DELETE /api/projects/:id/access-policies/:pid Remove app access policy
 GET /api/projects/:id/field-access List field-level access rules
 POST /api/projects/:id/field-access Add field access rule
 PUT /api/projects/:id/field-access/:pid Update field access rule
 DELETE /api/projects/:id/field-access/:pid Remove field access rule
 GET /api/projects/:id/field-access/matrix Get access matrix (model × role grid)
 GET /api/projects/:id/workflow-assignments List workflow assignments
 POST /api/projects/:id/workflow-assignments Add workflow assignment
 PUT /api/projects/:id/workflow-assignments/:pid Update workflow assignment

Projects (scoped to org)

POST /api/orgs/:orgId/projects Create new project in org
 GET /api/orgs/:orgId/projects List org's projects
 GET /api/projects/:id Get project details
 PUT /api/projects/:id Update project name/description
 DELETE /api/projects/:id Archive project

Generation & Chat

POST /api/projects/:id/chat Send message (SSE stream response)
 GET /api/projects/:id/conversations Get conversation history
 POST /api/projects/:id/generate Generate from plan (SSE stream)

Preview

POST /api/projects/:id/preview/start Start dev server + Postgres
 POST /api/projects/:id/preview/stop Stop dev server + Postgres
 GET /api/projects/:id/preview/status Get preview port/status

Files

GET /api/projects/:id/files List project files
 GET /api/projects/:id/file?path= Read file content
 PUT /api/projects/:id/file?path= Write file content (from code editor)
 GET /api/projects/:id/download Download project as ZIP

AppModel

GET /api/projects/:id/app-model Get full AppModel index
 GET /api/projects/:id/app-model/models Get just data models
 GET /api/projects/:id/app-model/pages Get just pages
 GET /api/projects/:id/app-model/rules Get just rules
 // etc. for each section

Modules

GET /api/projects/:id/modules List modules
 POST /api/projects/:id/modules Create module (triggers planning)
 GET /api/projects/:id/modules/:mid Get module details
 DELETE /api/projects/:id/modules/:mid Remove module

Visual Editor Actions

POST /api/projects/:id/visual/action Execute visual editor action (SSE stream)
 body: {
 "editor": "ui|data|workflow|rules|navigation|agent",
 "action": "add_field|change_style|add_node|create_agent|add_tool|...",
 "params": { ... },
 "instruction": "Generated text instruction"
 }

Agents (within generated apps – managed via visual editor)

GET /api/projects/:id/app-model/agents List agents in the app
 POST /api/projects/:id/agents/test Test agent with sample message
 body: { "agentId": "support-agent", "message": "Where is my order?", "userId": "test-

```

user" }

Database Browser
GET    /api/projects/:id/db/tables      List tables in generated app's DB
GET    /api/projects/:id/db/tables/:name Get table schema
GET    /api/projects/:id/db/query     Execute SELECT query
POST   /api/projects/:id/db/query     Execute INSERT/UPDATE/DELETE
POST   /api/projects/:id/db/seed      Re-run seed data
POST   /api/projects/:id/db/reset     Drop and recreate database

Version History
GET    /api/projects/:id/versions        List versions (git log)
POST   /api/projects/:id/versions/:hash/revert Revert to version

Templates
GET    /api/templates                    List all published templates
GET    /api/templates?category=Sales    Filter by category
GET    /api/templates?department=HR     Filter by relevant department
GET    /api/templates/:slug            Get template details + plan
GET    /api/orgs/:orgId/suggested-templates Templates relevant to org's departments
POST   /api/orgs/:orgId/projects/from-template Create project from template
      body: { "templateSlug": "crm-basic", "customizations": { "name": "Our CRM" } }

Discovery
POST   /api/orgs/:orgId/discovery/start Start discovery session (SSE stream)
      body: { "input": "I need something for my HR team" }
POST   /api/orgs/:orgId/discovery/:sid/message Continue discovery conversation (SSE stream)
      body: { "message": "Yes we need to track PTO and approvals" }
GET    /api/orgs/:orgId/discovery/:sid    Get discovery session details + brief
POST   /api/orgs/:orgId/discovery/:sid/convert Convert brief to project (triggers Planner)
GET    /api/orgs/:orgId/discovery         List active/recent discovery sessions

Auth (Platform)
POST   /api/auth/signup                  Create account
POST   /api/auth/login                  Login (returns JWT with orgId)
POST   /api/auth/logout                 Logout
GET    /api/auth/me                     Current user + org memberships
POST   /api/auth/switch-org             Switch active org context

```

6.2 SSE Event Types

All streaming endpoints use Server-Sent Events with these event types:

Generation events:

```

status      - General progress message: {message: string}
log         - Agent thinking/reasoning: {text: string}
tool_call   - Tool invocation: {tool: string, args: string}
file_created - File written: {path: string}
plan_ready  - Plan for user review: {plan: object}
plan_approved - User approved plan
complete    - Generation finished: {num_turns, cost_usd, duration_ms}

```

Refinement events:

```

refine_start - Refinement starting: {message: string}
refine_complete - Refinement done: {message, num_turns, cost_usd, duration_ms}

```

Agent builder events:

```

agent_build_start - Agent creation/edit starting: {message: string}
agent_build_complete - Agent creation/edit done: {message, agentId, num_turns, cost_usd}

```

```

Discovery events:
  discover_start      - Discovery conversation starting: {message: string}
  discover_question   - Agent asking clarifying question: {question: string, options?:
string[]}
  discover_suggestion - Agent suggesting a template or approach: {suggestion: object}
  discover_brief_ready - Structured brief ready for conversion: {brief: object}

Review events:
  review_start      - Visual review starting: {message, iteration}
  review_issues     - Issues found: {message, issues, iteration}
  fix_start         - Fixes being applied: {message, iteration}
  review_approved   - Design matches: {message, iteration}

Index events:
  index_updated     - AppModel index refreshed: {sections_changed: string[]}

Error:
  error             - Error occurred: {message: string}

```

7. Frontend Pages & Components

7.1 Page Structure

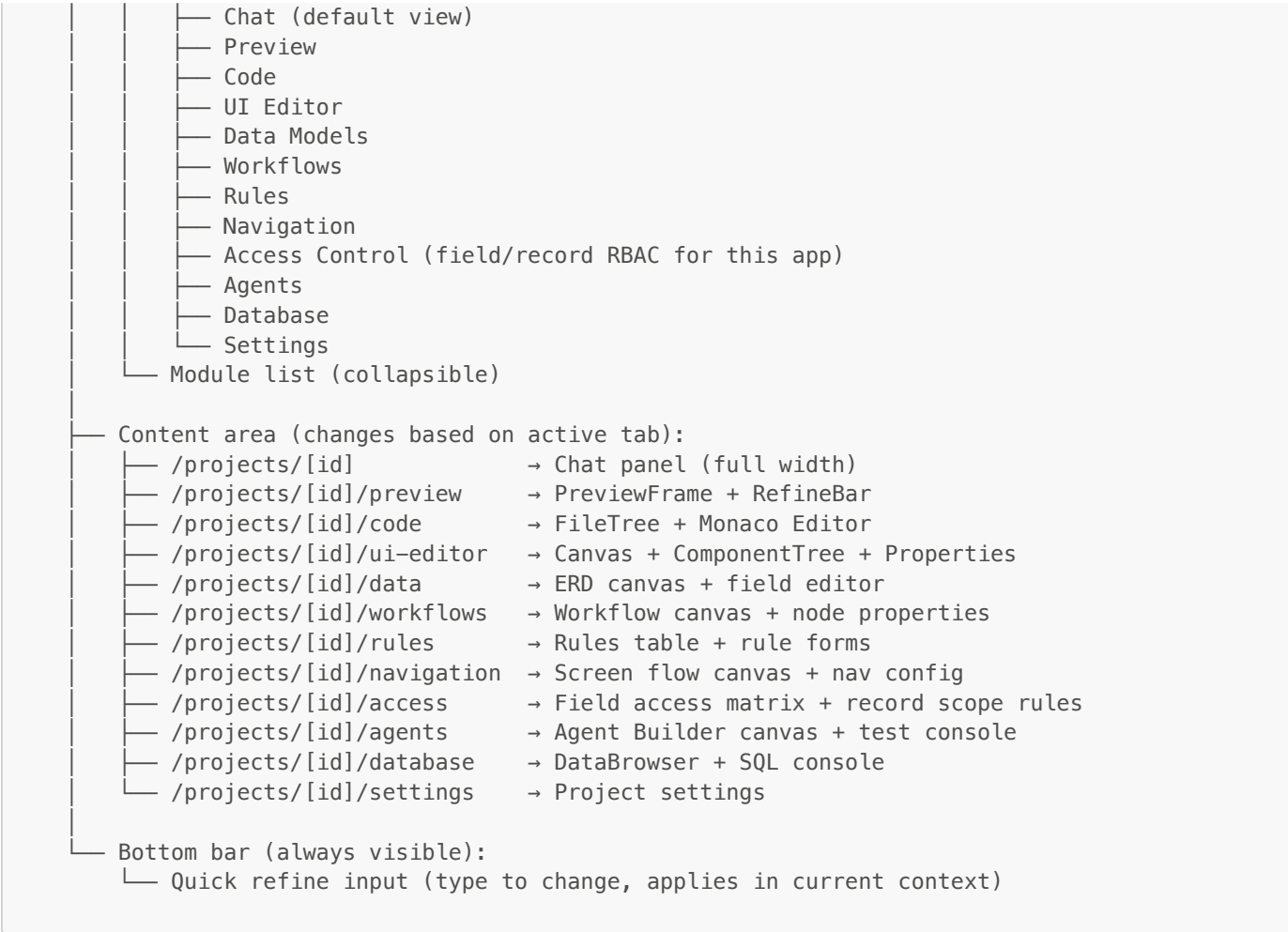
```

/ (landing - org selector)
├─ Org list cards (orgs the user belongs to)
├─ "Create Organization" button → org creation wizard
└─ Each card links to /orgs/[orgId]

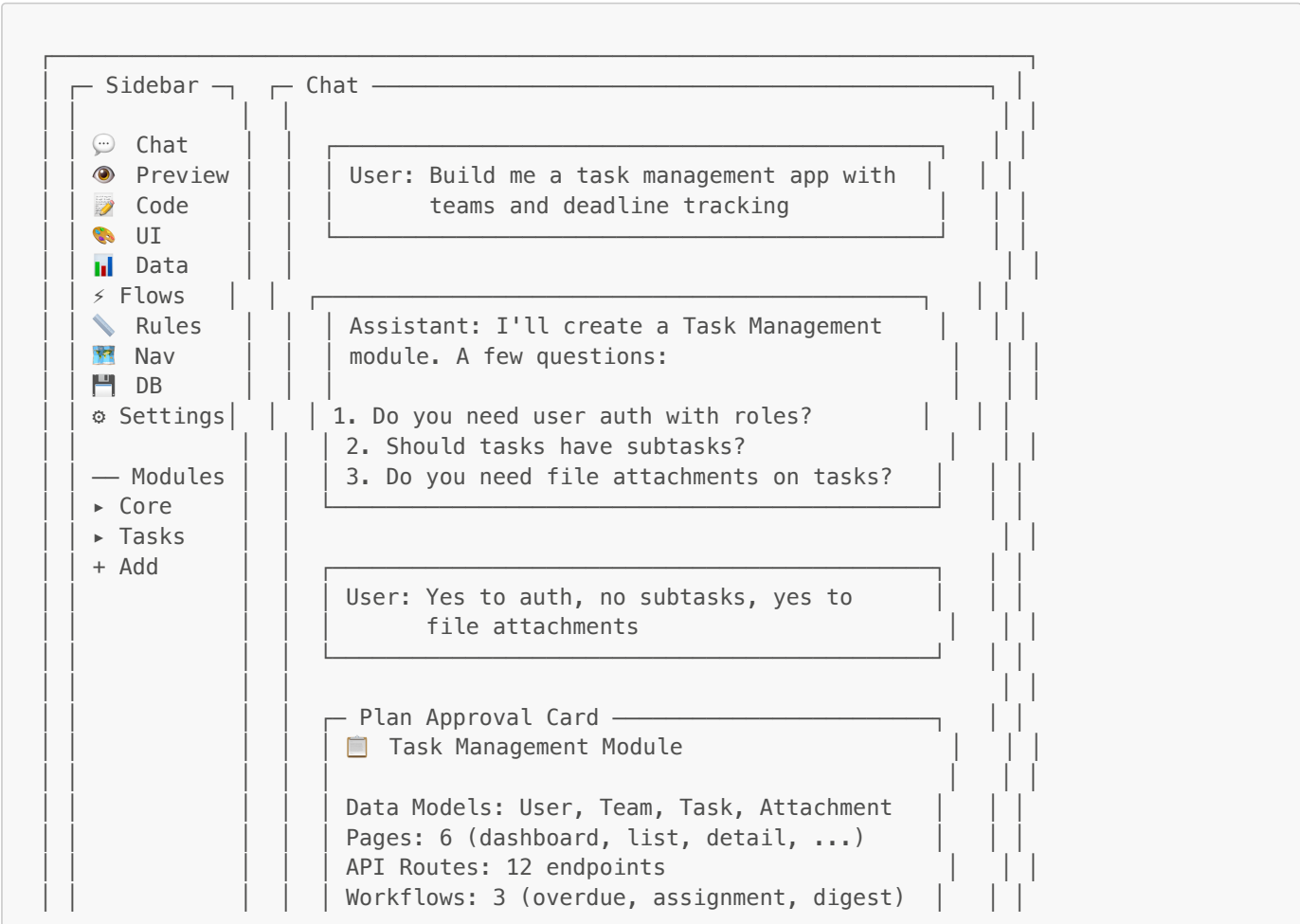
/orgs/[orgId] (org dashboard)
├─ App cards grid (all apps in this org)
├─ "New App" button → creation options:
│   ├─ "Describe what you need" → Chat (Planner)
│   ├─ "I'm not sure yet" → Discovery conversation
│   ├─ "Start from a template" → Template gallery
│   └─ "Import Figma design" → Figma URL input
├─ Suggested apps section (based on org departments without apps)
├─ Sidebar:
│   ├─ Apps (grid view)
│   ├─ Templates (template gallery)
│   ├─ Org Structure (org chart editor)
│   ├─ People (user directory)
│   ├─ Roles & Groups
│   ├─ Access Policies
│   └─ Org Settings
├─ /orgs/[orgId]/templates      → Template gallery (filterable by category/department)
├─ /orgs/[orgId]/discover      → Discovery conversation UI
├─ /orgs/[orgId]/org-chart     → Visual org chart editor (React Flow)
├─ /orgs/[orgId]/people        → People directory table + import
├─ /orgs/[orgId]/roles         → Role management + role hierarchy
├─ /orgs/[orgId]/groups        → Group management
├─ /orgs/[orgId]/access        → Cross-app access policy overview
└─ /orgs/[orgId]/settings      → Org name, logo, plan, invite members

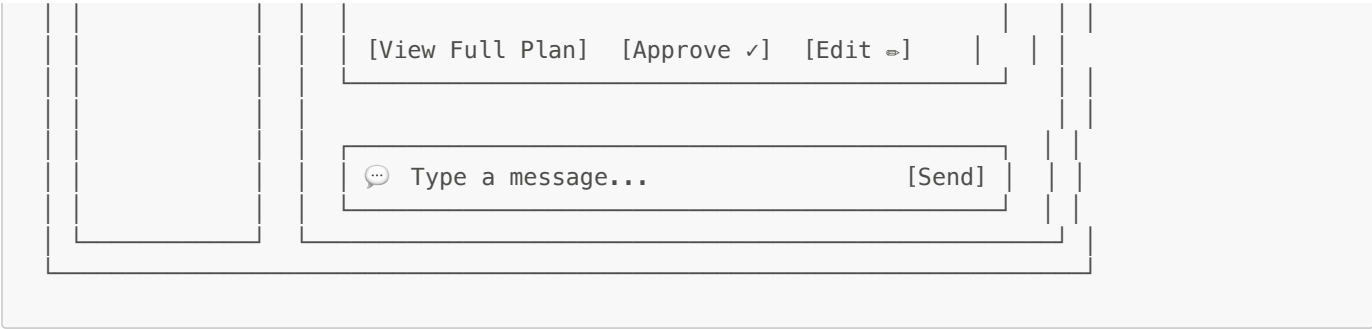
/projects/[id] (workspace - scoped to org)
├─ Layout: Sidebar + Header + Content area
├─ Sidebar (always visible):
│   ├─ ← Back to Org
│   ├─ Project name
│   └─ Navigation links:

```

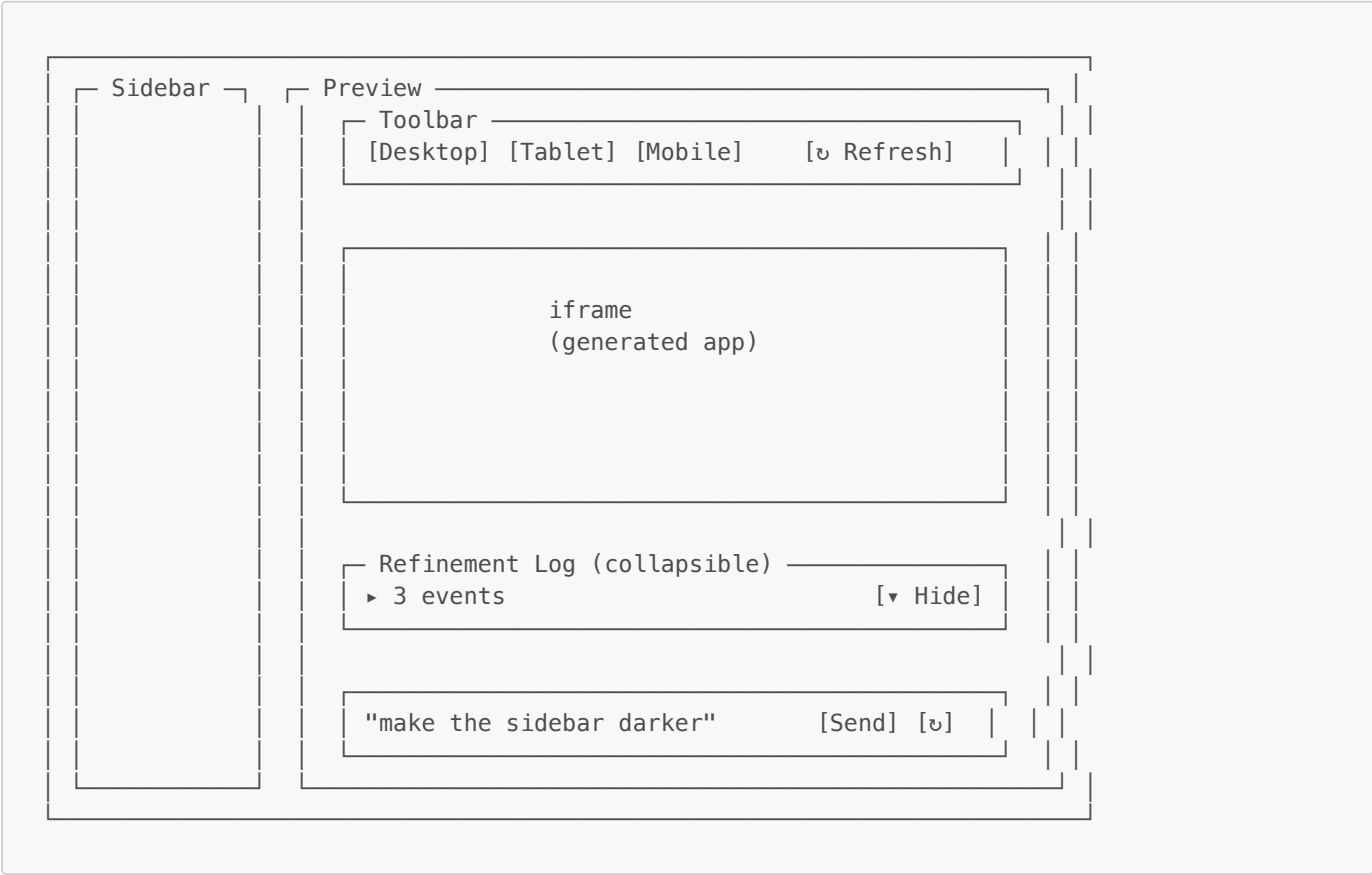


7.2 Chat Page Layout





7.3 Preview Page Layout



8. Visual Editors

8.1 Instruction Builder

Every visual editor action is converted to a text instruction. This is the core abstraction.

```
// frontend/src/lib/instruction.ts

export type EditorAction =
  | UIEditorAction
  | DataEditorAction
  | WorkflowEditorAction
  | RulesEditorAction
  | NavigationEditorAction
  | AgentEditorAction;

// --- UI Editor Actions ---

interface UIChangeStyle {
  editor: "ui";
  action: "change_style";
```

```

    component: string;           // "TaskCard"
    file: string;                // "src/components/TaskCard.tsx"
    element: string;             // "container" | "title" | "badge"
    property: string;            // "backgroundColor" | "padding" | "borderRadius"
    oldValue: string;
    newValue: string;
  }

  interface UIAddComponent {
    editor: "ui";
    action: "add_component";
    targetPage: string;          // "src/app/tasks/page.tsx"
    componentType: string;       // "Button" | "Table" | "Chart" | "Form"
    position: string;            // "after:TaskList" | "inside:Header" | "end"
    props?: Record<string, unknown>;
  }

  interface UIRemoveComponent {
    editor: "ui";
    action: "remove_component";
    component: string;
    file: string;
  }

  interface UIBindModel {
    editor: "ui";
    action: "bind_model";
    component: string;
    model: string;
    fieldMappings: Array<{
      field: string;
      displayAs: string;         // "text" | "badge" | "avatar" | "date" | "input"
      config?: Record<string, unknown>;
    }>;
  }

  // --- Data Editor Actions ---

  interface DataAddModel {
    editor: "data";
    action: "add_model";
    name: string;
    fields: Array<{
      name: string;
      type: string;
      constraints: string[];
    }>;
  }

  interface DataAddField {
    editor: "data";
    action: "add_field";
    model: string;
    field: {
      name: string;
      type: string;
      constraints: string[];
    };
    updateUI: boolean;           // whether to also update bound components
  }

  interface DataRemoveField {
    editor: "data";
    action: "remove_field";
  }

```

```
    model: string;
    fieldName: string;
  }

interface DataAddRelation {
  editor: "data";
  action: "add_relation";
  from: string;           // "SalesOrder"
  to: string;             // "Customer"
  type: string;           // "many-to-one" | "one-to-many" | "many-to-many"
  fieldName: string;      // "customerId"
  onDelete: string;       // "cascade" | "set_null" | "restrict"
}

// --- Workflow Editor Actions ---

interface WorkflowAddNode {
  editor: "workflow";
  action: "add_node";
  workflowId: string;
  nodeType: "trigger" | "action" | "condition" | "wait";
  config: Record<string, unknown>;
  connectAfter?: string;  // node ID to connect after
}

interface WorkflowRemoveNode {
  editor: "workflow";
  action: "remove_node";
  workflowId: string;
  nodeId: string;
}

interface WorkflowUpdateNode {
  editor: "workflow";
  action: "update_node";
  workflowId: string;
  nodeId: string;
  changes: Record<string, unknown>;
}

interface WorkflowConnect {
  editor: "workflow";
  action: "connect";
  workflowId: string;
  fromNodeId: string;
  toNodeId: string;
  edgeType?: "then" | "else" | "error";
}

// --- Build instruction from action ---

export function buildInstruction(action: EditorAction): string {
  switch (action.editor) {
    case "ui":
      return buildUIInstruction(action);
    case "data":
      return buildDataInstruction(action);
    case "workflow":
      return buildWorkflowInstruction(action);
    case "rules":
      return buildRulesInstruction(action);
    case "navigation":
      return buildNavInstruction(action);
    case "agent":
```

```

        return buildAgentInstruction(action);
    }
}

function buildDataInstruction(action: DataEditorAction): string {
    switch (action.action) {
        case "add_field": {
            const constraints = action.field.constraints.join(", ");
            let instruction = `Add a '${action.field.name}' field of type ${action.field.type} to the ${action.model} model.`;
            if (constraints) instruction += ` Constraints: ${constraints}.`;
            instruction += ` Update the database schema in src/infrastructure/db/schema.ts.`;
            instruction += ` Update the API routes to accept and return this field.`;
            if (action.updateUI) {
                instruction += ` Update any components bound to ${action.model} to display/input this field.`;
            }
            instruction += ` Update the seed data to include values for this field.`;
            return instruction;
        }
        case "remove_field": {
            return `Remove the '${action.fieldName}' field from the ${action.model} model. ` +
                `Remove it from the database schema, API routes, and all components that reference it. ` +
                `Do NOT drop the column from the database – just stop using it in code.`;
        }
        // ... more cases
    }
}

```

8.2 UI Editor

Library: GrapesJS (open-source, BSD 3-Clause) + @grapesjs/react

GrapesJS provides a battle-tested drag-and-drop editor with built-in Style Manager, Layer Manager, Device Manager, Undo/Redo, and Block Manager. We use [@grapesjs/react](#) with `<Canvas/>` to **disable all default UI** and render our own panels with `shadcn/ui` — fully white-labeled.

The UI editor does NOT directly edit source code.
 It renders a representation of the generated page components.
 When the user makes a change, it produces an instruction → LLM edits the real code.

Workflow:

1. Read AppModel index → know what pages and components exist
2. Read the actual source files → parse props, className, children
3. Register custom GrapesJS component types matching our component library
4. Render an editable representation in GrapesJS canvas (iframe-isolated)
5. User makes changes (drag, resize, edit properties, style)
6. On change commit → build instruction → send to Refiner
7. LLM edits source code → hot reload → UI editor re-reads and refreshes

White-Label Architecture

```

// EditorShell.tsx – all default GrapesJS UI disabled
import GjsEditor, { Canvas } from '@grapesjs/react';
import grapesjs from 'grapesjs';

export function EditorShell({ page, onInstruction }) {
    return (

```



```

<GjsEditor
  grapesjs={grapesjs}
  options={{
    storageManager: false,
    canvas: {
      styles: ['/generated-app-styles.css'], // Tailwind in canvas
    },
    plugins: [appforgeBlocks, appforgeComponents, formComponents, tailwindIntegration],
  }}
  onEditor={({editor}) => initEditor(editor, page)}
>
  {/* ALL default GrapesJS UI is hidden – we render our own */}
  <div className="flex h-full">
    <ComponentPalette /> {/* Our shadcn/ui blocks sidebar */}
    <div className="flex-1 flex flex-col">
      <EditorToolbar /> {/* Our undo/redo/device/preview bar */}
      <Canvas className="flex-1" /> {/* Only GrapesJS-rendered element */}
    </div>
    <div className="w-[320px] border-l flex flex-col">
      <LayerPanel /> {/* Our component tree */}
      <PropertiesPanel /> {/* Our trait/props editor */}
      <StylePanel /> {/* Our style manager */}
    </div>
  </div>
</GjsEditor>
);
}

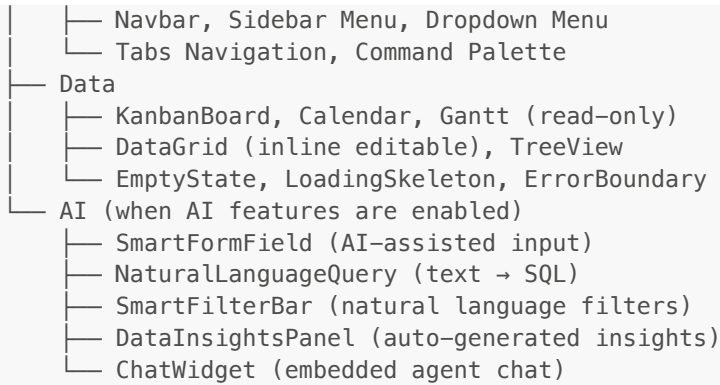
```

Component Palette (blocks)

All panels are our own React components using shadcn/ui, communicating with GrapesJS via `useEditor()`.

Component Palette (left sidebar, our shadcn/ui, not GrapesJS default)

- Layout
 - Container, Grid (2/3/4 col), Stack, Flex Row
 - Card, Section, Divider, Spacer
 - Tabs, Accordion, Collapsible
 - Sidebar Layout, Header/Footer
- Display
 - Heading (h1–h6), Text, Paragraph, Prose
 - Image, Avatar, Icon, Badge, Tag
 - Alert, Callout, Toast placeholder
 - Table, DataTable (with sorting/filtering/pagination)
 - List, DescriptionList, Timeline
 - Chart (bar, line, pie, area), Stat Card, Progress
- Form
 - TextInput, TextArea, RichTextEditor
 - Select, MultiSelect, Combobox, AsyncSelect
 - DatePicker, TimePicker, DateRangePicker
 - Checkbox, CheckboxGroup, RadioGroup
 - Switch, Toggle, ToggleGroup
 - NumberInput, Slider, Rating
 - FileUpload, ImageUpload, Dropzone
 - ColorPicker, TagInput, PhoneInput
 - EmailInput, URLInput, PasswordInput
 - Signature, CodeEditor
 - FormGroup (label + input + error + description)
 - FormStepper (multi-step wizard wrapper)
 - SubmitButton (bind to API route or workflow)
- Navigation
 - Button, LinkButton, IconButton
 - Breadcrumb, Pagination, Stepper



Each block is registered as a GrapesJS custom component type with:

- Editor-specific view (how it looks on canvas — may differ from production)
- Traits (editable properties shown in Properties Panel)
- Drag rules (where it can be dropped, what can be dropped inside)

Custom Component Types

```

// grapes-plugins/appforge-components.ts

export default function appforgeComponents(editor) {
  // Example: DataTable component type
  editor.Components.addType('appforge-datatable', {
    isComponent: (el) => el.dataset?.component === 'DataTable',
    model: {
      defaults: {
        tagName: 'div',
        attributes: { 'data-component': 'DataTable' },
        droppable: false,
        traits: [
          { type: 'select', name: 'model', label: 'Data Model',
            options: [] }, // populated dynamically from AppModel
          { type: 'text', name: 'columns', label: 'Visible Columns' },
          { type: 'checkbox', name: 'sortable', label: 'Sortable' },
          { type: 'checkbox', name: 'filterable', label: 'Filterable' },
          { type: 'checkbox', name: 'pagination', label: 'Pagination' },
          { type: 'number', name: 'pageSize', label: 'Page Size' },
          { type: 'checkbox', name: 'inlineEdit', label: 'Inline Editing' },
        ],
      },
    },
    view: {
      onRender({ el, model }) {
        // Render a rich preview on canvas (placeholder with column headers)
        const modelName = model.get('attributes')['data-model'] || 'Items';
        el.innerHTML = `
          <div class="border rounded-lg overflow-hidden">
            <div class="bg-gray-50 px-4 py-2 font-semibold text-sm">${modelName}</div>
            <table class="w-full text-sm">
              <thead><tr class="bg-gray-100">
                <th class="px-3 py-2 text-left">Name</th>
                <th class="px-3 py-2 text-left">Status</th>
                <th class="px-3 py-2 text-left">Date</th>
              </tr></thead>
              <tbody>
                <tr class="border-t"><td class="px-3 py-2">Sample row 1</td>
                  <td class="px-3 py-2">Active</td><td class="px-3 py-2">2024-01-01</td>
                </tr>
                <tr class="border-t"><td class="px-3 py-2">Sample row 2</td>

```

```

                <td class="px-3 py-2">Draft</td><td class="px-3 py-2">2024-01-02</td>
            </tr>
        </tbody>
    </table>
    <div class="bg-gray-50 px-4 py-2 text-xs text-gray-500">Showing 2 of 2</div>
</div>`;
    },
  },
});
}

```

Built-in Features We Get Free

From GrapesJS (no custom code needed):

- └─ Style Manager – visual CSS editor with 50+ properties, per-breakpoint
- └─ Layer Manager – component tree with nesting, visibility toggle, reorder
- └─ Device Manager – desktop / tablet / mobile preview with media queries
- └─ Undo/Redo Manager – full action history, Ctrl+Z / Ctrl+Shift+Z
- └─ Block Manager – drag blocks from palette to canvas
- └─ Selector Manager – CSS class management
- └─ Asset Manager – image upload/selection
- └─ Canvas selection – click to select, hover highlights, resize handles
- └─ Copy/paste/duplicate – keyboard shortcuts
- └─ Canvas iframe – style isolation (generated app CSS doesn't leak)
- └─ Storage Manager – save/load project state (we override with our own)

We only build the UI chrome (panels, toolbars) using shadcn/ui + useEditor() hook.

8.3 Data Model Editor

Library: React Flow

Nodes: Entity cards showing model name + fields

Edges: Relationship lines (one-to-many, many-to-many)

Interactions:

- Click entity → expand field list → edit fields
- Click "+ Add Model" → new entity card appears
- Drag from entity to entity → create relationship
- Click field → edit type, constraints, default
- Right-click entity → delete (with impact analysis)
- Click "Seed" tab → edit seed data in table format

8.4 Workflow Editor

Library: React Flow

Node types:

- TriggerNode (green, lightning icon): API event, schedule, DB change, webhook, manual
- ActionNode (blue, play icon): DB query, HTTP call, email, notification, custom function
- ConditionNode (yellow, branch icon): if/else based on expression
- WaitNode (gray, clock icon): delay, wait for event
- EndNode (red, stop icon): terminate workflow
- AssignmentNode (teal, user-plus icon): assign task to user/role/manager/dept/group
- ApprovalNode (green, check-circle icon): single/sequential/parallel approval
- TaskPoolNode (blue, users icon): assign to pool, claim or round-robin
- EscalationNode (red, arrow-up icon): escalate on SLA breach or condition

Edge types:

- Default (solid line): normal flow
- Then (green dashed): condition true path
- Else (red dashed): condition false path
- Error (orange dotted): error handler path

Variable system:

- Trigger outputs are available as `{{trigger.fieldName}}`
- Each action's output is available as `{{stepId.fieldName}}`
- Properties panel shows autocomplete for available variables
- Variables are resolved from the AppModel data models

8.5 Rules Editor

Main view: Filterable table of all rules

Columns: Name | Type | Attached To | Condition | Enforce At

Filters: by type, by model, by enforcement layer

Rule forms (one per type):

- Validation: model, field, condition builder, error message, enforce checkboxes
- Access Control: model, action, role table (role → allow/deny + condition)
- Business: model, action, condition, consequence
- Computed: model, field, expression
- State Machine: model, field, states list, transition diagram (React Flow)
- Trigger: model, field, condition, action description

Condition Builder:

Visual builder for expressions like "field > value AND field2 != value2"

Supports: equals, not equals, greater than, less than, in, not in,
is null, is not null, matches regex, AND, OR, NOT

Falls back to raw expression input for complex conditions

8.6 Navigation Editor

Library: React Flow

Nodes: Screen/page cards showing route + thumbnail

Edges: Navigation links (click → navigate, button → navigate)

Interactions:

- Drag screens to organize visual layout
- Draw edges between screens to define navigation
- Click screen → edit route, layout, access rules
- Configure sidebar/navbar menu items and ordering
- Set which screen is the default/home
- Configure 404 and error pages

8.7 Org Chart Editor

Library: React Flow (at the org level, not project level)

Nodes:

- DepartmentNode (blue rectangle): name, head, employee count
- TeamNode (blue rounded): name, lead, member count
- PersonNode (avatar card): name, title, email, role badges

Edges:

- Reporting lines (solid): manager → report hierarchy

- Department membership (dashed): team belongs to department

Interactions:

- Drag person between departments/teams → updates org structure
- Click person → edit title, role, manager, department
- Right-click department → add sub-department, add team, delete
- Click "+ Add Person" → manual entry form
- Import button → CSV/JSON upload with column mapping
- Filter: by department, by role, search by name
- View toggle: tree view | flat list | matrix view
- Collapse/expand departments

8.8 Field Access Editor

Located in project workspace under "Access Control" tab.
Shows a matrix of models × roles with field-level permissions.

Access Matrix: Product

Field	staff	manager	finance	admin
name				
sku				
description				
unitPrice				
unitCost	—	—		
margin	—			
supplierNotes	—	—		

= can view = can edit — = hidden

Model: [Product ▼] [Save Changes]

Record Scope Rules (below the matrix):

Record Scoping

Rule: staff sees only records from their own department

manager sees records from their dept + sub-depts

finance sees all records

admin sees all records

Scope column: [department_id ▼]

Scope source: [user's department from org structure]

[+ Add scope rule]

Instruction generated on save:

"Update the Product model's API routes to enforce field-level access:
hide unitCost and margin from staff role, hide supplierNotes from non-admin roles. Add department-based record scoping: staff sees only their department's products, managers see their department and sub-departments."

9. Generated App Structure

9.1 Standard Structure (every generated app follows this)

Generated apps follow **Clean Architecture** with strict layer separation:

- **Domain** — pure business logic, zero framework dependencies
- **Application** — use cases / services that orchestrate domain + infrastructure
- **Infrastructure** — external concerns (DB, email, AI, auth)
- **API** — thin HTTP handlers that delegate to services
- **Components** — presentation only, no business logic

Import direction: `domain ← application ← infrastructure, api → application, components → types`

```
{project_id}/
├── docker-compose.yml
├── .env
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json
├── next.config.ts
├── postcss.config.mjs
├── drizzle.config.ts
├── app-model.json          ← AppModel index (maintained by Indexer agent)
├── src/
│   ├── domain/            ← Pure business logic (no frameworks, no DB, no HTTP)
│   │   ├── {module}/
│   │   │   ├── entities.ts  ← TypeScript types + domain constants
│   │   │   ├── rules.ts    ← Business rules (validation, computation, state machines)
│   │   │   └── errors.ts    ← Domain-specific error classes
│   │
│   ├── application/       ← Use cases / services (orchestrates domain + infra)
│   │   ├── {module}/
│   │   │   ├── {resource}.service.ts ← CRUD + business operations
│   │   │   ├── {resource}.schema.ts  ← Zod schemas for input validation
│   │   │   └── {workflow}.handler.ts  ← Workflow action handlers
│   │
│   ├── infrastructure/    ← External concerns (DB, email, AI, external APIs)
│   │   ├── db/
│   │   │   ├── schema.ts    ← All table definitions (Drizzle pgTable)
│   │   │   ├── connection.ts ← Pool + drizzle instance
│   │   │   ├── seed.ts      ← Seed data script
│   │   │   └── repositories/
│   │   │       └── {resource}.repository.ts ← DB queries (Drizzle)
│   │   ├── auth/
│   │   │   ├── middleware.ts ← Auth middleware (org-aware, role resolver)
│   │   │   ├── rbac.ts       ← Field-level access + record scope enforcement
│   │   │   └── org-context.ts ← Org structure context (roles, dept, manager chain)
│   │   ├── ai/              ← (when AI features present)
│   │   │   ├── config.ts     ← AI model and rate limit configuration
│   │   │   ├── smart-fields.ts ← Smart field computation engine
│   │   │   ├── semantic-search.ts ← Embedding + pgvector hybrid search
│   │   │   └── usage.ts       ← Token usage tracking and cost logging
│   │   ├── realtime/        ← (when real-time features needed)
│   │   │   ├── socket-server.ts ← Socket.IO server setup
│   │   │   ├── events.ts      ← Event type definitions
│   │   │   └── use-realtime.ts ← React hook: useRealtime(channel, event, cb)
│   │   ├── email/
│   │   │   └── email.service.ts ← Email sending
│   │   └── external/
│   │       └── {api}.client.ts ← External API clients
│   │
│   └── app/                ← HTTP layer (thin – delegates to application layer)
│       ├── globals.css
│       ├── layout.tsx       ← Root layout (sidebar, header, fonts)
│       ├── page.tsx         ← Dashboard/home
│       └── {module}/
```



9.1.1 Layer Rules

Layer	May import from	Never imports from
-------	-----------------	--------------------

Layer	May import from	Never imports from
domain/	Nothing (pure TypeScript)	application/, infrastructure/, app/, any npm package
application/	domain/, infrastructure/	app/, components/
infrastructure/	domain/	application/, app/, components/
app/ (routes)	application/	infrastructure/, domain/ (only types via types/)
components/	types/	application/, infrastructure/, domain/

9.1.2 What Goes Where

Code Type	Location	Example
TypeScript type / interface	domain/{module}/entities.ts	Product, OrderStatus
Validation function (pure)	domain/{module}/rules.ts	canApproveExpense(user, expense)
Domain error class	domain/{module}/errors.ts	InsufficientStockError
Zod input schema	application/{module}/{resource}.schema.ts	createProductSchema
CRUD + business operation	application/{module}/{resource}.service.ts	ProductService.create()
Drizzle table definition	infrastructure/db/schema.ts	pgTable('products', {...})
DB query (Drizzle)	infrastructure/db/repositories/{resource}.repository.ts	ProductRepository.findById()
Auth middleware	infrastructure/auth/middleware.ts	requireAuth()
RBAC enforcement	infrastructure/auth/rbac.ts	applyScopeFilter()
Smart field computation	infrastructure/ai/smart-fields.ts	computeSmartFields()
Real-time socket server	infrastructure/realtime/socket-server.ts	initSocketServer()
Real-time React hook	infrastructure/realtime/use-realtime.ts	useRealtime()
API route handler	app/api/{resource}/route.ts	GET, POST — max 15-20 lines
React component	components/{module}/{Resource}Form.tsx	Presentation only

10. Preview System

10.1 Preview Manager

```
# backend/services/preview_manager.py

import asyncio
import os
import signal
import subprocess
from pathlib import Path
from typing import Optional
import httpx

# Port ranges
PREVIEW_PORT_MIN = 3200
PREVIEW_PORT_MAX = 3299
DB_PORT_MIN = 5500
DB_PORT_MAX = 5599

# Active previews: {project_id: {proc, preview_port, db_port, output_dir}}
_previews: dict[str, dict] = {}

async def start_preview(project_id: str, output_dir: str) -> dict:
    """Start Docker Postgres + Next.js dev server for a project.

    Returns: {"preview_port": int, "db_port": int}
    """
    # Already running?
    if project_id in _previews:
        existing = _previews[project_id]
        if existing["proc"].returncode is None:
            return {
                "preview_port": existing["preview_port"],
                "db_port": existing["db_port"],
            }
        del _previews[project_id]

    # Pick ports
    preview_port = _pick_port(PREVIEW_PORT_MIN, PREVIEW_PORT_MAX)
    db_port = _pick_port(DB_PORT_MIN, DB_PORT_MAX)

    # Write .env with project-specific DB port
    env_path = Path(output_dir) / ".env"
    env_path.write_text(
        f"DATABASE_URL=postgresql://app:app@localhost:{db_port}/app\n"
        f"DB_PORT={db_port}\n"
    )

    # Step 1: Start Postgres
    env = {**os.environ, "DB_PORT": str(db_port)}
    docker_up = await asyncio.create_subprocess_exec(
        "docker", "compose", "up", "-d", "--wait",
        cwd=output_dir, env=env,
        stdout=asyncio.subprocess.DEVNULL,
        stderr=asyncio.subprocess.DEVNULL,
    )
    await docker_up.wait()

    # Step 2: Install dependencies if needed
    if not (Path(output_dir) / "node_modules").exists():
        install = await asyncio.create_subprocess_exec(
            "npm", "install", cwd=output_dir,
            stdout=asyncio.subprocess.DEVNULL,
```

```

        stderr=asyncio.subprocess.DEVNULL,
    )
    await install.wait()

# Step 3: Push schema
push = await asyncio.create_subprocess_exec(
    "npx", "drizzle-kit", "push",
    cwd=output_dir, env=env,
    stdout=asyncio.subprocess.DEVNULL,
    stderr=asyncio.subprocess.DEVNULL,
)
await push.wait()

# Step 4: Seed data
seed = await asyncio.create_subprocess_exec(
    "npx", "tsx", "src/infrastructure/db/seed.ts",
    cwd=output_dir, env=env,
    stdout=asyncio.subprocess.DEVNULL,
    stderr=asyncio.subprocess.DEVNULL,
)
await seed.wait()

# Step 5: Start Next.js dev server
proc = await asyncio.create_subprocess_exec(
    "npx", "next", "dev", "--port", str(preview_port),
    cwd=output_dir, env=env,
    stdout=asyncio.subprocess.DEVNULL,
    stderr=asyncio.subprocess.DEVNULL,
    preexec_fn=os.setsid,
)

_previews[project_id] = {
    "proc": proc,
    "preview_port": preview_port,
    "db_port": db_port,
    "output_dir": output_dir,
}

# Poll until ready
url = f"http://localhost:{preview_port}"
async with httpx.AsyncClient(timeout=5) as client:
    for _ in range(60):
        await asyncio.sleep(0.5)
        try:
            resp = await client.get(url)
            if resp.status_code < 500:
                return {"preview_port": preview_port, "db_port": db_port}
        except (httpx.ConnectError, httpx.ReadError, httpx.TimeoutException):
            continue

stop_preview(project_id)
raise RuntimeError(f"Preview server failed to start")

def stop_preview(project_id: str) -> bool:
    """Stop dev server and Docker Postgres for a project."""
    entry = _previews.pop(project_id, None)
    if not entry:
        return False

    # Kill Next.js
    proc = entry["proc"]
    if proc.returncode is None:
        try:

```

```

        os.killpg(os.getpgid(proc.pid), signal.SIGTERM)
    except (ProcessLookupError, OSError):
        pass

# Stop Docker
subprocess.run(
    ["docker", "compose", "down"],
    cwd=entry["output_dir"],
    capture_output=True,
    env={**os.environ, "DB_PORT": str(entry["db_port"])},
)
return True

def stop_all_previews():
    for pid in list(_previews.keys()):
        stop_preview(pid)

```

10.2 Database Proxy

The platform backend proxies SQL queries to the generated app's Postgres instance:

```

# backend/services/db_proxy.py

import asyncpg
from typing import Any

async def execute_query(
    db_port: int,
    query: str,
    params: list | None = None,
    readonly: bool = True,
) -> dict:
    """Execute a SQL query against a generated app's database."""
    conn = await asyncpg.connect(
        f"postgresql://app:app@localhost:{db_port}/app"
    )
    try:
        if readonly:
            rows = await conn.fetch(query, *(params or []))
            columns = [col for col in rows[0].keys() if rows else []]
            return {
                "columns": columns,
                "rows": [dict(row) for row in rows],
                "count": len(rows),
            }
        else:
            result = await conn.execute(query, *(params or []))
            return {"result": result}
    finally:
        await conn.close()

async def list_tables(db_port: int) -> list[dict]:
    """List all tables in the generated app's database."""
    result = await execute_query(
        db_port,
        """
        SELECT table_name,
               (SELECT count(*) FROM information_schema.columns c
                WHERE c.table_name = t.table_name

```

```

        AND c.table_schema = 'public') as column_count
    FROM information_schema.tables t
    WHERE table_schema = 'public'
    AND table_type = 'BASE TABLE'
    ORDER BY table_name
    """
)
return result["rows"]

async def get_table_schema(db_port: int, table_name: str) -> list[dict]:
    """Get column definitions for a table."""
    result = await execute_query(
        db_port,
        """
        SELECT column_name, data_type, is_nullable, column_default,
               character_maximum_length
        FROM information_schema.columns
        WHERE table_schema = 'public' AND table_name = $1
        ORDER BY ordinal_position
        """,
        [table_name],
    )
    return result["rows"]

```

11. Module System

11.1 Module Lifecycle

1. CREATE
 - User: "Add an inventory module"
 - Planner Agent generates structured plan
 - User reviews and approves
 - Module record created in platform DB (status: generating)
2. GENERATE
 - Code Generator runs with the plan
 - Generates all files for the module
 - If module depends on existing modules, reads their schemas and APIs
 - Validator verifies build
 - Indexer updates AppModel
 - Module status → active
3. CONNECT
 - If cross-module dependencies exist:
 - Code Generator adds foreign keys, API calls, and shared types
 - Example: Sales.SalesOrderLine references Inventory.Product
 - Indexer records the cross-module bindings
4. REFINE
 - User makes changes to module via chat or visual editors
 - Refiner Agent scoped to module context
 - Impact analysis checks cross-module effects
5. DELETE
 - Impact analysis shows what depends on this module
 - If other modules depend on it, warn/block
 - Remove module files
 - Update AppModel index
 - Module status → deleted

11.2 Cross-Module Dependencies

```
// app-model.json - crossModuleBindings section

{
  "crossModuleBindings": [
    {
      "id": "xb1",
      "from": {"module": "sales", "model": "SalesOrderLine", "field": "productId"},
      "to": {"module": "inventory", "model": "Product", "field": "id"},
      "type": "foreign_key",
      "onDelete": "restrict"
    },
    {
      "id": "xb2",
      "from": {"module": "sales", "workflow": "on-order-confirmed"},
      "to": {"module": "inventory", "api": "POST /api/stock/reserve"},
      "type": "api_call",
      "description": "Reserve inventory when sales order is confirmed"
    },
    {
      "id": "xb3",
      "from": {"module": "sales", "workflow": "on-invoice-posted"},
      "to": {"module": "finance", "api": "POST /api/journal-entries"},
      "type": "api_call",
      "description": "Create AR journal entry when invoice is posted"
    }
  ]
}
```

12. AppModel Index

12.1 Full Schema

```
// app-model.json - complete structure

{
  "version": 1,
  "projectId": "abc123",
  "updatedAt": "2026-02-28T12:00:00Z",

  "modules": [
    {
      "id": "inventory",
      "name": "Inventory Management",
      "description": "Track products, warehouses, and stock levels",
      "status": "active"
    }
  ],

  "dataModels": [
    {
      "name": "Product",
      "module": "inventory",
      "file": "src/infrastructure/db/schema.ts",
      "tableName": "products",
      "fields": [
        {"name": "id", "type": "uuid", "constraints": ["pk", "auto"]},
        {"name": "sku", "type": "varchar(100)", "constraints": ["unique", "not_null"]},
        {"name": "name", "type": "varchar(255)", "constraints": ["not_null"]}
      ]
    }
  ]
}
```

```

      {"name": "unitPrice", "type": "decimal(10,2)", "constraints": ["not_null"]}
    ],
    "relations": [
      {"field": "categoryId", "target": "Category", "type": "many-to-one"}
    ],
    "indexes": ["idx_product_sku", "idx_product_category"]
  }
],
"pages": [
  {
    "route": "/inventory/products",
    "name": "Product List",
    "module": "inventory",
    "file": "src/app/inventory/products/page.tsx",
    "description": "Searchable product table with filters",
    "components": ["ProductTable", "ProductFilters"]
  }
],
"components": [
  {
    "name": "ProductTable",
    "module": "inventory",
    "file": "src/components/inventory/ProductTable.tsx",
    "props": ["products: Product[]", "onEdit: (id) => void", "onDelete: (id) => void"],
    "boundTo": {"model": "Product", "fields": ["sku", "name", "unitPrice", "isActive"]},
    "usedIn": ["/inventory/products"]
  }
],
"apiRoutes": [
  {
    "method": "GET",
    "path": "/api/products",
    "module": "inventory",
    "file": "src/app/api/products/route.ts",
    "description": "List products with pagination and search",
    "model": "Product",
    "params": ["page", "limit", "search", "category"]
  },
  {
    "method": "POST",
    "path": "/api/products",
    "module": "inventory",
    "file": "src/app/api/products/route.ts",
    "description": "Create product",
    "model": "Product",
    "accepts": ["sku", "name", "description", "categoryId", "unitCost", "unitPrice"]
  }
],
"workflows": [
  {
    "id": "wf-on-low-stock",
    "name": "Low Stock Alert",
    "module": "inventory",
    "definitionFile": "src/workflows/definitions/low-stock-alert.json",
    "trigger": {"type": "db_change", "model": "StockLevel", "condition": "available <
reorderPoint"},
    "steps": 3,
    "actionFiles": [
      "src/workflows/actions/check-reorder-point.ts",
      "src/workflows/actions/create-draft-po.ts",

```

```

    "src/workflows/actions/notify-purchasing.ts"
  ]
}
],
"rules": [
  {
    "id": "r1",
    "name": "Price must be positive",
    "type": "validation",
    "module": "inventory",
    "attachedTo": {"model": "Product", "field": "unitPrice"},
    "condition": "value > 0",
    "message": "Price must be greater than zero",
    "enforce": ["ui", "api"]
  },
  {
    "id": "r2",
    "name": "PO status transitions",
    "type": "state_machine",
    "module": "inventory",
    "attachedTo": {"model": "PurchaseOrder", "field": "status"},
    "states": ["draft", "submitted", "approved", "received", "cancelled"],
    "transitions": [
      {"from": "draft", "to": ["submitted", "cancelled"]},
      {"from": "submitted", "to": ["approved", "cancelled"]},
      {"from": "approved", "to": ["received", "cancelled"]},
      {"from": "received", "to": []},
      {"from": "cancelled", "to": ["draft"]}
    ],
    "enforce": ["ui", "api"]
  }
],
"bindings": [
  {
    "id": "b1",
    "model": "Product",
    "target": {"type": "component", "component": "ProductTable"},
    "fields": {
      "sku": {"display": "text", "sortable": true},
      "name": {"display": "text", "sortable": true},
      "unitPrice": {"display": "currency", "format": "USD"},
      "isActive": {"display": "badge", "colorMap": {"true": "green", "false": "gray"}}
    }
  },
  {
    "id": "b2",
    "model": "Product",
    "target": {"type": "api", "route": "/api/products"},
    "operations": ["list", "get", "create", "update", "delete"]
  }
],
"agents": [
  {
    "id": "agent-support",
    "name": "Customer Support",
    "module": "support",
    "description": "Answers customer questions, creates support tickets, checks order status",
    "definitionFile": "src/agents/definitions/customer-support.json",
    "runtimeFile": "src/agents/runtime.ts",
    "systemPrompt": "You are a helpful customer support agent for..."
  }
]

```

```

    "model": "claude-haiku-4-5-20251001",
    "tools": [
      {"name": "get_order", "source": "api", "route": "/api/orders/:id"},
      {"name": "create_ticket", "source": "api", "route": "/api/tickets"},
      {"name": "search_faq", "source": "custom", "file": "src/agents/tools/search-
faq.ts"}
    ],
    "memory": {"type": "conversation", "maxMessages": 50, "tableName":
"agent_conversations"},
    "guardrails": ["no_pii_in_response", "max_tokens_2000"],
    "chatRoute": "/support/chat",
    "chatComponent": "src/components/support/ChatWidget.tsx"
  }
],

"crossModuleBindings": [],

"theme": {
  "primaryColor": "#3b82f6",
  "font": "Inter",
  "borderRadius": "8px"
},

"navigation": {
  "type": "sidebar",
  "items": [
    {"label": "Dashboard", "route": "/", "icon": "LayoutDashboard"},
    {
      "label": "Inventory",
      "icon": "Package",
      "children": [
        {"label": "Products", "route": "/inventory/products"},
        {"label": "Warehouses", "route": "/inventory/warehouses"},
        {"label": "Purchase Orders", "route": "/inventory/purchase-orders"}
      ]
    }
  ]
}
}
}

```

13. Binding System

13.1 How Bindings Work

When a component is "bound" to a data model, the binding record tracks:

- Which model (Product)
- Which component (ProductTable)
- Which fields are mapped and how they're displayed
- What data source feeds the component (API route)

When the model changes:

1. Indexer detects the binding
2. Shows impact analysis: "Product model changed, these components use it"
3. If user opts to propagate:
 - Instruction generated for each affected component
 - Refiner updates each one

When a component is created:

1. UI Editor shows "Bind to model" dropdown
2. User selects model → fields auto-populate

3. User configures how each field is displayed
4. Instruction generated → LLM creates the component with proper data flow

13.2 Data Flow Through Bindings

Database (Postgres)
 ↓ Drizzle ORM query
API Route (route.ts)
 ↓ HTTP response (JSON)
Page Component (page.tsx)
 ↓ fetch() or server component
 ↓ passes data as props
Bound Component (ProductTable.tsx)
 ↓ renders each field according to binding config
UI (table rows, cards, forms)

The binding tracks this entire chain so that when the model changes, we know exactly which files need updating.

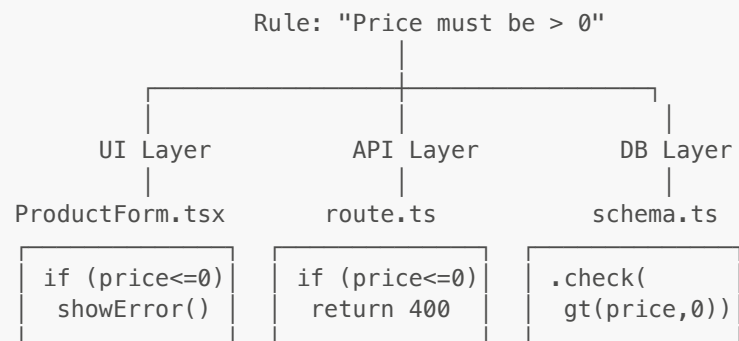
14. Rules Engine

14.1 Rule Types and Code Generation

Each rule type generates specific code patterns:

VALIDATION → src/domain/{module}/rules.ts + src/application/{module}/{resource}.schema.ts + API-level checks
ACCESS → middleware or route-level role checks + UI conditional rendering
BUSINESS → API route logic (if/else in handlers)
COMPUTED → getter function or derived column
STATE → transition validation function + UI dropdown filtering
TRIGGER → workflow trigger registration

14.2 Enforcement Layers



All three are generated from a single rule definition.
The instruction builder includes all enforcement points.

15. Workflow Engine

15.1 Runtime Structure

```
// Generated in app: src/workflows/runtime.ts

interface WorkflowDefinition {
  id: string;
  name: string;
  trigger: TriggerConfig;
  steps: StepDefinition[];
}

interface StepDefinition {
  id: string;
  type: "action" | "condition" | "wait" | "assignment" | "approval" | "task_pool" |
  "escalation"
  | "ai_classify" | "ai_extract" | "ai_decide" | "ai_generate";
  actionFile?: string; // path to action function
  condition?: string; // expression for condition nodes
  thenStep?: string; // step ID for true branch
  elseStep?: string; // step ID for false branch
  nextStep?: string; // step ID for normal flow
  onErrorStep?: string; // step ID for error handling
  config: Record<string, any>; // action-specific configuration
}

type TriggerConfig =
  | { type: "api_event"; event: string }
  | { type: "schedule"; cron: string }
  | { type: "db_change"; model: string; field?: string; condition?: string }
  | { type: "webhook"; path: string }
  | { type: "manual"; name: string };

// Executor
async function executeWorkflow(
  workflow: WorkflowDefinition,
  triggerData: Record<string, any>,
): Promise<ExecutionLog> {
  const context = { trigger: triggerData };
  const log: ExecutionLog = { workflowId: workflow.id, steps: [], startedAt: new Date() };

  let currentStepId = workflow.steps[0]?.id;

  while (currentStepId) {
    const step = workflow.steps.find(s => s.id === currentStepId);
    if (!step) break;

    const stepLog: StepLog = { stepId: step.id, startedAt: new Date(), status: "running" };

    try {
      if (step.type === "condition") {
        const result = evaluateExpression(step.condition!, context);
        stepLog.result = { conditionResult: result };
        currentStepId = result ? step.thenStep : step.elseStep;
      } else if (step.type === "action") {
        const actionFn = await import(step.actionFile!);
        const result = await actionFn.default(context, step.config);
        context[step.id] = result;
        stepLog.result = result;
        currentStepId = step.nextStep;
      } else if (step.type === "wait") {
        // For MVP: just delay. Later: persist and resume.
        await delay(step.config.durationMs);
        currentStepId = step.nextStep;
      }
    }

    stepLog.status = "completed";
  }
}
```

```

    } catch (error) {
      stepLog.status = "failed";
      stepLog.error = error.message;
      currentStepId = step.onErrorStep || undefined;
    }

    stepLog.completedAt = new Date();
    log.steps.push(stepLog);
  }

  log.completedAt = new Date();
  return log;
}

```

15.2 Trigger Registration

```

// Generated in app: src/workflows/triggers.ts
// This file is auto-generated based on workflow definitions

import { executeWorkflow } from "../runtime";
import { loadWorkflowDefinitions } from "../definitions";
const workflows = loadWorkflowDefinitions();

// API event triggers – called from API routes
export async function onEvent(eventName: string, data: Record<string, any>) {
  const matching = workflows.filter(
    w => w.trigger.type === "api_event" && w.trigger.event === eventName
  );
  for (const wf of matching) {
    // Run asynchronously (don't block the API response)
    executeWorkflow(wf, data).catch(err => {
      console.error(`Workflow ${wf.name} failed:`, err)
    });
  }
}

// Schedule triggers – registered via cron
// In production, use a proper scheduler. For dev, use node-cron.
import cron from "node-cron";

for (const wf of workflows.filter(w => w.trigger.type === "schedule")) {
  cron.schedule(wf.trigger.cron, () => {
    executeWorkflow(wf, { scheduledAt: new Date() });
  });
}

```

16. Database Management

16.1 Schema Changes (via Data Model Editor)

```

User adds field → Instruction → LLM edits schema.ts → drizzle-kit push → column added
User changes type → Instruction → LLM edits schema.ts + writes migration SQL → applied
User deletes field → Instruction → LLM removes from code (keeps column in DB)
User adds relation → Instruction → LLM adds foreign key + references()
User adds index → Instruction → LLM adds .index() in schema

```

16.2 Seed Data Management

Seed data lives in `src/infrastructure/db/seed.ts`.
 It's idempotent – checks before inserting.
 Visual editor: table UI where users can add/edit/delete seed rows.
 "Generate realistic data" button → LLM generates 10–20 rows of realistic data.
 "Reset database" → drops all data, re-runs seed.

16.3 Data Browser Endpoints

GET <code>/api/projects/:id/db/tables</code>	→ list tables (via <code>information_schema</code>)
GET <code>/api/projects/:id/db/tables/:name</code>	→ get table schema
GET <code>/api/projects/:id/db/query?sql=...</code>	→ execute SELECT (read-only)
POST <code>/api/projects/:id/db/query</code>	→ execute INSERT/UPDATE/DELETE
POST <code>/api/projects/:id/db/seed</code>	→ re-run seed script
POST <code>/api/projects/:id/db/reset</code>	→ docker compose down -v, up, push, seed

17. Authentication & Authorization

17.1 Platform Auth (who uses the builder)

Simple email/password auth via JWT.
 Stored in platform PostgreSQL (`platform_users` table).
 JWT in `HttpOnly` cookie, contains: `{userId, orgId, platformRole}`
 Middleware on all `/api/` routes except `/api/auth/*`.
 Org-scoped: all API routes resolve the current org from JWT or URL.

17.2 Generated App Auth (org-aware)

Generated apps authenticate against the org's people directory.
 Users don't create accounts per-app – they exist at the org level.

How it works:

- Platform maintains `org_people` with all end-users
- When an app is generated, the Code Generator creates:
 - NextAuth.js v5 configuration
 - Auth syncs from platform `org_people` (via API or embedded JWT)
 - Login page (email/password or SSO depending on org settings)
 - Auth middleware for protected routes
 - Role resolver: maps org roles → app-level permissions
 - Field access middleware: filters response fields per role
 - Record scope middleware: filters query by department/team ownership
 - Session includes: `{userId, orgRoles, appRoles, departmentId, teamId, managerId}`
- On each request the middleware:
 - Validates JWT → gets `userId`
 - Loads user's org roles and app roles from context
 - For read requests: filters out fields the user can't view
 - For write requests: validates field-level edit permissions
 - For list requests: applies record-scope filter (e.g., department-only)
- Auth data flow:
 - Platform (`org_people`, `org_roles`, `field_access_policies`)
 - synced to generated app on preview start / deploy
 - stored in generated app's own PostgreSQL (`users`, `roles`, `policies` tables)
 - runtime enforcement via middleware + Drizzle query filters

The org structure is the source of truth.
Generated apps consume it, they don't define it.

17.3 Multi-App SSO

Within an organization, all generated apps share identity:

- User logs into the org portal once
- Session token is valid across all apps in the org
- Apps resolve user identity from the shared token
- No separate registration per app

Implementation:

- Org portal issues JWT with orgId + userId
- Each app validates JWT against shared secret (set in org settings)
- Apps that need extra app-specific claims can enrich the session from their own roles/permissions table

18. Real-time & Collaboration

18.1 Preview Hot-Reload Communication

When an agent makes code changes:

1. Agent edits files → Next.js HMR detects file changes → iframe auto-refreshes
2. Platform sends WebSocket message to frontend: {type: "files_changed", paths: [...]}
3. Frontend refreshes:
 - File tree (re-fetches file list)
 - Code editor (re-reads current file)
 - AppModel store (re-fetches app-model.json)
 - Visual editors (re-read their relevant data)

18.2 Future: Multi-User Collaboration

Not in MVP. Future addition:

- WebSocket room per project
- Operational transform or CRDT for concurrent edits
- Cursor presence (see who's editing what)
- Lock visual editor sections to prevent conflicts
- Chat is naturally multi-user

19. Deployment & Export

19.1 Export Options

ZIP Download:

- Everything needed to run the app
- docker-compose.yml for Postgres
- README with setup instructions
- npm install && npm run dev works out of the box

Git Repository:

- Initialize git repo in output dir
- Every agent change is a commit
- User can push to their own remote

- Docker Image:
- Generate production Dockerfile
 - Multi-stage build (build + runtime)
 - docker-compose.production.yml with proper configs

19.2 Production Dockerfile (generated)

```
# Generated in the app: Dockerfile

FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

20. AI Agent Builder

20.1 Overview

The AI Agent Builder lets users create intelligent agents that live inside their generated apps. These agents can use the app's own API endpoints as tools, query the app's database, trigger workflows, and interact with users through embedded chat interfaces.

Key Insight: Since AppForge already generates full-stack apps with APIs, databases, and workflows, an AI agent inside a generated app can use the app's own endpoints as tools – no external integration needed.

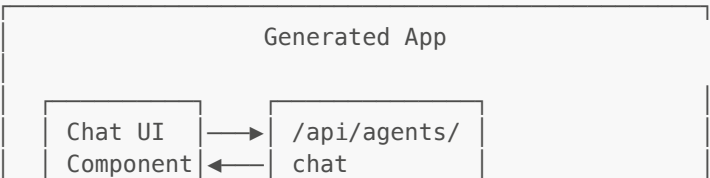
Example:

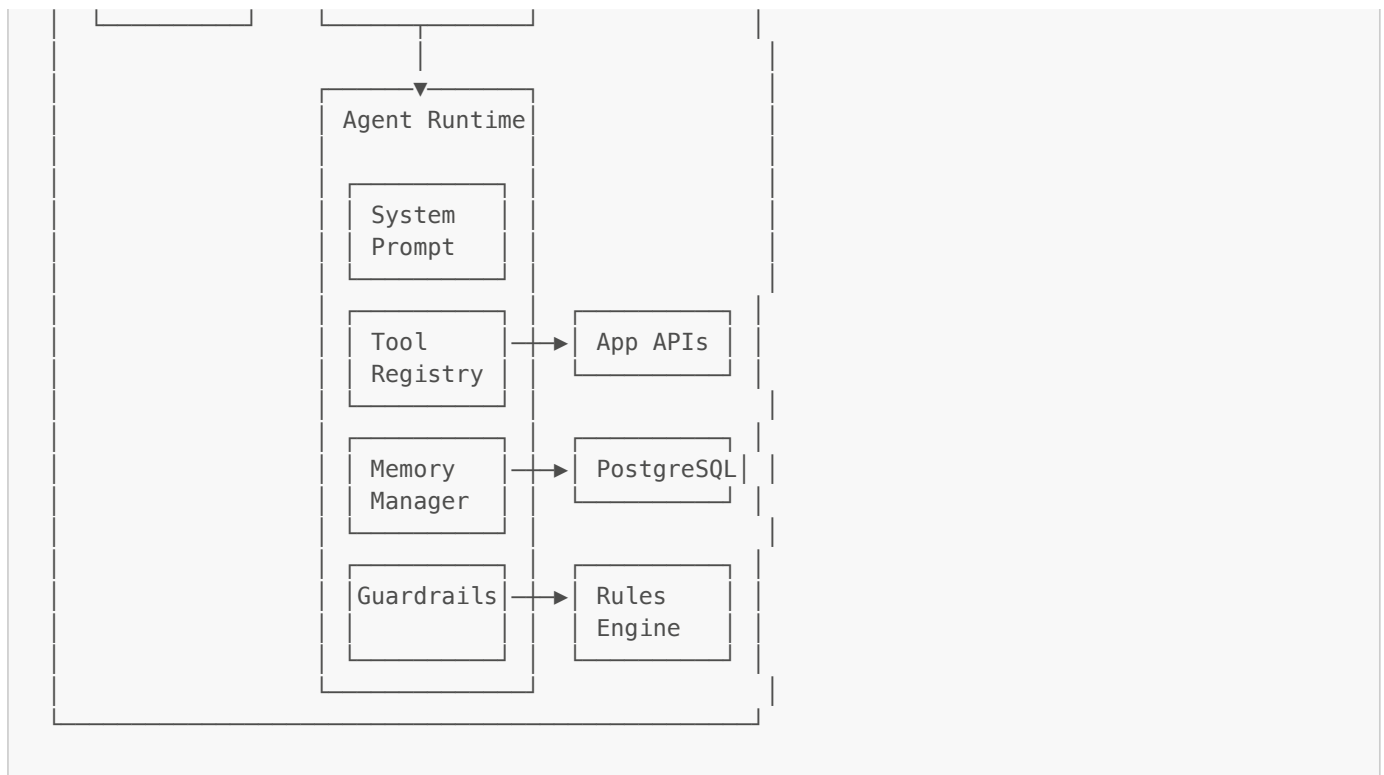
User says: "Add an AI assistant that can check inventory levels, create purchase orders, and alert me when items are low"

AppForge:

1. Generates agent definition (prompt, tools, config)
2. Auto-registers /api/products, /api/purchase-orders as agent tools
3. Connects low-stock workflow as an agent-triggerable action
4. Generates chat UI component embedded in the app
5. Sets up conversation history table in the app's PostgreSQL

20.2 Agent Architecture in Generated Apps





20.3 Agent Definition Schema

// src/agents/definitions/{agent-name}.json – defines agents in the generated app

```

{
  "agents": [
    {
      "id": "support-agent",
      "name": "Customer Support",
      "description": "Handles customer inquiries, order status, and ticket creation",
      "enabled": true,

      "model": {
        "provider": "anthropic",
        "model": "claude-haiku-4-5-20251001",
        "maxTokens": 2048,
        "temperature": 0.3
      },
    },
  ],

```

"systemPrompt": "You are a helpful customer support agent for {{appName}}.\n\nYou can help customers with:\n- Checking order status\n- Creating support tickets\n- Answering product questions\n- Processing returns and refunds\n\n## Rules\n- Always be polite and professional\n- Never share other customers' data\n- If you can't help, offer to escalate to a human agent\n- Always verify the customer's identity before sharing order details\n\n## Available Data\nYou have access to the following through your tools:\n- Orders (read-only)\n- Products (read-only)\n- Support tickets (read + create)\n- Customer profile (read-only, current user only)",

```

    "tools": [
      {
        "name": "get_order",
        "description": "Look up an order by ID or order number",
        "source": "api",
        "method": "GET",
        "route": "/api/orders/:id",
        "paramMapping": {"id": "orderId"},
        "responseFilter": ["id", "status", "items", "total", "createdAt"]
      },
    ],

```

```

    {
      "name": "list_orders",
      "description": "List recent orders for the current customer",
      "source": "api",
      "method": "GET",
      "route": "/api/orders",
      "defaultParams": {"limit": 10, "sort": "createdAt:desc"},
      "responseFilter": ["id", "status", "total", "createdAt"]
    },
    {
      "name": "create_ticket",
      "description": "Create a new support ticket",
      "source": "api",
      "method": "POST",
      "route": "/api/tickets",
      "inputSchema": {
        "subject": {"type": "string", "required": true, "maxLength": 200},
        "description": {"type": "string", "required": true},
        "priority": {"type": "enum", "values": ["low", "medium", "high"], "default":
"medium"},
        "category": {"type": "enum", "values": ["order", "product", "billing",
"technical", "other"]}
      },
    },
    {
      "name": "search_products",
      "description": "Search products by name or category",
      "source": "api",
      "method": "GET",
      "route": "/api/products",
      "paramMapping": {"query": "search"},
      "responseFilter": ["id", "name", "description", "price", "inStock"]
    },
    {
      "name": "search_faq",
      "description": "Search the knowledge base for relevant articles",
      "source": "custom",
      "file": "src/agents/tools/search-faq.ts",
      "inputSchema": {
        "query": {"type": "string", "required": true}
      }
    }
  ],

  "memory": {
    "type": "conversation",
    "maxMessages": 50,
    "summarizeAfter": 30,
    "persistTable": "agent_conversations"
  },

  "guardrails": {
    "inputValidation": {
      "maxLength": 2000,
      "blockPatterns": ["(?:ignore previous instructions", "(?:)system prompt"],
      "requireAuth": true
    },
    "outputValidation": {
      "maxTokens": 2000,
      "blockPatterns": ["(?:)password", "(?:)ssn", "(?:)credit.?card.?number"],
      "contentFilter": "standard"
    },
    "toolPermissions": {
      "get_order": {"requireAuth": true, "rateLimit": "10/min"},

```



```

    "create_ticket": {"requireAuth": true, "rateLimit": "5/min"},
    "search_products": {"rateLimit": "20/min"},
    "search_faq": {"rateLimit": "20/min"}
  },
  "escalation": {
    "trigger": "user requests human agent OR agent confidence < 0.3",
    "action": "create_ticket with category='escalation' and transfer message"
  }
},

"ui": {
  "chatRoute": "/support/chat",
  "widgetPosition": "bottom-right",
  "widgetTitle": "Need help?",
  "widgetSubtitle": "Chat with our AI assistant",
  "avatar": "/images/support-bot.svg",
  "welcomeMessage": "Hi! I'm here to help. What can I assist you with today?",
  "suggestedQuestions": [
    "Where is my order?",
    "I need to return an item",
    "How do I change my address?"
  ],
  "theme": "inherit"
}
},
],

"multiAgent": {
  "enabled": false,
  "router": {
    "model": "claude-haiku-4-5-20251001",
    "prompt": "Route the user's message to the most appropriate agent based on the
topic.",
    "agents": ["support-agent", "sales-agent"]
  }
}
}
}

```

20.4 Agent Runtime (in Generated App)

```

// src/agents/runtime.ts - core agent execution engine

import Anthropic from '@anthropic-ai/sdk';
import { db } from '@/infrastructure/db/connection';
import { agentConversations, agentMessages } from '@/infrastructure/db/schema';
import { loadAgentDefinition } from './definitions';
import { resolveTools } from './tools/registry';
import { validateInput, validateOutput } from './guardrails';
import { getConversationHistory, saveMessage, summarizeIfNeeded } from './memory';
import { eq, desc } from 'drizzle-orm';

const anthropic = new Anthropic();

interface AgentResponse {
  message: string;
  toolCalls?: Array<{name: string; input: Record<string, unknown>; result: unknown}>;
  conversationId: string;
}

export async function runAgent(
  agentId: string,
  userMessage: string,

```

```

    conversationId: string | null,
    userId: string
  ): Promise<ReadableStream> {
    const agent = loadAgentDefinition(agentId);
    if (!agent || !agent.enabled) throw new Error(`Agent ${agentId} not found or disabled`);

    // Input validation
    const inputCheck = validateInput(userMessage, agent.guardrails.inputValidation);
    if (!inputCheck.valid) {
      return streamResponse(`I'm sorry, I can't process that request. ${inputCheck.reason}`);
    }

    // Get or create conversation
    const convId = conversationId ?? await createConversation(agentId, userId);

    // Load conversation history
    const history = await getConversationHistory(convId, agent.memory.maxMessages);

    // Resolve tools (convert API route references to Anthropic tool format)
    const tools = resolveTools(agent.tools, userId);

    // Build messages
    const messages = [
      ...history.map(m => ({role: m.role, content: m.content})),
      {role: 'user' as const, content: userMessage}
    ];

    // Save user message
    await saveMessage(convId, 'user', userMessage);

    // Create streaming response
    return new ReadableStream({
      async start(controller) {
        try {
          let fullResponse = '';
          const toolResults: Array<{name: string; input: any; result: any}> = [];

          // Agentic loop – keep running until no more tool calls
          let currentMessages = messages;
          while (true) {
            const stream = await anthropic.messages.stream({
              model: agent.model.model,
              max_tokens: agent.model.maxTokens,
              temperature: agent.model.temperature,
              system: agent.systemPrompt.replace('{{appName}}', process.env.APP_NAME ?? 'the application'),
              messages: currentMessages,
              tools: tools,
            });

            let hasToolUse = false;
            let assistantContent: any[] = [];

            for await (const event of stream) {
              if (event.type === 'content_block_delta' && event.delta.type === 'text_delta') {
                fullResponse += event.delta.text;
                controller.enqueue(new TextEncoder().encode(
                  `data: ${JSON.stringify({type: 'text', content: event.delta.text})}\n\n`
                ));
              }
            }

            const finalMessage = await stream.finalMessage();

```

```

assistantContent = finalMessage.content;

// Check for tool use
const toolUseBlocks = assistantContent.filter((b: any) => b.type === 'tool_use');
if (toolUseBlocks.length === 0) break; // No tool calls, we're done

hasToolUse = true;

// Execute each tool call
const toolResultBlocks = [];
for (const toolUse of toolUseBlocks) {
  controller.enqueue(new TextEncoder().encode(
    `data: ${JSON.stringify({type: 'tool_call', tool: toolUse.name, input:
toolUse.input})}\n\n`
  ));

  // Check tool permissions
  const permission = agent.guardrails.toolPermissions?.[toolUse.name];
  if (permission?.requireAuth && !userId) {
    toolResultBlocks.push({
      type: 'tool_result' as const,
      tool_use_id: toolUse.id,
      content: 'Error: Authentication required for this action.',
      is_error: true
    });
    continue;
  }

  // Execute the tool
  try {
    const result = await executeTool(toolUse.name, toolUse.input, agent.tools,
userId);

    toolResults.push({name: toolUse.name, input: toolUse.input, result});
    toolResultBlocks.push({
      type: 'tool_result' as const,
      tool_use_id: toolUse.id,
      content: JSON.stringify(result)
    });
  } catch (error: any) {
    toolResultBlocks.push({
      type: 'tool_result' as const,
      tool_use_id: toolUse.id,
      content: `Error: ${error.message}`,
      is_error: true
    });
  }
}

// Continue the conversation with tool results
currentMessages = [
  ...currentMessages,
  {role: 'assistant' as const, content: assistantContent},
  {role: 'user' as const, content: toolResultBlocks}
];

// Output validation
const outputCheck = validateOutput(fullResponse,
agent.guardrails.outputValidation);
if (!outputCheck.valid) {
  fullResponse = "I apologize, but I'm unable to provide that information. Please
contact support directly.";
}

```

```

    // Save assistant response
    await saveMessage(convId, 'assistant', fullResponse, toolResults);

    // Summarize conversation if needed
    await summarizeIfNeeded(convId, agent.memory);

    // Send completion
    controller.enqueue(new TextEncoder().encode(
      `data: ${JSON.stringify({type: 'done', conversationId: convId})}\n\n`
    ));
    controller.close();
  } catch (error: any) {
    controller.enqueue(new TextEncoder().encode(
      `data: ${JSON.stringify({type: 'error', message: error.message})}\n\n`
    ));
    controller.close();
  }
}
});
}

```

20.5 Tool Registry (in Generated App)

```

// src/agents/tools/registry.ts – auto-registers app API routes as agent tools

import { type Tool } from '@anthropic-ai/sdk';

interface AgentToolDef {
  name: string;
  description: string;
  source: 'api' | 'custom';
  method?: string;
  route?: string;
  file?: string;
  paramMapping?: Record<string, string>;
  inputSchema?: Record<string, {type: string; required?: boolean; values?: string[]};
  default?: any;
  maxLength?: number;
  responseFilter?: string[];
}

export function resolveTools(toolDefs: AgentToolDef[], userId: string): Tool[] {
  return toolDefs.map(def => ({
    name: def.name,
    description: def.description,
    input_schema: buildInputSchema(def)
  }));
}

function buildInputSchema(def: AgentToolDef): Record<string, unknown> {
  if (def.inputSchema) {
    const properties: Record<string, unknown> = {};
    const required: string[] = [];
    for (const [key, val] of Object.entries(def.inputSchema)) {
      if (val.type === 'enum') {
        properties[key] = {type: 'string', enum: val.values, description: key};
      } else {
        properties[key] = {type: val.type, description: key};
        if (val.maxLength) (properties[key] as any).maxLength = val.maxLength;
      }
      if (val.required) required.push(key);
    }
    return {type: 'object', properties, required};
  }
}

```

```

    }

    // For API-sourced tools, infer from route params
    if (def.source === 'api' && def.route) {
      const params = def.route.match(/:(\w+)/g)?.map(p => p.slice(1)) ?? [];
      const properties: Record<string, unknown> = {};
      for (const p of params) {
        properties[def.paramMapping?.[p] ?? p] = {type: 'string', description: p};
      }
      return {type: 'object', properties, required: params};
    }

    return {type: 'object', properties: {}};
  }

export async function executeTool(
  toolName: string,
  input: Record<string, unknown>,
  toolDefs: AgentToolDef[],
  userId: string
): Promise<unknown> {
  const def = toolDefs.find(t => t.name === toolName);
  if (!def) throw new Error(`Unknown tool: ${toolName}`);

  if (def.source === 'api') {
    // Call the app's own API route internally
    let url = def.route!;
    // Replace route params
    for (const [key, value] of Object.entries(input)) {
      const routeParam = Object.entries(def.paramMapping ?? {}).find(([_, v]) => v ===
key)?.[0] ?? key;
      url = url.replace(`:${routeParam}`, String(value));
    }

    // Build query params for GET requests
    const baseUrl = process.env.NEXT_PUBLIC_APP_URL ?? 'http://localhost:3000';
    const fullUrl = new URL(url, baseUrl);
    if (def.method === 'GET') {
      for (const [key, value] of Object.entries({...def.defaultParams, ...input})) {
        if (!url.includes(`:${key}`)) {
          fullUrl.searchParams.set(key, String(value));
        }
      }
    }
  }

  const response = await fetch(fullUrl.toString(), {
    method: def.method ?? 'GET',
    headers: {
      'Content-Type': 'application/json',
      'X-Agent-User-Id': userId // Pass user context for access control
    },
    ...(def.method !== 'GET' ? {body: JSON.stringify(input)} : {})
  });

  if (!response.ok) {
    throw new Error(`API error: ${response.status} ${response.statusText}`);
  }

  const data = await response.json();

  // Apply response filter
  if (def.responseFilter && Array.isArray(data)) {
    return data.map((item: any) => filterObject(item, def.responseFilter!));
  } else if (def.responseFilter && typeof data === 'object') {

```

```

    return filterObject(data, def.responseFilter);
  }
  return data;
}

if (def.source === 'custom') {
  // Dynamic import of custom tool
  const toolModule = await import(`@/${def.file!.replace('src/', '')}`);
  return toolModule.default(input, {userId});
}

throw new Error(`Unknown tool source: ${def.source}`);
}

function filterObject(obj: Record<string, unknown>, fields: string[]): Record<string, unknown> {
  const result: Record<string, unknown> = {};
  for (const field of fields) {
    if (field in obj) result[field] = obj[field];
  }
  return result;
}

```

20.6 Agent Memory System

```

// src/agents/memory.ts – conversation persistence and context management

import { db } from '@/infrastructure/db/connection';
import { agentConversations, agentMessages } from '@/infrastructure/db/schema';
import { eq, desc, count } from 'drizzle-orm';
import Anthropic from '@anthropic-ai/sdk';

interface MemoryConfig {
  type: 'conversation';
  maxMessages: number;
  summarizeAfter: number;
  persistTable: string;
}

export async function getConversationHistory(
  conversationId: string,
  maxMessages: number
): Promise<Array<{role: 'user' | 'assistant'; content: string}>> {
  const messages = await db
    .select()
    .from(agentMessages)
    .where(eq(agentMessages.conversationId, conversationId))
    .orderBy(desc(agentMessages.createdAt))
    .limit(maxMessages);

  // Check if there's a summary for older messages
  const conversation = await db
    .select()
    .from(agentConversations)
    .where(eq(agentConversations.id, conversationId))
    .limit(1);

  const history: Array<{role: 'user' | 'assistant'; content: string}> = [];

  // Prepend summary as system context if exists
  if (conversation[0]?.summary) {
    history.push({

```

```

        role: 'user',
        content: `[Previous conversation summary: ${conversation[0].summary}]`
    });
    history.push({
        role: 'assistant',
        content: 'Understood, I have context from our previous conversation.'
    });
}

// Add recent messages (reversed to chronological order)
history.push(...messages.reverse().map(m => ({
    role: m.role as 'user' | 'assistant',
    content: m.content
})));

return history;
}

export async function saveMessage(
    conversationId: string,
    role: 'user' | 'assistant',
    content: string,
    toolCalls?: Array<{name: string; input: any; result: any}>
): Promise<void> {
    await db.insert(agentMessages).values({
        conversationId,
        role,
        content,
        toolCalls: toolCalls ? JSON.stringify(toolCalls) : null,
        createdAt: new Date()
    });
}

export async function summarizeIfNeeded(
    conversationId: string,
    config: MemoryConfig
): Promise<void> {
    const messageCount = await db
        .select({count: count()})
        .from(agentMessages)
        .where(eq(agentMessages.conversationId, conversationId));

    if (messageCount[0].count < config.summarizeAfter) return;

    // Get all messages
    const messages = await db
        .select()
        .from(agentMessages)
        .where(eq(agentMessages.conversationId, conversationId))
        .orderBy(agentMessages.createdAt);

    // Keep last N messages, summarize the rest
    const toSummarize = messages.slice(0, -config.maxMessages);
    if (toSummarize.length === 0) return;

    const anthropic = new Anthropic();
    const summaryResponse = await anthropic.messages.create({
        model: 'claude-haiku-4-5-20251001',
        max_tokens: 500,
        messages: [{
            role: 'user',
            content: `Summarize this conversation in 2-3 sentences, focusing on key facts,
decisions, and context that would be needed for future messages:\n\n${toSummarize.map(m =>
`${m.role}: ${m.content}`).join('\n')}`
        }
    ]
    });

```

```

    }]
  });

  const summary = summaryResponse.content[0].type === 'text'
    ? summaryResponse.content[0].text : '';

  // Update conversation summary and delete summarized messages
  await db.update(agentConversations)
    .set({summary})
    .where(eq(agentConversations.id, conversationId));

  const idsToDelete = toSummarize.map(m => m.id);
  for (const id of idsToDelete) {
    await db.delete(agentMessages).where(eq(agentMessages.id, id));
  }
}

```

20.7 Agent Guardrails

```

// src/agents/guardrails.ts - input/output validation and content filtering

interface InputValidation {
  maxLength: number;
  blockPatterns: string[];
  requireAuth: boolean;
}

interface OutputValidation {
  maxTokens: number;
  blockPatterns: string[];
  contentFilter: 'none' | 'standard' | 'strict';
}

interface ValidationResult {
  valid: boolean;
  reason?: string;
}

export function validateInput(input: string, rules: InputValidation): ValidationResult {
  // Length check
  if (input.length > rules.maxLength) {
    return {valid: false, reason: `Message too long (max ${rules.maxLength} characters)`};
  }

  // Injection pattern check
  for (const pattern of rules.blockPatterns) {
    if (new RegExp(pattern).test(input)) {
      return {valid: false, reason: 'Message contains disallowed content'};
    }
  }

  return {valid: true};
}

export function validateOutput(output: string, rules: OutputValidation): ValidationResult {
  // PII/sensitive data pattern check
  for (const pattern of rules.blockPatterns) {
    if (new RegExp(pattern).test(output)) {
      return {valid: false, reason: 'Response contains potentially sensitive information'};
    }
  }
}

```



```
// Content filter
if (rules.contentFilter === 'strict') {
  // Additional checks for strict mode
  const sensitivePatterns = [
    /\b\d{3}-\d{2}-\d{4}\b/, // SSN
    /\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?\d{4}\b/, // Credit card
    /\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b/ // Email (if not needed)
  ];
  for (const pat of sensitivePatterns) {
    if (pat.test(output)) {
      return {valid: false, reason: 'Response may contain sensitive data'};
    }
  }
}

return {valid: true};
}
```

20.8 Agent Database Schema (in Generated App)

```
// Added to src/infrastructure/db/schema.ts when agents are present

import { pgTable, uuid, text, varchar, timestamp, jsonb, pgEnum } from 'drizzle-orm/pg-core';

export const agentConversations = pgTable('agent_conversations', {
  id: uuid('id').primaryKey().defaultRandom(),
  agentId: varchar('agent_id', {length: 100}).notNull(),
  userId: uuid('user_id').references(() => users.id),
  summary: text('summary'), // Rolling summary of older messages
  metadata: jsonb('metadata'), // Custom metadata
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

export const messageRoleEnum = pgEnum('message_role', ['user', 'assistant']);

export const agentMessages = pgTable('agent_messages', {
  id: uuid('id').primaryKey().defaultRandom(),
  conversationId: uuid('conversation_id').references(() => agentConversations.id, {onDelete: 'cascade'}).notNull(),
  role: messageRoleEnum('role').notNull(),
  content: text('content').notNull(),
  toolCalls: jsonb('tool_calls'), // [{name, input, result}]
  tokenCount: integer('token_count'), // For usage tracking
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// Indexes
// idx_agent_conv_user (userId, createdAt DESC) - for listing user's conversations
// idx_agent_msg_conv (conversationId, createdAt) - for loading conversation history
```

20.9 Agent API Routes (in Generated App)

```
// src/app/api/agents/[agentId]/chat/route.ts

import { NextRequest } from 'next/server';
import { runAgent } from '@agents/runtime';
import { getAuthUser } from '@infrastructure/auth/middleware';
```

```

export async function POST(
  request: NextRequest,
  { params }: { params: { agentId: string } }
) {
  const user = await getAuthUser(request);
  const { message, conversationId } = await request.json();

  if (!message || typeof message !== 'string') {
    return Response.json({error: 'Message is required'}, {status: 400});
  }

  const stream = await runAgent(
    params.agentId,
    message,
    conversationId ?? null,
    user?.id ?? 'anonymous'
  );

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    }
  });
}

// GET - list conversations for current user
export async function GET(
  request: NextRequest,
  { params }: { params: { agentId: string } }
) {
  const user = await getAuthUser(request);
  if (!user) return Response.json({error: 'Unauthorized'}, {status: 401});

  const conversations = await db
    .select({
      id: agentConversations.id,
      createdAt: agentConversations.createdAt,
      updatedAt: agentConversations.updatedAt,
      summary: agentConversations.summary,
    })
    .from(agentConversations)
    .where(
      and(
        eq(agentConversations.agentId, params.agentId),
        eq(agentConversations.userId, user.id)
      )
    )
    .orderBy(desc(agentConversations.updatedAt))
    .limit(50);

  return Response.json(conversations);
}

```

20.10 Agent Chat UI Component (in Generated App)

```

// src/components/agents/ChatWidget.tsx - embeddable chat widget

'use client';

import { useState, useRef, useEffect } from 'react';

```

```

import { MessageCircle, X, Send, Loader2, Bot, User } from 'lucide-react';

interface ChatMessage {
  role: 'user' | 'assistant';
  content: string;
  toolCalls?: Array<{name: string; input: any; result: any}>;
  streaming?: boolean;
}

interface ChatWidgetProps {
  agentId: string;
  title?: string;
  subtitle?: string;
  position?: 'bottom-right' | 'bottom-left' | 'full-page';
  welcomeMessage?: string;
  suggestedQuestions?: string[];
}

export function ChatWidget({
  agentId,
  title = 'AI Assistant',
  subtitle = 'How can I help?',
  position = 'bottom-right',
  welcomeMessage,
  suggestedQuestions = []
}: ChatWidgetProps) {
  const [isOpen, setIsOpen] = useState(false);
  const [messages, setMessages] = useState<ChatMessage[]>(
    welcomeMessage ? [{role: 'assistant', content: welcomeMessage}] : []
  );
  const [input, setInput] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);
  const [conversationId, setConversationId] = useState<string | null>(null);
  const messagesEndRef = useRef<HTMLDivElement>(null);
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({behavior: 'smooth'});
  }, [messages]);

  const sendMessage = async (text: string) => {
    if (!text.trim() || isStreaming) return;

    const userMessage: ChatMessage = {role: 'user', content: text};
    setMessages(prev => [...prev, userMessage]);
    setInput('');
    setIsStreaming(true);

    // Add placeholder for assistant response
    setMessages(prev => [...prev, {role: 'assistant', content: '', streaming: true}]);

    try {
      const response = await fetch(`/api/agents/${agentId}/chat`, {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({message: text, conversationId})
      });

      const reader = response.body?.getReader();
      const decoder = new TextDecoder();
      let fullText = '';

      while (reader) {
        const {done, value} = await reader.read();

```

```
    if (done) break;

    const chunk = decoder.decode(value);
    const lines = chunk.split('\n').filter(l => l.startsWith('data: '));

    for (const line of lines) {
      const data = JSON.parse(line.slice(6));

      if (data.type === 'text') {
        fullText += data.content;
        setMessages(prev => {
          const updated = [...prev];
          updated[updated.length - 1] = {
            role: 'assistant',
            content: fullText,
            streaming: true
          };
          return updated;
        });
      }

      if (data.type === 'tool_call') {
        // Show tool call indicator
        setMessages(prev => {
          const updated = [...prev];
          const last = updated[updated.length - 1];
          last.toolCalls = [...(last.toolCalls ?? []), {
            name: data.tool,
            input: data.input,
            result: null
          }];
          return updated;
        });
      }

      if (data.type === 'done') {
        setConversationId(data.conversationId);
      }
    }

    // Mark as no longer streaming
    setMessages(prev => {
      const updated = [...prev];
      updated[updated.length - 1].streaming = false;
      return updated;
    });
  } catch (error) {
    setMessages(prev => {
      const updated = [...prev];
      updated[updated.length - 1] = {
        role: 'assistant',
        content: 'Sorry, something went wrong. Please try again.',
        streaming: false
      };
      return updated;
    });
  }

  setIsStreaming(false);
};

if (position === 'full-page') {
  return (
```

```

<div className="flex flex-col h-full">
  <ChatHeader title={title} subtitle={subtitle} />
  <ChatMessages messages={messages} messagesEndRef={messagesEndRef} />
  {messages.length === 1 && suggestedQuestions.length > 0 && (
    <SuggestedQuestions questions={suggestedQuestions} onSelect={sendMessage} />
  )}
  <ChatInput
    input={input}
    setInput={setInput}
    onSend={() => sendMessage(input)}
    isStreaming={isStreaming}
    inputRef={inputRef}
  />
</div>
);
}

// Floating widget mode
return (
  <>
    {/* Toggle button */}
    {!isOpen && (
      <button
        onClick={() => setIsOpen(true)}
        className={`fixed ${position === 'bottom-right' ? 'right-6' : 'left-6'} bottom-6
          w-14 h-14 rounded-full bg-primary text-white shadow-lg
          flex items-center justify-center hover:scale-105 transition-transform z-50`}
      >
        <MessageCircle className="w-6 h-6" />
      </button>
    )}

    {/* Chat window */}
    {isOpen && (
      <div className={`fixed ${position === 'bottom-right' ? 'right-6' : 'left-6'}
bottom-6
w-[400px] h-[600px] bg-white rounded-2xl shadow-2xl border
flex flex-col overflow-hidden z-50`}>
        <ChatHeader title={title} subtitle={subtitle} onClose={() => setIsOpen(false)} />
        <ChatMessages messages={messages} messagesEndRef={messagesEndRef} />
        {messages.length <= 1 && suggestedQuestions.length > 0 && (
          <SuggestedQuestions questions={suggestedQuestions} onSelect={sendMessage} />
        )}
        <ChatInput
          input={input}
          setInput={setInput}
          onSend={() => sendMessage(input)}
          isStreaming={isStreaming}
          inputRef={inputRef}
        />
      </div>
    )}
  </>
);
}

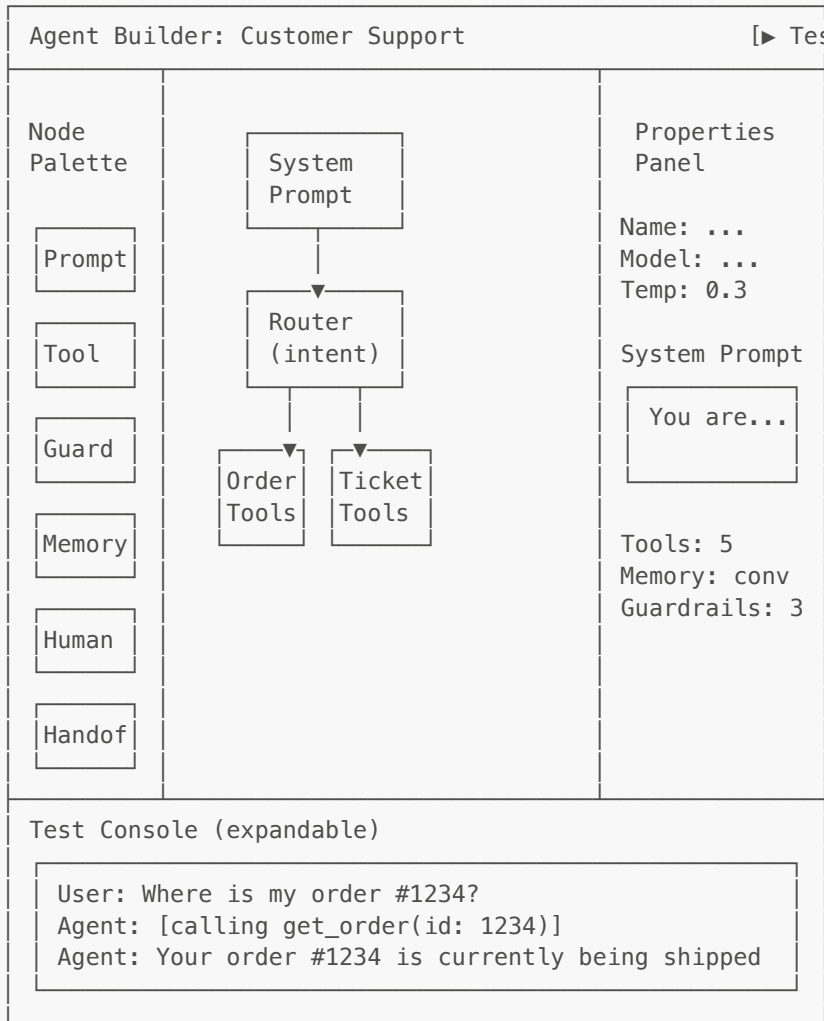
// Sub-components: ChatHeader, ChatMessages, ChatInput, SuggestedQuestions
// (standard chat UI patterns – header with title, scrollable message list,
// input bar with send button, suggested question chips)

```

Library: React Flow

The Agent Builder visual editor lets users design agent behavior visually. Like all visual editors, actions produce instructions → LLM generates code.

Canvas Layout:



Node Types:

- SystemPromptNode (purple, brain icon):
- Rich text editor for the system prompt
 - Variable interpolation: {{appName}}, {{userName}}
 - Every agent starts with one, cannot be deleted

- ToolNode (blue, wrench icon):
- Select from app's API routes (auto-discovered from AppModel)
 - Or define custom tool (name, description, input schema, handler file)
 - Configure: response filter, rate limit, auth requirement
 - Drag from API routes list → creates ToolNode automatically

- GuardrailNode (red, shield icon):
- Input validation rules (max length, block patterns)
 - Output validation rules (PII filter, content filter level)
 - Tool-specific permissions (auth, rate limit per tool)
 - Escalation rules (when to hand off to human)

- MemoryNode (green, database icon):
- Conversation history settings (max messages, summarization threshold)
 - Knowledge base toggle (enable pgvector for RAG)
 - File upload support (PDFs, docs → chunked + embedded)

- HumanHandoffNode (orange, user icon):

- Trigger conditions (user request, low confidence, specific topics)
- Handoff action (create ticket, send notification, transfer chat)
- Return flow (agent resumes after human resolves)

RouterNode (yellow, git-branch icon):

- Multi-agent: route to different agents based on topic
- Configure routing prompt and agent list
- Fallback agent if no match

Edge Types:

- Flow (solid): normal execution path
- Condition (dashed): conditional routing
- Error (dotted red): error handling path

Properties Panel (right sidebar):

- Changes based on selected node
- Immediate preview of prompt changes
- Tool test button (try a tool with sample input)
- "Apply" button → builds instruction → sends to Agent Builder agent

20.12 Agent Builder Instruction Builder

```
// frontend/src/lib/instruction.ts - add to existing instruction builder

// --- Agent Editor Actions ---

interface AgentCreate {
  editor: "agent";
  action: "create_agent";
  name: string;
  description: string;
  model: string;
  systemPrompt: string;
  tools: Array<{
    name: string;
    source: 'api' | 'custom';
    route?: string;
    description: string;
  }>;
  memory: {type: 'conversation'; maxMessages: number};
  chatPosition: 'bottom-right' | 'bottom-left' | 'full-page';
}

interface AgentUpdatePrompt {
  editor: "agent";
  action: "update_prompt";
  agentId: string;
  newPrompt: string;
}

interface AgentAddTool {
  editor: "agent";
  action: "add_tool";
  agentId: string;
  tool: {
    name: string;
    source: 'api' | 'custom';
    route?: string;
    description: string;
    inputSchema?: Record<string, unknown>;
  };
}
```

```

interface AgentRemoveTool {
  editor: "agent";
  action: "remove_tool";
  agentId: string;
  toolName: string;
}

interface AgentUpdateGuardrails {
  editor: "agent";
  action: "update_guardrails";
  agentId: string;
  guardrails: {
    inputValidation?: Partial<InputValidation>;
    outputValidation?: Partial<OutputValidation>;
    toolPermissions?: Record<string, {requireAuth?: boolean; rateLimit?: string}>;
  };
}

interface AgentConfigureMemory {
  editor: "agent";
  action: "configure_memory";
  agentId: string;
  memory: {
    maxMessages: number;
    summarizeAfter: number;
    enableKnowledgeBase?: boolean;
  };
}

interface AgentAddHandoff {
  editor: "agent";
  action: "add_handoff";
  fromAgentId: string;
  toAgentId: string;
  condition: string;
}

// Instruction builder for agent actions
function buildAgentInstruction(action: AgentEditorAction): string {
  switch (action.action) {
    case "create_agent": {
      return `Create a new AI agent called "${action.name}".

Description: ${action.description}
Model: ${action.model}

System Prompt:
${action.systemPrompt}

Tools: ${action.tools.map(t =>
  t.source === 'api'
    ? `${t.name} (API: ${t.route}) - ${t.description}`
    : `${t.name} (custom) - ${t.description}`
  ).join('\n')}`

Memory: ${action.memory.type}, max ${action.memory.maxMessages} messages

Generate the following files:
1. Create agent definition at src/agents/definitions/{agent-name}.json
2. Create any custom tool files needed in src/agents/tools/
3. Add agent_conversations and agent_messages tables to src/infrastructure/db/schema.ts (if

```



```

not present)
4. Create the chat API route at
src/app/api/agents/${action.name.toLowerCase().replace(/\s+/g, '-')}/chat/route.ts
5. Create a ChatWidget component at src/components/agents/${action.name.replace(/\s+/g,
'')}Chat.tsx
6. Add the chat widget to the appropriate page (position: ${action.chatPosition})
7. If src/agents/runtime.ts doesn't exist, create the agent runtime, tool registry, memory,
and guardrails modules`;
    }

    case "add_tool": {
      return `Add a new tool "${action.tool.name}" to agent "${action.agentId}".
Source: ${action.tool.source}${action.tool.route ? `, Route: ${action.tool.route}` : ''}
Description: ${action.tool.description}
Update the agent definition in src/agents/definitions/.
${action.tool.source === 'custom' ? `Create a custom tool handler at
src/agents/tools/${action.tool.name.replace(/_/g, '-')}.ts` : ''}`;
    }

    case "update_prompt": {
      return `Update the system prompt for agent "${action.agentId}" to:
      ""
      ${action.newPrompt}
      ""
      Update the agent definition in src/agents/definitions/.`;
    }

    // ... more cases follow the same pattern
  }
}

```

20.13 Platform Agent #11: Agent Builder

```
# backend/agents/agent_builder.py
```

```
AGENT_BUILDER_SYSTEM_PROMPT = """You are a developer creating AI agents within an existing
Next.js + TypeScript application. You generate all the code needed for an AI agent to run
inside the generated app.
```

What You Generate

When creating a new agent:

1. Agent definition in src/agents/definitions/{agent-name}.json
2. Agent runtime (src/agents/runtime.ts) – if not already present
3. Tool registry (src/agents/tools/registry.ts) – if not already present
4. Memory manager (src/agents/memory.ts) – if not already present
5. Guardrails (src/agents/guardrails.ts) – if not already present
6. Custom tool handlers (src/agents/tools/*.ts) – for non-API tools
7. Database tables for conversations (update src/infrastructure/db/schema.ts)
8. Chat API route (src/app/api/agents/[agentId]/chat/route.ts)
9. Chat UI component (src/components/agents/*Chat.tsx)
10. Integration into the app's pages (add chat widget)

When modifying an agent:

- Update src/agents/definitions/{agent-name}.json for config changes (prompt, tools, guardrails)
- Update custom tool files for tool logic changes
- Update chat component for UI changes

Rules

- The agent runtime calls the Anthropic API directly (not through our platform)

- The generated app must have @anthropic-ai/sdk in its package.json dependencies
- Agent tools should use the app's own API routes whenever possible
- Use streaming responses (SSE) for the chat API
- Conversation history is stored in the app's PostgreSQL database
- System prompts should be specific, include available tools, and set clear boundaries
- Always include basic guardrails: input length limit, prompt injection patterns, PII filter
- Rate limit all tools (sensible defaults: 10-20/min for reads, 5/min for writes)
- The agent should authenticate users through the app's existing auth system
- When creating the first agent, scaffold the full agent infrastructure (runtime, registry, etc.)
- When adding subsequent agents, only add the agent-specific files

Agent Runtime Architecture

The runtime uses a simple agentic loop:

1. Build messages (history + new user message)
2. Call Anthropic API with streaming
3. If tool_use in response → execute tool → append result → go to step 2
4. If text response → stream to client → done

Tool execution calls the app's own API routes internally via fetch().

The X-Agent-User-Id header passes user context for access control.

Database Tables

agent_conversations: id, agentId, userId, summary, metadata, createdAt, updatedAt

agent_messages: id, conversationId, role, content, toolCalls (jsonb), tokenCount, createdAt

Test After Generation

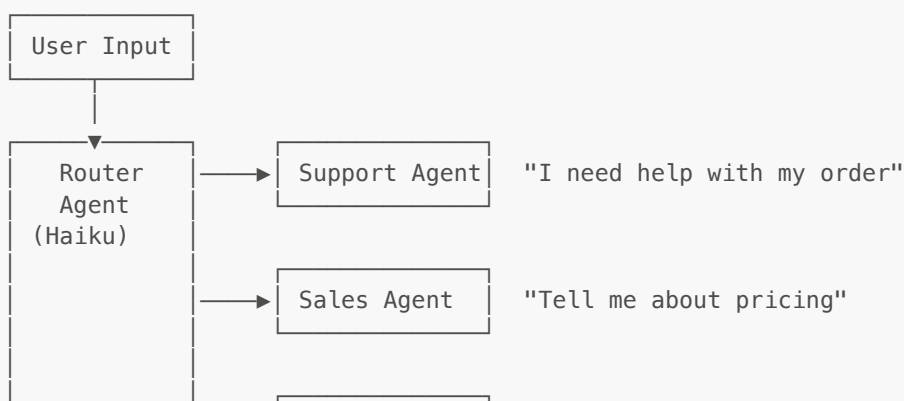
After creating/modifying an agent:

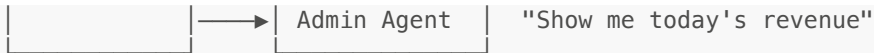
1. Verify npm run build passes
 2. Check that all imported modules exist
 3. Verify the agent definition in src/agents/definitions/ is valid JSON
- """"

```
AGENT_BUILDER_OPTIONS = ClaudeAgentOptions(
    system_prompt=AGENT_BUILDER_SYSTEM_PROMPT,
    allowed_tools=["Write", "Edit", "Read", "Bash", "Glob"],
    permission_mode="bypassPermissions",
    max_turns=30,
    model="claude-sonnet-4-20250514",
)
```

20.14 Multi-Agent Orchestration (in Generated App)

For apps with multiple agents, the runtime supports a router pattern:





Router Configuration (in `src/agents/definitions/router.json`):

```

{
  "multiAgent": {
    "enabled": true,
    "router": {
      "model": "claude-haiku-4-5-20251001",
      "prompt": "You are a message router. Based on the user's message, determine which agent should handle it. Respond with ONLY the agent ID.

      Available agents:
      - support-agent: Customer questions, orders, returns, complaints
      - sales-agent: Pricing, plans, features, demos, trials
      - admin-agent: Analytics, reports, settings, user management

      If unclear, default to support-agent.",
      "agents": ["support-agent", "sales-agent", "admin-agent"],
      "fallback": "support-agent"
    }
  }
}
  
```

The router runs as a quick Haiku call before the main agent.

Cost: ~\$0.0001 per routing decision (negligible).

The conversation stays with the routed agent unless explicitly transferred.

Handoff Pattern:

Agent A can hand off to Agent B mid-conversation:

- Agent A returns a special `tool_result`: `{"handoff": "agent-b", "reason": "...", "context": "..."}"`
- Runtime detects handoff, switches to Agent B with context
- Agent B picks up the conversation with full context
- Conversation history is shared via the same `conversationId`

20.15 Agent Templates

Pre-built agent templates available in the Agent Builder:

1. Customer Support Bot
 - Tools: order lookup, ticket creation, FAQ search, product search
 - Guardrails: PII filter, auth required, escalation to human
 - Memory: conversation with summarization
 - UI: floating widget, bottom-right
2. Data Assistant
 - Tools: natural language SQL query, chart generation, export CSV
 - Guardrails: read-only DB access, query complexity limit, auth required
 - Memory: conversation (tracks query context)
 - UI: full-page chat with data visualization panel
3. Workflow Automation Agent
 - Tools: trigger workflows, check workflow status, list pending tasks
 - Guardrails: restricted to user's permitted workflows, audit log
 - Memory: conversation
 - UI: sidebar chat or slash commands in any page
4. Admin Copilot
 - Tools: all CRUD operations, user management, analytics queries
 - Guardrails: admin role required, audit log, confirmation for destructive ops

- Memory: conversation with summarization
- UI: command palette (Cmd+K) or full-page

5. Onboarding Guide

- Tools: read app structure, check user progress, update user profile
- Guardrails: read-only access, friendly tone enforcement
- Memory: persistent per-user (remembers onboarding progress)
- UI: slide-out panel, auto-appears for new users

Templates are selected in the Agent Builder UI and customized via the visual editor. Each template pre-populates the system prompt, tools, guardrails, and UI config. The user then customizes to their app's specific data models and workflows.

20.16 Knowledge Base (RAG) Support

For agents that need to search through documents, FAQs, or other knowledge:

Architecture:

PostgreSQL + pgvector extension → vector similarity search

Setup (added to docker-compose.yml when knowledge base is enabled):

- pgvector extension enabled in PostgreSQL
- Embedding model: text-embedding-3-small (OpenAI) or local alternative
- Chunk size: 512 tokens with 50-token overlap
- Index: HNSW for fast approximate nearest neighbor search

Database Schema:

```
knowledge_chunks:
  id: uuid
  agentId: varchar(100)
  source: varchar(500)      - file name, URL, or manual entry
  content: text             - the actual text chunk
  embedding: vector(1536)   - embedding vector
  metadata: jsonb           - {page, section, title, etc.}
  createdAt: timestamp
```

Ingestion Flow:

1. User uploads document (PDF, DOCX, TXT, MD) in the Agent Builder
2. Platform extracts text → chunks → embeds → stores in knowledge_chunks
3. Agent's search_knowledge tool queries by vector similarity

Search Tool (auto-generated):

```
name: search_knowledge
description: "Search the knowledge base for relevant information"
input: {query: string, limit?: number}
implementation:
  1. Embed the query
  2. SELECT * FROM knowledge_chunks
     WHERE agent_id = $agentId
     ORDER BY embedding <=> $queryEmbedding
     LIMIT $limit
  3. Return top chunks as context
```

Note: Knowledge base is optional. Most agents work fine with just API tools. Enable it when the agent needs to reference static documents, manuals, or FAQs.

20.17 Agent Analytics & Monitoring

Every generated app with agents includes a built-in agent dashboard:

Route: /admin/agents (protected, admin role required)

Dashboard shows:

- Conversations per day (line chart)
- Total messages / avg per conversation
- Tool call frequency (bar chart by tool name)
- Average response time (latency)
- Error rate (failed tool calls, guardrail blocks)
- Token usage and estimated cost
- Escalation rate (human handoff frequency)

Conversation Inspector:

- Browse all conversations
- Filter by agent, user, date range, has-error
- View full conversation with tool call details
- Replay conversation step by step
- Flag conversations for review

Data Source:

- All metrics are derived from the agent_messages table (no extra tracking needed)
- token_count field tracks usage
- tool_calls jsonb tracks tool activity
- Guardrail blocks logged as system messages

Alert Rules (optional):

- Error rate > 10% → notification
- Escalation rate > 20% → notification
- Token usage > daily budget → disable agent or switch to cheaper model
- New unrecognized intent patterns → suggest new tools or prompt updates

21. AI-Powered Application Features

21.1 Overview

Beyond chatbot agents (Section 20), generated apps can embed LLM intelligence directly into their business logic. The platform recognizes when a user's requirement implies AI capabilities and automatically generates the necessary infrastructure.

Three Layers of AI in Generated Apps:

Layer 1: App Generation (Platform)

- LLM builds the application code
- Already covered in Sections 5–8

Layer 2: Conversational Agents (Agent Builder)

- Chatbots/assistants running inside the app
- Already covered in Section 20

Layer 3: Intelligent Features (THIS SECTION)

- LLM embedded into the app's data processing, UI, and workflows
- Smart fields, semantic search, content generation, AI decisions
- The app itself becomes intelligent, not just the chatbot

21.2 How the Planner Recognizes AI Requirements

The Planner agent (Agent #1) is trained to detect natural language patterns that imply LLM intelligence and automatically include them in the structured plan.

User Says	Planner Generates
"auto-categorize support tickets"	→ Smart Field: ai_classify
"summarize meeting notes"	→ Smart Field: ai_summarize
"detect sentiment in reviews"	→ Smart Field: ai_sentiment
"extract key info from emails"	→ Smart Field: ai_extract
"generate product descriptions"	→ Smart Field: ai_generate
"translate content to Spanish"	→ Smart Field: ai_translate
"find similar products"	→ Semantic Search component
"search with natural language"	→ Semantic Search component
"smart recommendations"	→ AI Recommendation component
"auto-tag content"	→ Smart Field: ai_classify (multi-label)
"detect duplicate entries"	→ AI Rule: similarity_check
"moderate user content"	→ AI Rule: content_moderation
"route tickets to the right team"	→ Workflow AI Decision node
"predict delivery date"	→ Smart Field: ai_predict
"score lead quality"	→ Smart Field: ai_score
"generate weekly summary report"	→ Scheduled AI Generation
"smart form auto-fill"	→ AI-Assisted Input component

21.3 Smart Fields

Smart Fields are data model fields whose values are computed by an LLM. They are triggered automatically when a record is created or updated.

Smart Field Types:

1. ai_classify – Categorize text into predefined labels

Input: source field (text)

Output: single label from a set

Example: Ticket.description → Ticket.category (billing|technical|shipping|account)

2. ai_classify_multi – Multi-label classification

Input: source field (text)

Output: array of labels

Example: Article.body → Article.tags (["react", "nextjs", "performance"])

3. ai_summarize – Generate a summary

Input: source field (text) or multiple fields

Output: summary text

Example: Meeting.transcript → Meeting.summary (2–3 sentence summary)

4. ai_sentiment – Detect sentiment

Input: source field (text)

Output: enum (positive|negative|neutral|mixed) + confidence score

Example: Review.text → Review.sentiment + Review.sentimentScore

5. ai_extract – Extract structured data from unstructured text

Input: source field (text)

Output: jsonb with extracted fields

Example: Email.body → Email.extracted {name, company, phone, intent, urgency}

6. ai_generate – Generate content from other fields

Input: multiple fields as context

Output: generated text

Example: Product.{name, specs, features} → Product.description (marketing copy)

7. ai_translate – Translate text to target language

Input: source field (text) + target language

Output: translated text

Example: Article.body → Article.bodyEs (Spanish translation)

8. `ai_score` – Score/rank based on criteria
 Input: multiple fields
 Output: numeric score (0–100) + reasoning
 Example: `Lead.{company, title, engagement}` → `Lead.qualityScore` + `Lead.scoreReason`
9. `ai_predict` – Predict a value based on patterns
 Input: multiple fields + historical data
 Output: predicted value + confidence
 Example: `Order.{items, shipping, warehouse}` → `Order.estimatedDelivery`

Smart Field Schema in Plan Output

```
{
  "data_models": [
    {
      "name": "SupportTicket",
      "fields": [
        {"name": "id", "type": "uuid", "constraints": ["pk", "auto"]},
        {"name": "subject", "type": "varchar(255)", "constraints": ["not_null"]},
        {"name": "description", "type": "text", "constraints": ["not_null"]},
        {"name": "category", "type": "varchar(50)", "constraints": []},
        "smart": {
          "type": "ai_classify",
          "source": "description",
          "labels": ["billing", "technical", "shipping", "account", "feature_request"],
          "trigger": "on_create"
        }
      ],
      {"name": "priority", "type": "varchar(20)", "constraints": [],
        "smart": {
          "type": "ai_classify",
          "source": "description",
          "labels": ["low", "medium", "high", "critical"],
          "trigger": "on_create",
          "prompt": "Classify priority based on urgency indicators, impact scope, and emotional tone."
        }
      },
      {"name": "sentiment", "type": "varchar(20)", "constraints": [],
        "smart": {
          "type": "ai_sentiment",
          "source": "description",
          "trigger": "on_create"
        }
      },
      {"name": "summary", "type": "text", "constraints": [],
        "smart": {
          "type": "ai_summarize",
          "source": "description",
          "maxLength": 100,
          "trigger": "on_create"
        }
      },
      {"name": "extracted", "type": "jsonb", "constraints": [],
        "smart": {
          "type": "ai_extract",
          "source": "description",
          "fields": ["customerName", "accountNumber", "product", "errorMessage"],
          "trigger": "on_create"
        }
      }
    ]
  }
}
```

```

    ]
  }
]
}

```

Smart Field Runtime (in Generated App)

```

// src/infrastructure/ai/smart-fields.ts - LLM-powered field computation

import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic();

interface SmartFieldConfig {
  type: string;
  source: string | string[];
  trigger: 'on_create' | 'on_update' | 'on_create_update' | 'manual';
  labels?: string[];
  fields?: string[];
  maxLength?: number;
  targetLanguage?: string;
  prompt?: string;
}

// Called from API routes after record creation/update
export async function computeSmartFields(
  record: Record<string, unknown>,
  smartFields: Record<string, SmartFieldConfig>
): Promise<Record<string, unknown>> {
  const results: Record<string, unknown> = {};

  // Batch all smart field computations into a single LLM call when possible
  const fieldsToCompute = Object.entries(smartFields);
  if (fieldsToCompute.length === 0) return results;

  // Build a combined prompt for all fields
  const sourceText = typeof smartFields[fieldsToCompute[0][0]].source === 'string'
    ? String(record[smartFields[fieldsToCompute[0][0]].source as string] ?? '')
    : (smartFields[fieldsToCompute[0][0]].source as string[])
      .map(s => `${s}: ${record[s] ?? ''}`).join('\n');

  const prompt = buildSmartFieldPrompt(record, fieldsToCompute);

  const response = await anthropic.messages.create({
    model: 'claude-haiku-4-5-20251001', // Haiku for speed + cost efficiency
    max_tokens: 1024,
    messages: [{role: 'user', content: prompt}]
  });

  const text = response.content[0].type === 'text' ? response.content[0].text : '';

  // Parse structured response
  try {
    const parsed = JSON.parse(text);
    for (const [fieldName] of fieldsToCompute) {
      if (parsed[fieldName] !== undefined) {
        results[fieldName] = parsed[fieldName];
      }
    }
  } catch {
    // Fallback: run fields individually if batch parse fails
    for (const [fieldName, config] of fieldsToCompute) {

```



```

    results[fieldName] = await computeSingleField(record, fieldName, config);
  }
}

return results;
}

function buildSmartFieldPrompt(
  record: Record<string, unknown>,
  fields: [string, SmartFieldConfig][]
): string {
  let prompt = `Analyze the following record and compute the requested fields.
Respond with a JSON object containing the results for each field.

Record:
${JSON.stringify(record, null, 2)}

Compute these fields:\n`;

  for (const [fieldName, config] of fields) {
    switch (config.type) {
      case 'ai_classify':
        prompt += `\n"${fieldName}": Classify into ONE of: [${config.labels!.join(', ')}]`;
        if (config.prompt) prompt += ` - ${config.prompt}`;
        break;
      case 'ai_classify_multi':
        prompt += `\n"${fieldName}": Classify into ALL applicable labels from:
[${config.labels!.join(', ')}]. Return as array.`;
        break;
      case 'ai_summarize':
        prompt += `\n"${fieldName}": Summarize in ${config.maxLength ?? 100} characters or
less`;
        break;
      case 'ai_sentiment':
        prompt += `\n"${fieldName}": Classify sentiment as "positive", "negative",
"neutral", or "mixed"`;
        break;
      case 'ai_extract':
        prompt += `\n"${fieldName}": Extract these fields: [${config.fields!.join(', ')}].
Return as object.`;
        break;
      case 'ai_generate':
        prompt += `\n"${fieldName}": Generate ${config.prompt ?? 'descriptive text'} based
on the record fields`;
        break;
      case 'ai_translate':
        prompt += `\n"${fieldName}": Translate the source text to
${config.targetLanguage}`;
        break;
      case 'ai_score':
        prompt += `\n"${fieldName}": Score 0-100 based on ${config.prompt ??
'quality/relevance'}. Return as number.`;
        break;
    }
  }

  prompt += '\n\nRespond with ONLY valid JSON, no explanation.';
  return prompt;
}

```

Integration with API Routes

```
// Example: how smart fields are called in a generated API route
// src/app/api/tickets/route.ts – thin handler delegates to service

import { TicketService } from '@application/support/ticket.service';
import { createTicketSchema } from '@application/support/ticket.schema';

export async function POST(request: NextRequest) {
  const body = await request.json();
  const parsed = createTicketSchema.parse(body);
  const ticket = await TicketService.create(parsed, user.id);
  return Response.json(ticket, { status: 201 });
}

// The service handles DB insert + async smart field computation:
// src/application/support/ticket.service.ts
import { TicketRepository } from '@infrastructure/db/repositories/ticket.repository';
import { computeSmartFields } from '@infrastructure/ai/smart-fields';

export class TicketService {
  static async create(data: CreateTicketInput, userId: string) {
    const ticket = await TicketRepository.insert({ ...data, createdBy: userId });

    // Compute smart fields asynchronously (don't block the response)
    computeSmartFields(ticket, ticketSmartFields)
      .then(async (smartValues) => {
        await TicketRepository.update(ticket.id, smartValues);
      })
      .catch(err => console.error('Smart field computation failed:', err));

    return ticket;
  }
}
```

Smart Field UI Indicator

In the generated app's UI, smart fields have a visual indicator:

```
Ticket #1234

Subject: Can't access my account
Description: I've been trying to log in for the...

+ Category: account          (auto-classified)
+ Priority: high              (auto-classified)
+ Sentiment: negative        (auto-detected)
+ Summary: Customer unable to login, tried
                        password reset without success.

[🔧 Override]  [🔄 Recompute]
```

The + icon indicates an AI-computed field.
Users can override the value manually if the AI got it wrong.
"Recompute" re-runs the LLM on the current source text.

21.4 Semantic Search

Beyond keyword search, generated apps can include semantic (meaning-based) search powered by embeddings and pgvector.

When the Planner sees requirements like:

```
"find similar products"
"search using natural language"
"smart search"
"content-aware search"
```

It generates:

1. Embedding column on the searchable model (vector(1536))
2. Embedding computation on create/update (via smart field trigger)
3. Search API endpoint with hybrid search (keyword + semantic)
4. Search UI component with natural language input

Semantic Search Runtime

```
// src/infrastructure/ai/semantic-search.ts

import Anthropic from '@anthropic-ai/sdk';
import { db } from '@/infrastructure/db/connection';
import { sql } from 'drizzle-orm';

// Embedding via Anthropic's Voyager or OpenAI's text-embedding-3-small
// (configurable per project)
async function embed(text: string): Promise<number[]> {
  // Using Anthropic's embedding endpoint (or OpenAI if configured)
  const response = await fetch('https://api.openai.com/v1/embeddings', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      model: 'text-embedding-3-small',
      input: text
    })
  });
  const data = await response.json();
  return data.data[0].embedding;
}

export async function semanticSearch(
  table: string,
  query: string,
  options: {
    embeddingColumn?: string;
    textColumns?: string[]; // for keyword fallback
    limit?: number;
    threshold?: number; // minimum similarity score
    filters?: Record<string, unknown>;
  } = {}
): Promise<Array<{record: any; score: number; matchType: 'semantic' | 'keyword'}>> {
  const {
    embeddingColumn = 'embedding',
    textColumns = ['name', 'description'],
    limit = 20,
    threshold = 0.3,
    filters = {}
  } = options;

  const queryEmbedding = await embed(query);

  // Hybrid search: semantic + keyword
```

```
// Semantic: cosine similarity via pgvector
// Keyword: PostgreSQL full-text search as fallback

const results = await db.execute(sql`
  WITH semantic AS (
    SELECT *, 1 - (${sql.raw(embeddingColumn)}) <=>
    ${JSON.stringify(queryEmbedding)}::vector) AS score,
    'semantic' AS match_type
    FROM ${sql.raw(table)}
    WHERE 1 - (${sql.raw(embeddingColumn)}) <=> ${JSON.stringify(queryEmbedding)}::vector)
> ${threshold}
    ORDER BY score DESC
    LIMIT ${limit}
  ),
  keyword AS (
    SELECT *, ts_rank(
      to_tsvector('english', ${sql.raw(textColumns.map(c => `COALESCE(${c}, '')`).join("
|| ' ' || ")`))),
      plainto_tsquery('english', ${query})
    ) AS score, 'keyword' AS match_type
    FROM ${sql.raw(table)}
    WHERE to_tsvector('english', ${sql.raw(textColumns.map(c => `COALESCE(${c},
'')`).join(" || ' ' || ")`)))
      @@ plainto_tsquery('english', ${query})
    LIMIT ${limit}
  )
  SELECT DISTINCT ON (id) * FROM (
    SELECT * FROM semantic
    UNION ALL
    SELECT * FROM keyword
  ) combined
  ORDER BY id, score DESC
  LIMIT ${limit}
`);

return results.rows;
}
```

Search UI Component

The generated search component supports both modes:

 Find products that are good for outdoor use
[< AI Search]

✦ Waterproof Hiking Boots score: 0.92
Durable boots for all-terrain outdoor...

✦ UV-Protection Sunglasses score: 0.87
Polarized lenses for bright outdoor...

✦ Camping Backpack 65L score: 0.84
Large capacity pack with rain cover...

— Also matching by keyword —

• Outdoor Furniture Set score: 0.65

• Garden Outdoor Lights score: 0.52

- ✦ = semantic match (meaning-based)
- = keyword match (text-based)

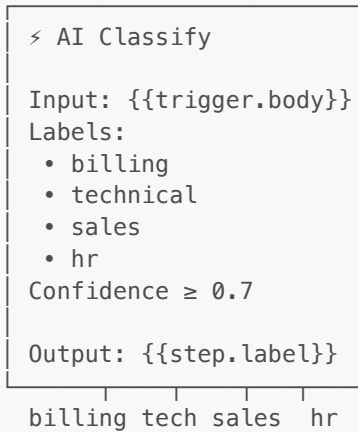
21.5 Workflow AI Nodes

The Workflow Engine (Section 15) gains new AI-powered node types that use LLM for intelligent decision-making within workflows.

New Workflow Node Types:

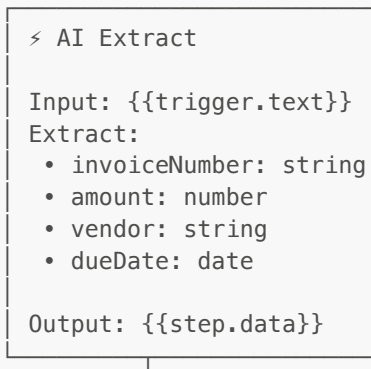
AI Classification Node (purple, brain icon):

Input: data from previous step
Config: classification labels, custom prompt
Output: {label, confidence, reasoning}
Example: Route incoming email to the right department



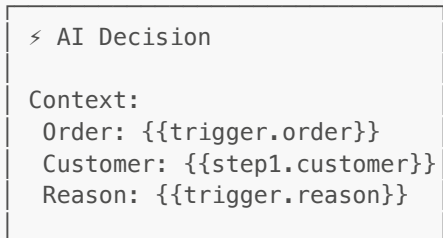
AI Extraction Node (purple, scan icon):

Input: text data from previous step
Config: fields to extract, output schema
Output: structured object with extracted fields
Example: Extract invoice details from email attachment



AI Decision Node (purple, git-branch icon):

Input: context from workflow
Config: decision prompt, options
Output: chosen option + reasoning
Example: Decide if a refund should be auto-approved or needs review



Question: Should this refund be auto-approved?

Options: approve | review

Rules:

- Auto-approve if < \$50
- Auto-approve if loyal customer (>10 orders)
- Review if > \$200
- Review if suspicious

approve review

AI Generation Node (purple, sparkles icon):

Input: context data

Config: generation prompt, output format

Output: generated text/content

Example: Generate a personalized email response

✂ AI Generate

Prompt: Write a friendly response to the customer acknowledging their issue and providing the status update.

Context:

{{trigger.ticket}}
{{step1.status}}

Max length: 300 words

Tone: professional, warm

Output: {{step.text}}

Workflow AI Node Runtime

```
// Added to src/workflows/runtime.ts - new step types

import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic();

async function executeAISTep(
  step: StepDefinition,
  context: Record<string, any>
): Promise<Record<string, unknown>> {
  const resolvedConfig = resolveVariables(step.config, context);

  switch (step.type) {
    case 'ai_classify': {
      const response = await anthropic.messages.create({
        model: 'claude-haiku-4-5-20251001',
        max_tokens: 256,
        messages: [{
          role: 'user',
          content: `Classify the following into ONE of these categories:
[${resolvedConfig.labels.join(', ')}]`
        }]
      });
    }
  }
}
```

```

    ${resolvedConfig.prompt ? `Instructions: ${resolvedConfig.prompt}\n` : ''}
    Input: ${resolvedConfig.input}

    Respond with JSON: {"label": "chosen_label", "confidence": 0.0-1.0, "reasoning": "brief
    explanation"}`
      }}
    });
    return JSON.parse(response.content[0].text);
  }

  case 'ai_extract': {
    const response = await anthropic.messages.create({
      model: 'claude-haiku-4-5-20251001',
      max_tokens: 512,
      messages: [{
        role: 'user',
        content: `Extract the following fields from this text:
Fields: ${JSON.stringify(resolvedConfig.fields)}

Text: ${resolvedConfig.input}

Respond with ONLY a JSON object containing the extracted values. Use null for fields you
can't find.`
      }]
    });
    return {data: JSON.parse(response.content[0].text)};
  }

  case 'ai_decide': {
    const response = await anthropic.messages.create({
      model: 'claude-haiku-4-5-20251001',
      max_tokens: 256,
      messages: [{
        role: 'user',
        content: `${resolvedConfig.prompt}

Context: ${JSON.stringify(resolvedConfig.context)}

Options: ${resolvedConfig.options.join(', ')}

${resolvedConfig.rules ? `Rules:\n${resolvedConfig.rules.map((r: string) => `-
${r}`}).join('\n')}` : ''}

Respond with JSON: {"decision": "chosen_option", "confidence": 0.0-1.0, "reasoning":
"explanation"}`
      }]
    });
    return JSON.parse(response.content[0].text);
  }

  case 'ai_generate': {
    const response = await anthropic.messages.create({
      model: 'claude-haiku-4-5-20251001',
      max_tokens: resolvedConfig.maxTokens ?? 1024,
      messages: [{
        role: 'user',
        content: `${resolvedConfig.prompt}

Context: ${JSON.stringify(resolvedConfig.context)}

${resolvedConfig.tone ? `Tone: ${resolvedConfig.tone}` : ''}
${resolvedConfig.maxLength ? `Max length: ${resolvedConfig.maxLength} words` : ''}

```

```

Generate the content directly, no preamble.`
    }}
  });
  return {text: response.content[0].text};
}
}

throw new Error(`Unknown AI step type: ${step.type}`);
}

```

21.6 AI-Assisted UI Components

Generated apps can include UI components that use LLM for enhanced user experience.

AI-Assisted Component Types:

1. SmartFormField – Auto-suggests values based on context

Product Description

Premium wireless headphones with active noise cancellation...

+ [Generate from name + specs]

- "Generate" button calls LLM to fill field from other form values
- User can edit the generated text before saving

2. InlineAssistant – Writing helper for text areas

Reply to customer

Thank you for reaching out. I've looked into your issue and...

+ Improve + Shorten + More formal

- Quick action buttons for LLM-powered text transformations
- Operates on the current text in the field

3. DataInsightsPanel – Auto-generated insights from data

+ Insights [v]

- Revenue is up 12% from last month
- 3 products are below reorder point
- Customer satisfaction dropped in the "shipping" category
- Top seller: Widget Pro (142 units)

[Ask a question about this data...]

- Reads aggregated data from the app's API
- LLM generates natural language insights
- Users can ask follow-up questions

4. SmartFilterBar – Natural language filtering

🔍 Show me overdue orders over \$500 that were placed last month

[< Apply]

Applied: status=overdue AND
total>500 AND
createdAt>=2026-01-01 AND
createdAt<=2026-01-31
[Clear all]

- LLM converts natural language to structured filters
- Shows the interpreted filters for transparency
- Users can clear or modify individual filters

5. NaturalLanguageQuery – Ask questions about app data

+ Ask about your data

What were our top 5 products by revenue last quarter?

Generated SQL:
SELECT p.name, SUM(oi.quantity * oi.unit_price) as revenue
FROM products p JOIN order_items oi...
Results:

Product	Revenue
Widget Pro	\$14,200
Gadget Max	\$11,800
Tool Deluxe	\$9,450
Part Standard	\$7,200
Component Basic	\$6,100

+ Summary: Widget Pro leads with \$14,200 in revenue, 20% more than the next product.

- LLM reads the database schema from Drizzle
- Generates safe, read-only SQL
- Executes query and formats results
- Adds natural language summary
- Guards: read-only (SELECT only), query complexity limits, no access to sensitive columns (passwords, tokens)

21.7 AI Rules

The Rules Engine (Section 14) gains AI-powered rule types that use LLM for fuzzy, context-aware enforcement.

New Rule Types:

```
1. content_moderation – LLM-based content policy enforcement
{
  "type": "ai_content_moderation",
  "model": "Review",
  "field": "body",
  "policy": "No hate speech, no spam, no personal attacks. Marketing links are OK.",
  "action": "flag", // "flag" | "reject" | "redact"
  "enforce": ["api"]
}
```

How it works:

- On create/update, the API route sends the field value to LLM
- LLM evaluates against the policy
- Returns: {allowed: bool, reason: string, confidence: float}
- If not allowed: flag for review, reject with message, or redact offending content

2. similarity_check – Detect near-duplicate records

```
{
  "type": "ai_similarity_check",
  "model": "Product",
  "fields": ["name", "description"],
  "threshold": 0.85,
  "action": "warn",           // "warn" | "block" | "merge_suggest"
  "enforce": ["api", "ui"]
}
```

How it works:

- On create, compute embedding of the new record
- Query pgvector for records above similarity threshold
- If found: warn user ("Similar product exists: Widget Pro"), block creation, or suggest merging

3. ai_validation – Complex validation that can't be expressed as a simple rule

```
{
  "type": "ai_validation",
  "model": "Expense",
  "fields": ["description", "amount", "category", "receipt"],
  "prompt": "Validate that the expense description matches the category and the amount is reasonable for the category. Flag suspicious patterns like round numbers over $500 or luxury items categorized as office supplies.",
  "action": "flag",
  "enforce": ["api"]
}
```

4. ai_enrichment – Auto-enrich records with external context

```
{
  "type": "ai_enrichment",
  "model": "Lead",
  "triggerField": "email",
  "enrichFields": ["companyName", "industry", "companySize", "linkedinUrl"],
  "prompt": "Based on the email domain, infer the company and fill in available details.",
  "enforce": ["api"]
}
```

21.8 Scheduled AI Tasks

Generated apps can include scheduled LLM tasks that run periodically.

Scheduled Task Types:

1. Periodic Summary Report

```
Config: {
  schedule: "0 9 * * 1",           // Every Monday at 9am
  type: "ai_summary_report",
  dataQuery: "SELECT * FROM orders WHERE created_at > NOW() - INTERVAL '7 days'",
  prompt: "Generate a weekly sales summary highlighting trends, top products, and concerns",
  deliverTo: "email:admin@company.com",
  format: "markdown"
}
```

2. Anomaly Detection

```
Config: {
  schedule: "0 */6 * * *",           // Every 6 hours
  type: "ai_anomaly_detection",
  dataQuery: "SELECT * FROM transactions WHERE created_at > NOW() - INTERVAL '6 hours'",
  prompt: "Identify any unusual patterns: sudden spikes, amounts far from average,
    unusual timing, or suspicious sequences",
  action: "create_alert",
  threshold: "medium"                // low | medium | high sensitivity
}
```

3. Data Quality Check

```
Config: {
  schedule: "0 2 * * *",             // Daily at 2am
  type: "ai_data_quality",
  tables: ["customers", "products", "orders"],
  checks: ["missing_required", "inconsistent_formats", "orphaned_records",
    "duplicate_detection", "value_outliers"],
  action: "create_report"
}
```

Implementation: These are generated as workflow definitions with a schedule trigger and an AI Generation node. No new runtime infrastructure needed – they use the existing Workflow Engine (Section 15) + Workflow AI Nodes (Section 21.5).

21.9 AI Configuration in Generated Apps

Every generated app with AI features includes:

1. src/infrastructure/ai/config.ts – Central AI configuration

```
{
  provider: "anthropic",
  models: {
    fast: "claude-haiku-4-5-20251001", // Smart fields, classification, extraction
    standard: "claude-sonnet-4-20250514", // Complex generation, decisions
  },
  embedding: {
    provider: "openai", // or "local" for self-hosted
    model: "text-embedding-3-small",
    dimensions: 1536
  },
  rateLimits: {
    smartFields: "100/min", // Per-model rate limit
    search: "50/min",
    generation: "20/min",
    nlQuery: "10/min"
  },
  costTracking: true, // Log token usage per feature
  fallback: {
    onError: "skip", // "skip" | "retry" | "default_value"
    retryAttempts: 2,
    retryDelayMs: 1000
  }
}
```

2. src/infrastructure/ai/usage.ts – Token usage tracking

- Logs every LLM call: model, tokens_in, tokens_out, feature, cost_usd
- Stored in ai_usage_log table
- Dashboard shows cost breakdown by feature type
- Budget alerts when daily/monthly spend exceeds threshold

3. Environment variables:

```

ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...           # For embeddings (if using OpenAI)
AI_RATE_LIMIT_PER_MINUTE=100
AI_MONTHLY_BUDGET_USD=50
AI_FALLBACK_ON_ERROR=skip

```

21.10 AppModel Index — AI Features Section

```

// Added to app-model.json

"aiFeatures": {
  "smartFields": [
    {
      "model": "SupportTicket",
      "field": "category",
      "type": "ai_classify",
      "source": "description",
      "labels": ["billing", "technical", "shipping", "account"],
      "trigger": "on_create",
      "model_used": "claude-haiku-4-5-20251001"
    },
    {
      "model": "SupportTicket",
      "field": "summary",
      "type": "ai_summarize",
      "source": "description",
      "trigger": "on_create",
      "model_used": "claude-haiku-4-5-20251001"
    }
  ],
  "semanticSearch": [
    {
      "model": "Product",
      "embeddingColumn": "embedding",
      "sourceColumns": ["name", "description"],
      "searchRoute": "/api/products/search"
    }
  ],
  "aiComponents": [
    {
      "type": "NaturalLanguageQuery",
      "page": "/dashboard",
      "component": "DataQueryPanel",
      "file": "src/components/shared/DataQueryPanel.tsx"
    },
    {
      "type": "SmartFilterBar",
      "page": "/orders",
      "component": "OrderFilters",
      "file": "src/components/orders/OrderFilters.tsx"
    }
  ],
  "workflowAINodes": [
    {
      "workflow": "wf-ticket-routing",
      "node": "classify-department",
      "type": "ai_classify",
      "labels": ["billing", "technical", "sales"]
    }
  ]
}

```

```

],

"aiRules": [
  {
    "id": "r-content-mod",
    "type": "ai_content_moderation",
    "model": "Review",
    "field": "body"
  }
],

"scheduledAI": [
  {
    "name": "Weekly Sales Summary",
    "schedule": "0 9 * * 1",
    "type": "ai_summary_report"
  }
],

"config": {
  "primaryModel": "claude-haiku-4-5-20251001",
  "embeddingModel": "text-embedding-3-small",
  "costTrackingEnabled": true,
  "monthlyBudget": 50
}
}

```

21.11 AI Features Visual Editor (in AppForge Platform)

A dedicated panel in the Data Model Editor and Workflow Editor for configuring AI features – no separate editor needed.

In Data Model Editor – Smart Field Configuration:

When clicking a field in the ERD, the properties panel shows:

Field Properties

Name: category
Type: varchar(50)
Constraints: ☐ not_null ☐ unique

AI Smart Field

☒ Enable AI computation

Type: [ai_classify ▼]
Source field: [description ▼]
Labels: billing, technical, shipping
account, feature_request
Trigger: [on_create ▼]
Custom prompt: (optional)

Classify based on urgency
indicators and customer impact

[Test with sample data]
Input: "I can't access my account"
Result: "account" (confidence: 0.94)

[Apply]

In Workflow Editor – AI Node Configuration:

(Already covered in Section 21.5 – AI nodes appear in the node palette alongside regular action/condition nodes)

In Rules Editor – AI Rule Configuration:
When creating a new rule, "AI Rules" appears as a rule category:

New Rule

Category: [AI Rules ▼]

Type: [Content Moderation ▼]

Model: [Review ▼]

Field: [body ▼]

Policy:

No hate speech, no spam, no personal attacks. Marketing links are OK if relevant.

Action: [Flag for review ▼]

Enforce at: ☒ API ☐ UI

[Test with sample]

Input: "This product is terrible..."

Result: ✓ Allowed (negative but valid)

[Save Rule]

Dashboard Page – AI Features Overview:
Route: /projects/[id]/ai (or tab within Settings)

AI Features

Smart Fields (5 active)

SupportTicket.category ai_classify

SupportTicket.priority ai_classify

SupportTicket.sentiment ai_sentiment

SupportTicket.summary ai_summarize

Product.description ai_generate

Semantic Search (1 model)

Product – name, description

AI Workflow Nodes (2 active)

ticket-routing: classify-department

refund-flow: auto-approve-decision

AI Rules (1 active)

content-moderation on Review.body

— Usage (this month) —

Tokens: 245,000 in / 89,000 out

Cost: \$3.20 / \$50.00 budget

Calls: 1,247 (avg 24ms latency)

Errors: 3 (0.2%)

22. Organization & Multi-Tenancy

22.1 Overview

AppForge is a multi-tenant platform. Developers create organizations, define their org structure (departments, teams, roles, reporting lines), and build multiple apps within the org. All apps share the same identity system — end users don't create separate accounts per app.

```

AppForge Platform (multi-tenant SaaS)
├── Organization: Acme Corp (Tenant 1)
│   ├── Org Structure
│   │   ├── Departments: Sales, Engineering, Finance, HR
│   │   ├── Teams: Sales-East, Sales-West, Dev-Frontend, Dev-Backend
│   │   ├── Users: 150 employees
│   │   ├── Roles: staff, lead, manager, director, VP, CEO
│   │   └── Groups: Audit Committee, Safety Team, Product Council
│   ├── RBAC Policies (span all apps)
│   │   ├── App access: which roles can access which apps
│   │   ├── Field access: which roles can see/edit which fields
│   │   ├── Record scope: which records are visible (dept/team/user scoped)
│   │   └── Workflow assignments: which roles handle which approvals
│   ├── App: Inventory Management
│   │   ├── Own PostgreSQL database (Docker Compose)
│   │   ├── Users synced from org (filtered by app access policies)
│   │   ├── Field-level access enforced in API routes
│   │   └── Workflows assign tasks to org users/roles
│   ├── App: Expense Tracker
│   ├── App: CRM
│   └── App: HR Portal
├── Organization: Beta Inc (Tenant 2)
│   └── Completely separate org, users, apps
└── Organization: Gamma Ltd (Tenant 3)
    └── ...
  
```

22.2 Organization Lifecycle

1. Developer signs up to AppForge (platform_users)
2. Developer creates Organization (organizations)
3. Developer sets up org structure:
 - a. Upload CSV of people (name, email, department, title, manager)
 - b. OR manually add via org chart editor
 - c. OR sync from AD/LDAP/HR system (future)
4. System auto-creates default roles: admin, manager, staff, viewer
5. Developer customizes roles, creates groups
6. Developer creates first app → Planner reads org structure → auto-infers RBAC
7. App is generated with org-aware auth, field access, workflow assignments
8. End-users are invited or synced → they log into the org portal
9. Org portal shows apps the user has access to
10. User clicks an app → authenticated with org identity, sees only permitted data

22.3 Org Structure Data Model

The org structure is stored in the platform database (not in generated apps).
Generated apps consume it via sync on preview start / deploy.

Entity Relationships:

Organization (1) — (N) Department
Department (1) — (N) Department (self-referencing: sub-departments)
Department (1) — (N) Team
Department (1) — (N) OrgPerson (belongs to)
Team (1) — (N) OrgPerson (belongs to)
OrgPerson (1) — (N) OrgPerson (manager → reports)
OrgPerson (N) — (M) OrgRole (person has roles)
OrgPerson (N) — (M) OrgGroup (person belongs to groups)
Organization (1) — (N) OrgRole
Organization (1) — (N) OrgGroup
Organization (1) — (N) Project (org has apps)

Key attributes:

OrgPerson:

- email, name, title, avatarUrl
- departmentId (which department they belong to)
- teamId (which team, optional)
- managerId (who they report to → org chart hierarchy)
- status: active | inactive | invited
- metadata: jsonb (phone, location, hireDate, custom fields)

OrgRole:

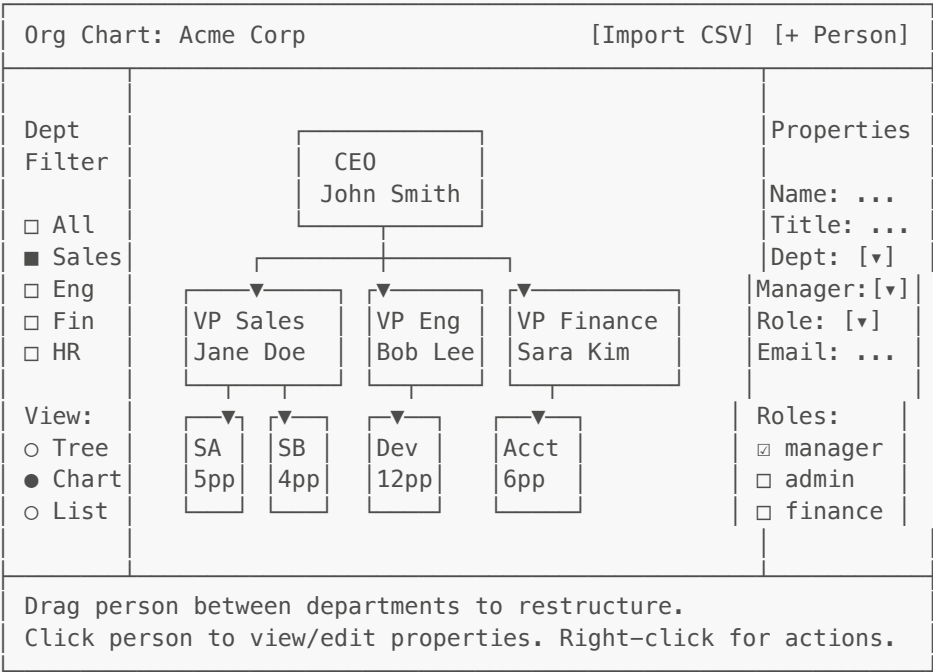
- name (admin, manager, staff, viewer, or custom)
- level (integer: 0=lowest, higher=more authority)
- isSystem (true for default roles, can't be deleted)

Department:

- name, parentId (hierarchy), headUserId (department head)
- metadata: jsonb (cost center, location, etc.)

22.4 Org Chart Visual Editor

Library: React Flow
Location: /orgs/[orgId]/org-chart



Interactions:

- Drag person → different department/team: updates departmentId/teamId

- Drag person → under another person: changes managerId (reporting line)
- Click person → properties panel shows details, inline edit
- Right-click person → Change manager, Change dept, Deactivate, Remove
- Right-click department → Add sub-department, Add team, Rename, Delete
- "+ Person" button → form: name, email, title, department, manager, roles
- "Import CSV" → upload dialog with column mapping preview

CSV Import Format:

```
name,email,title,department,team,manager_email,role
"Alice Smith","alice@acme.com","Sales Rep","Sales","Sales-East","jane@acme.com","staff"
"Bob Jones","bob@acme.com","Senior Dev","Engineering","Backend","bob.lee@acme.com","lead"
```

Import Preview:

```
Import Preview (47 people)
Column Mapping:
  name → Name ✓    email → Email ✓    title → Title ✓
  department → Department ✓    manager_email → Manager ✓

New departments to create: 2 (Marketing, Legal)
New people: 32    Updates: 15    Skipped: 0
Unresolved managers: 1 (ceo@acme.com – not in file)

[Cancel] [Import]
```

22.5 RBAC Architecture

Three layers of access control compose together:

Layer 1: Org-Level Roles

Defines WHO the person is in the organization.
Set once, applies across all apps.

Default roles (auto-created per org):

```
admin    (level: 100) – full access to everything
director (level: 80)  – department-level authority
manager  (level: 60)  – team-level authority
lead     (level: 40)  – senior individual contributor
staff    (level: 20)  – standard user
viewer   (level: 10)  – read-only access
extern   (level: 5)   – contractors, external users
```

Custom roles: org can define any additional roles

Level-based comparison: "manager+" means level >= 60

Layer 2: App-Level Roles

Defines WHAT the person can do in a specific app.
Mapped from org roles (automatic) or assigned explicitly.

Example mappings for Inventory app:

```
inventory_admin ← maps from org role "admin" + "manager" in warehouse dept
warehouse_staff ← maps from org role "staff" in warehouse department
inventory_viewer ← maps from org role "staff" (all departments)
procurement     ← maps from org group "Procurement Team"
```

Mapping rules:

```
{
  "appRole": "inventory_admin",
  "when": [
    { "orgRole": "admin" },
    { "orgRole": "manager", "department": "warehouse" }
  ]
}
```

Layer 3: Field/Record-Level Access

Defines granular data visibility and editability.

Field-level:

- Which roles can VIEW each field
- Which roles can EDIT each field
- Default: all fields visible and editable to all app users

Record-level (data scoping):

- Which records the user can see
- Based on: department, team, ownership (created_by), manager chain
- Example: "staff sees only records from their department"
- Example: "manager sees records from their dept + all sub-departments"
- Example: "admin sees all records"

Composition example:

User: Alice (Sales Manager, NY Office)

Org roles: manager (level 60), sales_department

App roles (CRM): crm_admin

App roles (Inventory): inventory_viewer

In CRM app:

- ✓ Can see all customers (crm_admin: full access)
- ✓ Can edit deals she owns (record-level: owner = self)
- ✓ Can see commission field (field-level: sales_department)
- x Cannot see customer SSN (field-level: restricted to compliance)

In Inventory app:

- ✓ Can view products (inventory_viewer: read-only)
- x Cannot create/edit products (viewer role)
- ✓ Can see prices (field-level: visible to manager+)
- x Cannot see cost/margin (field-level: restricted to finance_dept)

22.6 Field-Level Access Enforcement

Field-level access is enforced at THREE layers:

1. API Layer (mandatory – security boundary)

Every API route filters response fields based on the user's roles.

```
// Generated in API routes:
const userRoles = getUserRoles(request);
const allowedFields = getViewableFields('Product', userRoles);
const products = await db.select(
  ...allowedFields.map(f => products[f])
).from(products);
```

For writes:

```
const editableFields = getEditableFields('Product', userRoles);
const filteredBody = filterObject(body, editableFields);
await db.update(products).set(filteredBody).where(...);
```

2. UI Layer (convenience – better UX)

Components hide fields the user can't see, disable fields they can't edit.
The UI reads the access matrix from a /api/me/permissions endpoint.

```
// Generated in components:
const { canView, canEdit } = useFieldPermissions('Product');
{canView('unitCost') && <td>{product.unitCost}</td>}
```

```
{canEdit('name')} ? <input .../> : <span>{product.name}</span>}
```

3. Database Layer (optional – defense in depth)

For highly sensitive fields, a PostgreSQL row-level security policy can be applied. This is only used for fields marked as "restricted" in the access matrix (e.g., SSN, salary).

-- Generated RLS policy:

```
CREATE POLICY product_cost_access ON products
  FOR SELECT
  USING (current_setting('app.user_role') IN ('finance', 'admin')
        OR unitCost IS NULL);
```

22.7 Record-Level Access (Data Scoping)

Record-level access controls WHICH records a user can see. Based on the user's position in the org structure.

Scope Types:

1. Department scope

"User sees records belonging to their department"

Implementation: WHERE department_id = :userDepartmentId

2. Department + sub-departments scope

"Manager sees records from their dept and all sub-departments"

```
Implementation: WHERE department_id IN (
  SELECT id FROM departments WHERE id = :userDeptId
  UNION
  SELECT id FROM department_closure WHERE ancestor_id = :userDeptId
)
```

3. Team scope

"User sees records belonging to their team"

Implementation: WHERE team_id = :userTeamId

4. Owner scope

"User sees only records they created"

Implementation: WHERE created_by = :userId

5. Manager chain scope

"Manager sees records created by their direct reports"

```
Implementation: WHERE created_by IN (
  SELECT id FROM org_people WHERE manager_id = :userId
)
```

6. Unrestricted

"User sees all records" (admin, finance, etc.)

Implementation: no WHERE clause added

Scope Resolution:

Each model can have a scope rule in the access matrix.

The highest-privilege scope wins (if user has multiple roles).

Scopes are additive: if any role grants access, the record is visible.

Generated Middleware:

```
// src/infrastructure/auth/rbac.ts
export function applyScopeFilter(
  query: DrizzleQuery,
  model: string,
  user: AuthUser
): DrizzleQuery {
```

```

const scope = getScopeRule(model, user.appRoles);
switch (scope.type) {
  case 'department':
    return query.where(eq(table.departmentId, user.departmentId));
  case 'department_tree':
    return query.where(inArray(table.departmentId, user.departmentTree));
  case 'owner':
    return query.where(eq(table.createdBy, user.id));
  case 'unrestricted':
    return query;
}
}

```

22.8 Workflow Assignments & Approvals

Workflows in generated apps can assign tasks to people based on their position in the org structure.

Assignment Types:

1. By Role

Assign to anyone with a specific org or app role.

Config: `{"type": "role", "role": "manager", "scope": "department"}`

Resolved: finds all managers in the requester's department

2. By Org Position

Assign to a specific position relative to the requester.

Config: `{"type": "manager_of", "target": "requester"}`

Resolved: looks up requester.managerId in org_people

3. By Department

Assign to the department head or any member.

Config: `{"type": "department_head", "department": "finance"}`

Resolved: looks up departments.headUserId where name = 'finance'

4. By Group

Assign to any member of a specific group.

Config: `{"type": "group", "group": "Audit Committee"}`

Resolved: random or round-robin from group members

5. By Specific Person

Assign to a named individual.

Config: `{"type": "person", "personId": "uuid-xxx"}`

Resolved: directly assigned

6. Dynamic Rule

Assign based on a computed expression.

Config: `{"type": "rule", "expression": "person with fewest open tasks in role 'reviewer'"}`

Resolved: LLM or query-based resolution

Task Assignment Flow:

1. Workflow reaches an AssignmentNode or ApprovalNode
2. Runtime resolves the assignment target from org structure
3. Creates a task record in the app's tasks table
4. Sends notification (email + in-app) to assignee(s)
5. Task appears in assignee's task inbox (cross-app portal)
6. Assignee completes task (fills form, clicks approve/reject)
7. Workflow resumes with the task result

Approval Types:

- Single: one person approves → done
- Sequential: A approves → B approves → C approves (in order)

- Parallel (all): all approvers must approve (any reject = rejected)
- Parallel (any): first approval wins
- Threshold: 3 of 5 must approve

Escalation:

If task not completed within SLA:

1. Send reminder notification
2. After 2nd reminder: escalate to next level in org chart
3. Escalation target: explicitly configured OR auto (requester's manager's manager)
4. Log escalation event in workflow execution log

Generated Schema (in app):

```
tasks:
  id, workflowId, workflowRunId, nodeId,
  title, description, formSchema (jsonb),
  assigneeId, assigneeType (person|role|group),
  status (pending|claimed|completed|expired|escalated),
  result (jsonb), dueAt, completedAt, completedBy,
  slaHours, escalatedAt, escalatedTo,
  createdAt

task_comments:
  id, taskId, authorId, content, createdAt
```

22.9 Multi-App Portal

End-users of generated apps need a single entry point to access all their apps.

The org portal is auto-generated as a special app within each org.
It aggregates data from all apps the user has access to.

Route: `https://{org-slug}.appforge.app/` (or self-hosted equivalent)

Acme Corp

Welcome, Alice ▾

Your Applications



Inventory

2 alerts



CRM

5 leads



Expenses

1 pending

Pending Tasks (3)

[\[View All →\]](#)



Approve expense #4521 from Bob (\$340)

Expenses app · Due in 4 hours



Review Purchase Order #892

Inventory app · Due tomorrow



Sign off on Q1 budget

Finance app · Due in 3 days

Recent Activity

• Inventory: 3 products below reorder point (2h ago)

• CRM: New lead assigned to your team (5h ago)

• Expenses: Monthly report ready (1d ago)

Portal Features:

- App grid with role-based visibility (shows only apps user can access)
- Badge counts from each app (pending tasks, alerts, new items)
- Unified task inbox (tasks from all apps in one list)
- Cross-app notifications
- Quick search across all apps
- User profile and settings
- App switching without re-authentication (shared SSO token)

Implementation:

The portal is a generated Next.js app like any other.

It queries each app's API for:

- GET /api/portal/tasks → pending tasks for current user
- GET /api/portal/badges → alert counts
- GET /api/portal/activity → recent activity feed

These endpoints are auto-generated in every app that has the portal module.

22.10 Org-Aware App Generation

When the Planner generates an app plan, it reads the org structure and auto-infers the access model. The developer does NOT need to specify roles, permissions, or approval chains – the Planner infers them.

Example:

Developer: "Build me an expense approval system"

Planner reads org structure:

- 4 departments: Sales (25), Engineering (40), Finance (15), HR (10)
- Roles: staff, lead, manager, director, VP, CEO
- Reporting: staff → lead/manager → director → VP → CEO

Planner auto-generates access_control in the plan:

- App roles mapped from org roles
- Field access for sensitive fields
- Record scope based on department
- Workflow assignments using org hierarchy
- Approval chain following reporting lines

The developer reviews the plan and can adjust before approving.

22.11 Multi-Tenancy Architecture

Multi-tenancy operates at TWO levels:

Level 1: Platform (AppForge itself)

Strategy: Shared database with org_id column

- All organizations share the same platform PostgreSQL
- Every query is scoped by org_id
- Platform tables: organizations, org_people, org_roles, projects, etc.
- Middleware automatically injects org_id filter
- This is standard SaaS multi-tenancy

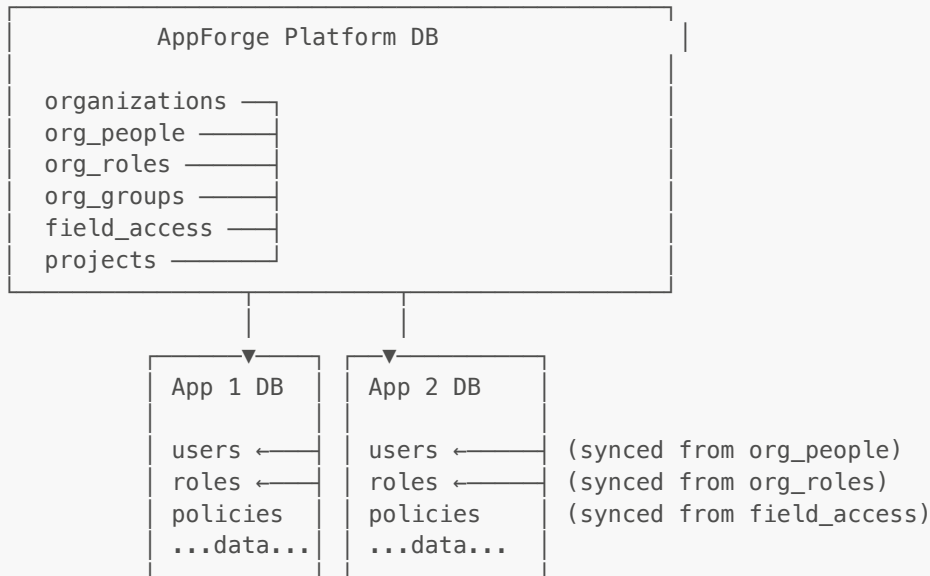
Level 2: Generated Apps (what users build)

Strategy: Database-per-app (already in the blueprint)

- Each generated app gets its own Docker Compose + PostgreSQL
- Complete data isolation between apps
- Apps within the same org share IDENTITY but not DATA
- Identity is synced from platform org_people → app's users table

- Each app is a standalone deployment

Data Flow:



Sync triggers:

- On preview start: full sync (org_people → app users table)
- On org structure change: incremental sync to all running previews
- On deploy: full sync embedded in the app's startup script
- App checks for updates every 5 minutes in production

What gets synced:

- People: id, email, name, title, departmentId, teamId, managerId, roles
- Roles: id, name, level
- Field access policies: model, field, role → view/edit permissions
- Workflow assignments: which roles handle which workflow steps
- Groups: id, name, member IDs

What stays in the app DB:

- All business data (products, orders, tickets, etc.)
- Conversations (agent chat history)
- Workflow execution logs
- Task assignments and completions
- File uploads

22.12 Org Structure in AppModel Index

```
// Added to app-model.json when org structure is active

"orgAware": {
  "enabled": true,
  "syncedFrom": "platform",

  "appRoles": [
    {
      "name": "expense_submitter",
      "mapsFrom": [{"type": "all"}],
      "permissions": ["expense:create", "expense:read:own"]
    },
    {
      "name": "expense_approver",
      "mapsFrom": [{"type": "org_role", "role": "manager", "level_gte": 60}],
      "permissions": ["expense:read:reports", "expense:approve"]
    }
  ]
}
```

```

    {
      "name": "expense_admin",
      "mapsFrom": [{"type": "department", "department": "Finance"}],
      "permissions": ["expense:read:all", "expense:edit:all", "expense:admin"]
    }
  ],

  "fieldAccess": [
    {"model": "Expense", "field": "receiptUrl", "view": ["*"], "edit": ["expense_submitter"]},
    {"model": "Expense", "field": "approverNotes", "view": ["expense_approver", "expense_admin"], "edit": ["expense_approver"]},
    {"model": "Expense", "field": "costCenter", "view": ["expense_admin"], "edit": ["expense_admin"]}
  ],

  "recordScope": [
    {
      "model": "Expense",
      "rules": [
        {"role": "expense_submitter", "scope": "owner", "column": "created_by"},
        {"role": "expense_approver", "scope": "manager_chain", "column": "created_by"},
        {"role": "expense_admin", "scope": "unrestricted"}
      ]
    }
  ],

  "workflowAssignments": [
    {
      "workflow": "expense-approval",
      "node": "manager-approve",
      "assignTo": {"type": "manager_of", "target": "requester"},
      "slaHours": 48,
      "escalateTo": {"type": "manager_of", "target": "current_assignee"}
    }
  ]
}

```

23. Discovery & Templates

23.1 Overview

Not every user arrives with clear requirements or a Figma design. The Discovery system handles four entry points:

1. **Problem-first:** "Our onboarding takes too long" → Discovery Agent identifies the problem, asks about current process, proposes an app
2. **Reference-based:** "Build something like Trello" → Agent identifies the reference app's features, asks which are relevant, builds a tailored plan
3. **Department-need:** "I need something for my HR team" → Agent checks org structure, identifies the department's gaps, suggests apps
4. **Vague idea:** "I want to track something" → Agent guides through structured exploration to clarify what, who, and why

All paths converge on a **structured brief** that feeds into the Planner agent — so the downstream pipeline is unchanged.

23.2 Discovery Agent (#12)

```
# backend/agents/discovery.py
```

```
DISCOVERY_SYSTEM_PROMPT = """You are a product discovery agent. Your job is to help users who
```


don't have clear requirements figure out what application they need.

You are warm, patient, and curious. You ask focused questions and build understanding incrementally. Never overwhelm with options – guide the conversation naturally.

Your Context

You have access to:

- The user's organization structure (departments, teams, roles, people)
- The org's existing apps (so you don't suggest duplicates)
- A template library of pre-built app plans

Discovery Types

Detect which type of discovery the user needs:

PROBLEM_FIRST

User describes a pain point, not a solution.

Strategy: Understand the current process → identify bottlenecks → propose a solution.

Questions:

1. "Walk me through how this works today – step by step."
2. "Where does it break down? What takes too long or goes wrong?"
3. "Who's involved in this process? What are their roles?"
4. "What would 'fixed' look like? What's the ideal outcome?"

REFERENCE_BASED

User names an existing product ("like Trello", "like Salesforce").

Strategy: Identify the reference → list its key features → ask which matter → customize.

Questions:

1. "What do you like most about [reference]? Which features do you actually use?"
2. "What's missing or annoying about it?"
3. "Who on your team would use this? How many people?"
4. "Any specific workflows you need that [reference] doesn't handle well?"

DEPARTMENT_NEED

User mentions a team or department without specifying what they need.

Strategy: Check org structure → identify department's function → suggest relevant apps.

Questions:

1. "I can see your [department] has [N] people. What are their main responsibilities?"
2. "What tools do they currently use? Spreadsheets, email, other software?"
3. "What's the biggest time sink for the team right now?"
4. "Are there any compliance or reporting requirements I should know about?"

VAGUE_IDEA

User has a loose concept but can't articulate requirements.

Strategy: Structured exploration through who/what/why framework.

Questions:

1. "Who will use this app? Just you, your team, or the whole org?"
2. "What's the core thing they need to do? (Track something? Approve something? Report?)"
3. "How often will they use it – daily, weekly, occasionally?"
4. "Is there a deadline or event driving this? Why now?"

Rules

- Ask 3–5 questions total, not all at once. One question per turn, maybe two.
- After each answer, summarize your understanding and ask the next question.
- When you have enough info, produce a STRUCTURED BRIEF (see format below).
- If a template matches well (>70% fit), suggest it and ask about customizations.
- Always check existing org apps to avoid suggesting duplicates.
- Include org structure context: suggest using existing roles for RBAC.

Structured Brief Format

When you have enough information, produce this JSON:

```
{
  "discovery_type": "problem_first|reference_based|department_need|vague_idea",
  "app_name": "Suggested name",
  "description": "One-paragraph summary of what the app does",
  "target_users": {
    "departments": ["HR"],
    "roles": ["HR Manager", "Recruiter"],
    "estimated_users": 15
  },
  "core_entities": ["Candidate", "Interview", "JobPosting", "Offer"],
  "key_workflows": [
    "Job posting approval flow",
    "Interview scheduling",
    "Offer approval chain"
  ],
  "must_have_features": [
    "Candidate pipeline view (kanban)",
    "Interview calendar integration",
    "Offer letter generation"
  ],
  "nice_to_have_features": [
    "AI resume screening",
    "Automated reference check emails"
  ],
  "matched_template": "hr-recruiting" or null,
  "template_customizations": ["Add interview scoring rubric", "Remove onboarding module"],
  "rbac_suggestions": {
    "HR Manager": "full access",
    "Recruiter": "own candidates only",
    "Hiring Manager": "view assigned candidates, approve/reject"
  },
  "integrations": ["email", "calendar"],
  "complexity": "medium"
}

This brief will be handed to the Planner agent to produce a full technical plan.
"""

DISCOVERY_OPTIONS = ClaudeAgentOptions(
    system_prompt=DISCOVERY_SYSTEM_PROMPT,
    allowed_tools=["Read"], # Can read org structure, existing apps, templates
    permission_mode="bypassPermissions",
    max_turns=20,
    model="claude-sonnet-4-20250514",
)
```

23.3 Template Library

Templates are pre-built Planner-format plans that can be used as-is or customized.

Template Categories

Category	Templates
Operations	Task Manager, Project Tracker, Asset Manager, Inventory System, Facility Booking, Incident Tracker
Sales	CRM Basic, CRM Advanced, Lead Pipeline, Quote Generator, Commission Tracker, Partner Portal
HR	Employee Directory, PTO Tracker, Recruiting Pipeline,

	Onboarding Checklist, Performance Reviews, Training LMS
Finance	Expense Tracker, Invoice Manager, Budget Planner, Purchase Orders, Reimbursement System, Petty Cash
Support	Help Desk, Knowledge Base, Bug Tracker, Customer Feedback, SLA Monitor, Escalation Manager
IT	Change Request, Service Catalog, License Manager, Device Inventory, Access Request, Audit Logger
Marketing	Campaign Tracker, Content Calendar, Event Manager, Lead Scoring, Survey Builder, Brand Asset Library
Legal	Contract Manager, NDA Tracker, Compliance Checklist, Policy Library, Case Manager, Document Approval

Template Structure

```
interface AppTemplate {
  slug: string // "crm-basic"
  name: string // "Basic CRM"
  description: string
  category: string // "Sales"
  subcategory?: string // "Pipeline"
  icon: string // lucide icon name
  tags: string[] // ["crm", "sales", "pipeline"]
  complexity: 'simple' | 'medium' | 'complex'
  estimatedModules: number

  // The full plan in Planner output format
  plan: {
    entities: EntityDefinition[]
    pages: PageDefinition[]
    workflows: WorkflowDefinition[]
    rules: RuleDefinition[]
    modules: ModuleDefinition[]
    navigation: NavigationDefinition
    seedData: SeedDataDefinition
    access_control?: AccessControlDefinition
    ai_features?: AIFeaturesDefinition
    agents?: AgentDefinition[]
  }

  // Org awareness
  relevantDepartments: string[] // ["Sales", "Marketing"]
  suggestedRoles: { // default RBAC for this template
    roleName: string
    accessLevel: string
    description: string
  }[]

  // Customization hints
  customizableAreas: {
    area: string // "entities", "workflows", "pages"
    description: string // "Add custom fields to Contact"
    optional: boolean
  }[]

  previewImage?: string
}
```

23.4 Org-Aware Suggestions

When an org has departments set up, the platform proactively suggests relevant apps:

```
// Suggestion engine logic (backend)

interface AppSuggestion {
  template: AppTemplate
  reason: string // "Your HR department has 12 people but no HR apps"
  relevance: number // 0-100 score
  existingApps: string[] // apps already covering this area
  gap: string // "No recruiting pipeline tool found"
}

async function getSuggestedTemplates(orgId: string): Promise<AppSuggestion[]> {
  const org = await getOrgWithStructure(orgId)
  const existingApps = await getOrgProjects(orgId)
  const templates = await getAllTemplates()

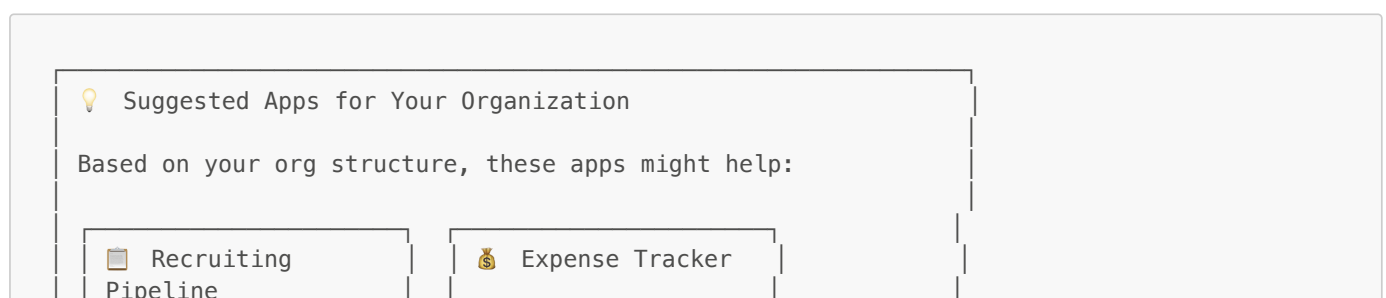
  const suggestions: AppSuggestion[] = []

  for (const dept of org.departments) {
    // Find templates relevant to this department
    const relevant = templates.filter(t =>
      t.relevantDepartments.some(d =>
        d.toLowerCase() === dept.name.toLowerCase()
      )
    )

    // Check which areas are already covered by existing apps
    for (const template of relevant) {
      const covered = existingApps.some(app =>
        app.tags?.some(tag => template.tags.includes(tag))
      )
      if (!covered) {
        suggestions.push({
          template,
          reason: `Your ${dept.name} department (${dept.people.length} people) has no
${template.category.toLowerCase()} tools`,
          relevance: calculateRelevance(dept, template, org),
          existingApps: [],
          gap: `No ${template.name.toLowerCase()} found`
        })
      }
    }
  }

  return suggestions.sort((a, b) => b.relevance - a.relevance)
}
```

Suggestions UI (on Org Dashboard)



Your HR dept (12 people) has no recruiting tool

[Use Template]

[Customize First]

[Dismiss]

Your Finance team (8 people) has no expense management

[Use Template]

[Customize First]

[Dismiss]

23.5 Template Gallery Page

Template Gallery

Search

Categories: [All] [Operations] [Sales] [HR] [Finance] [Support] [IT] [Marketing] [Legal]

Basic CRM

Track leads, deals, and pipeline

Sales

●●○ Medium

[Preview]

[Use →]

Employee Directory

Searchable org-wide people directory

HR

●○○ Simple

[Preview]

[Use →]

Task Manager

Kanban boards, deadlines, assignments

Operations

●●○ Medium

[Preview]

[Use →]

Template Detail (expanded on click)

Basic CRM

[Use →]

Data Models: Contact, Company, Deal, Activity, Pipeline

Pages: Dashboard, Contacts, Deals, Pipeline Board

Workflows: Deal stage change → notify, Weekly forecast

Modules: 3 (Core, Pipeline, Reporting)

Customizations available:

☒ Add custom fields to Contact

☒ Change pipeline stages

☒ Add email integration

☐ Add AI lead scoring (requires AI features)

Suggested roles: Sales Rep, Sales Manager, Admin

[Customize & Create] [Use As-Is]

23.6 Discovery Conversation UI

← Back to Org Dashboard

109 / 123

Let's figure out what you need

 I can help you figure out what app to build.

Tell me about:

- A problem your team faces
- An existing tool you want to replace
- A department that needs better tools
- Or just a rough idea – we'll work it out together

 Our HR team spends hours every week tracking PTO requests through email. It's a mess.

 I see your HR department has 12 people. Let me understand the current process:

How does someone request PTO today – do they email their manager directly, or is there a shared inbox?

... (conversation continues) ...

 I have a good understanding now. Here's what I recommend:

 PTO & Leave Manager

- PTO request form with date picker
- Manager approval workflow
- Leave balance tracking (accrual rules)
- Team calendar view
- HR dashboard with reports

Based on template: PTO Tracker
Customized: + approval chain via managers, + accrual rules, + Slack notifications

RBAC: HR Manager (full), Managers (approve for team), Employees (own requests only)

[Build This App →] [Adjust Requirements]

Type a message...

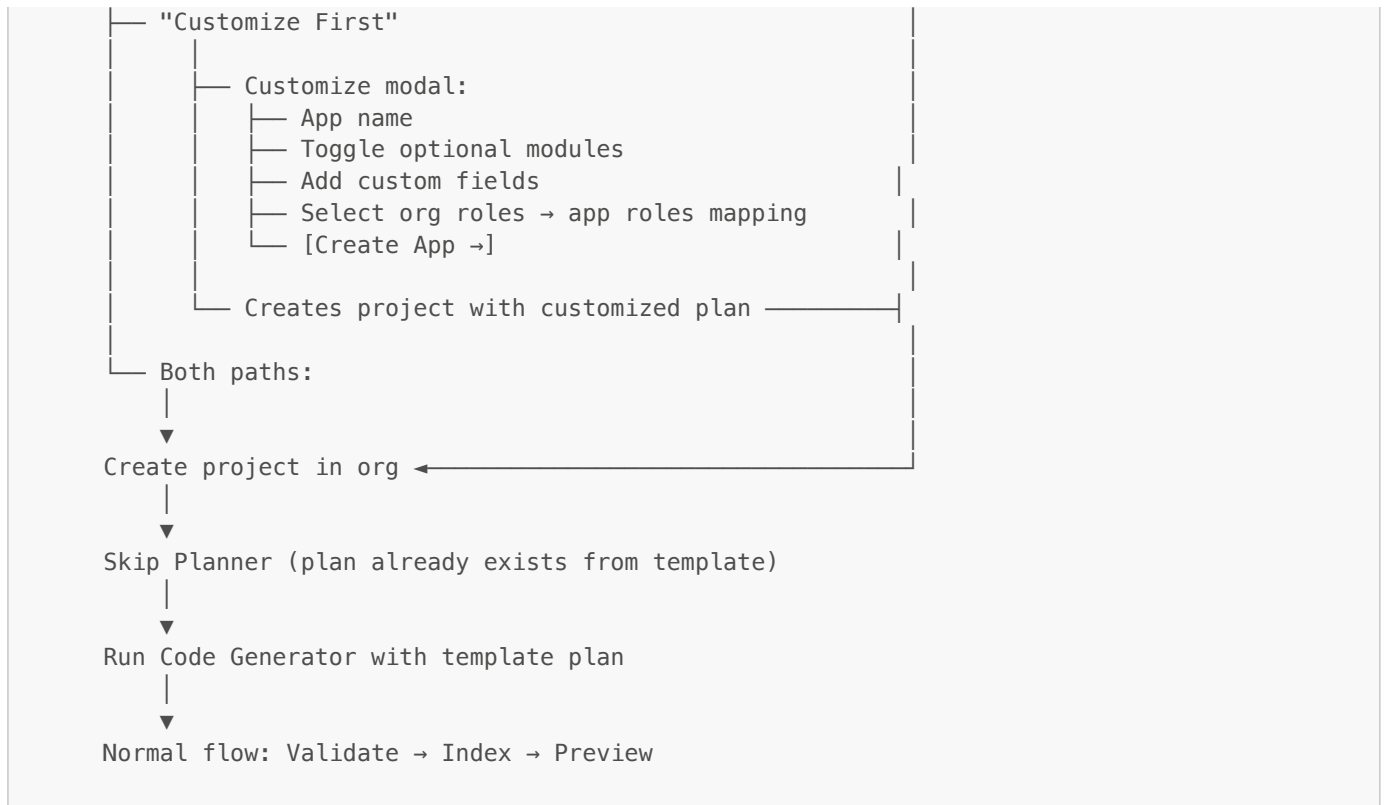
[Send]

23.7 Template-to-Project Flow

Template Selected

"Use As-Is"

110 / 123



23.8 Discovery-to-Project Flow



24. Implementation Phases

Phase 0: Organization Foundation (Weeks 1-2)

Goal: Multi-tenant org structure with people, roles, and groups

Tasks:

- Platform PostgreSQL schema: organizations, org_members, departments, teams, org_people, org_roles, org_person_roles, org_groups, org_group_members
- Platform auth (signup/login/JWT with orgId in token)
- Organization CRUD endpoints
- Org membership and invitation flow
- Org people CRUD endpoints
- Department and team CRUD endpoints
- Org role management (default roles auto-created on org creation)
- Group management endpoints
- CSV/JSON import for bulk people upload with column mapping
- Org chart API endpoint (full tree for React Flow)
- Frontend: org creation wizard
- Frontend: org dashboard (apps grid + sidebar navigation)
- Frontend: org chart visual editor (React Flow: departments, people, drag)
- Frontend: people directory table with search/filter
- Frontend: role and group management pages
- Frontend: CSV import with preview and column mapping

Deliverable: Developer can create an org, upload their people via CSV, visually arrange the org chart, define roles and groups.

Phase 1: Foundation (Weeks 3-5)

Goal: Basic conversational app generation with live preview (org-scoped)

Tasks:

- Set up monorepo structure (backend + frontend)
- Project model scoped to org (org_id foreign key)
- Project CRUD endpoints (scoped to current org)
- Code Generator agent (Agent #4) with PostgreSQL/Drizzle prompt
- Validator agent (Agent #8)
- Indexer agent (Agent #7) – basic version
- Preview manager with Docker Compose support
- SSE streaming infrastructure
- Frontend: project list, creation, chat panel
- Frontend: progress stream display
- Frontend: live preview iframe with device frames
- Frontend: basic file tree + Monaco code editor
- Version history via git (auto-commit after each agent run)
- Template DB tables and seed with initial templates (8 categories)
- Template gallery page (filterable grid)
- "New App" creation options (describe / template / discover / figma)

Deliverable: User can describe an app in chat, get it generated with PostgreSQL, preview it live, browse the source code, or start from a template.

Phase 2: Refinement & Chat (Weeks 6-7)

Goal: Conversational refinement of generated apps

Tasks:

- ☐ Orchestrator agent (Agent #0) – intent classification
- ☐ Refiner agent (Agent #2) – index-aware code editing
- ☐ Planner agent (Agent #1) – multi-turn planning with structured output
- ☐ Explainer agent (Agent #3)
- ☐ Scaffold agent (Agent #6) – add features to existing apps
- ☐ Code Editor agent (Agent #5) – focused single-file edits
- ☐ Conversation persistence (platform DB)
- ☐ Plan approval UI (review + approve/reject in chat)
- ☐ Undo/revert via git (version history UI)
- ☐ Frontend: refine bar below preview
- ☐ Frontend: chat history persistence
- ☐ Discovery Agent (Agent #12) – system prompt and multi-turn conversation
- ☐ "DISCOVER" intent in Orchestrator routing
- ☐ Discovery session persistence (DB table)
- ☐ Discovery conversation UI page
- ☐ Org-aware template suggestions engine
- ☐ Template-to-project and discovery-to-project flows
- ☐ Suggested apps section on org dashboard

Deliverable: User can have multi-turn conversations, plan modules, refine the app through chat, undo changes, discover requirements through guided conversation, or start from suggested templates.

Phase 3: Data Model Editor (Weeks 8-9)

Goal: Visual ERD editor for data models

Tasks:

- ☐ ERD canvas with React Flow
- ☐ Entity cards with field lists
- ☐ Add/edit/delete models
- ☐ Add/edit/delete fields (type picker, constraint checkboxes)
- ☐ Relationship drawing (drag entity to entity)
- ☐ Enum editor
- ☐ Index editor
- ☐ Seed data table editor
- ☐ "Generate realistic data" button (LLM call)
- ☐ Impact analysis on model changes
- ☐ Instruction builder for data editor actions
- ☐ Schema change → drizzle-kit push integration
- ☐ Database browser (table grid, inline editing)
- ☐ SQL console

Deliverable: User can visually manage all data models, see relationships, edit seed data, and browse the live database.

Phase 4: Rules & Access Control Editor (Weeks 10-12)

Goal: Visual rules management + org-aware access control

Tasks:

- ☐ Rules table with filtering
- ☐ Validation rule form (model, field, condition builder, message, enforcement)
- ☐ Access control rule form (role matrix)
- ☐ Business rule form
- ☐ Computed field rule form
- ☐ State machine editor (React Flow mini-diagram)
- ☐ Trigger rule form
- ☐ Condition builder component (visual expression builder)
- ☐ Instruction builder for rule actions

- Cross-reference: show rules in data model editor field properties
- Cross-reference: show access rules in UI editor component properties
- Field Access Editor: model × role matrix (view/edit permissions per field)
- Record Scope editor: scope rules per model (department, team, owner, etc.)
- App-role mapping: org roles → app roles configuration
- App access policies: which org roles/groups can access this app
- RBAC middleware template for generated apps (field filter + scope filter)
- Org structure sync: platform org_people → generated app users table
- Planner agent: auto-infer access_control from org structure in plans
- useFieldPermissions React hook template for generated app components
- /api/me/permissions endpoint template for generated apps

Deliverable: User can create rules, configure field-level and record-level access visually, map org roles to app roles. Generated apps enforce RBAC at API, UI, and DB layers.

Phase 5: Workflow Editor (Weeks 13-15)

Goal: Visual workflow builder with execution

Tasks:

- Workflow canvas with React Flow
- Node types: trigger, action, condition, wait, end
- Node palette (draggable node types)
- Edge types: default, then, else, error
- Node properties panel
- Variable picker with autocomplete ({{trigger.field}}, {{step.output}})
- Workflow definition JSON format
- Workflow runtime (executor in generated app)
- Trigger registration system
- Workflow action generator agent (Agent #10)
- Instruction builder for workflow actions
- Workflow tester (run with sample data, highlight active node)
- Execution log viewer
- Built-in action types: DB query, HTTP call, email, notification
- Assignment node: assign task to user/role/manager/dept/group
- Approval node: single/sequential/parallel approval with SLA
- Task pool node: claim model, round-robin, skills-based
- Escalation node: SLA breach → escalate up org chart
- Task inbox page template for generated apps
- Org structure integration: resolve "requester's manager" dynamically
- Notification system: email + in-app notifications for task assignments

Deliverable: User can visually design workflows with human-in-the-loop assignments and approvals that resolve against the org structure.

Phase 6: UI Editor (Weeks 16-19)

Goal: Visual UI editing with component drag-and-drop

Tasks:

- GrapesJS + @grapesjs/react integration (EditorShell with <Canvas/>)
- White-label: disable all default GrapesJS UI, render custom shadcn/ui panels
- Custom component types plugin (appforge-components.ts) – register all block types
- Form components plugin (form-components.ts) – 25+ form field types
- Tailwind integration plugin – Tailwind CSS inside canvas iframe
- Component palette sidebar (shadcn/ui, grouped by category: Layout/Display/Form/Nav/Data/AI)
- Layer panel (shadcn/ui tree, powered by GrapesJS Layer Manager API)
- Properties panel (shadcn/ui, powered by GrapesJS Trait Manager API)

- Style panel (shadcn/ui, powered by GrapesJS Style Manager API)
- Device toolbar – responsive preview (desktop/tablet/mobile via Device Manager)
- Editor toolbar – undo/redo/preview/save (powered by GrapesJS commands)
- Field mapper – bind data model fields to component traits
- Canvas component views – rich editor previews (DataTable, Chart, KanbanBoard)
- Instruction builder for UI editor actions → Refiner agent
- Page-level layout editing (sidebar, header, content areas)

Deliverable: User can visually edit UI components, bind data models, and see changes reflected in the live preview.

Phase 7: Navigation & Module System (Weeks 20-21)

Goal: Multi-module apps with visual navigation editing

Tasks:

- Navigation editor (React Flow screen diagram)
- Screen nodes with route + thumbnail
- Navigation wiring (draw links between screens)
- Sidebar/navbar menu configuration
- Module management UI (list, create, dependency graph)
- Cross-module dependency tracking
- Module creation wizard (triggers Planner agent)
- Connections map (visual overview of all module relationships)
- Module-scoped file organization
- Module-scoped AppModel index sections

Deliverable: User can build multi-module apps, manage module dependencies, and configure navigation between modules.

Phase 8: AI Agent Builder (Weeks 22-24)

Goal: Visual agent builder with runtime in generated apps

Tasks:

- Agent Builder agent (Agent #11) – system prompt and orchestration
- Agent Builder visual editor (React Flow canvas)
- Node types: SystemPrompt, Tool, Guardrail, Memory, HumanHandoff, Router
- Node palette and drag-to-canvas
- Properties panel for each node type
- System prompt editor with variable interpolation
- Tool picker – auto-discover from AppModel API routes
- Custom tool editor (name, description, input schema, handler file)
- Guardrail configuration forms (input/output validation, rate limits)
- Memory configuration (conversation, summarization, knowledge base toggle)
- Multi-agent router configuration
- Instruction builder for agent editor actions
- Agent runtime template (src/agents/runtime.ts)
- Tool registry template (src/agents/tools/registry.ts)
- Memory manager template (src/agents/memory.ts)
- Guardrails template (src/agents/guardrails.ts)
- Chat API route template (src/app/api/agents/[agentId]/chat/route.ts)
- ChatWidget component template (floating + full-page modes)
- Agent conversation tables in generated app schema
- Agent templates (support bot, data assistant, workflow agent, admin copilot, onboarding)
- Test console in agent builder (send messages, inspect tool calls)
- Knowledge base: pgvector setup, document upload, chunking, embedding, search tool
- Agent analytics dashboard template (conversations, tool usage, costs, errors)
- "AGENT" intent in Orchestrator routing
- AppModel index: agents section

Deliverable: User can visually create AI agents that run inside their generated app, using the app's own APIs as tools, with conversation memory, guardrails, and embedded chat UI.

Phase 9: AI-Powered App Features (Weeks 25-27)

Goal: Intelligent features embedded in generated apps

Tasks:

- ☐ Smart field runtime (src/infrastructure/ai/smart-fields.ts template)
- ☐ Smart field types: ai_classify, ai_summarize, ai_sentiment, ai_extract, ai_generate, ai_translate, ai_score
- ☐ Smart field configuration in Data Model Editor (properties panel)
- ☐ Smart field test console ("Test with sample data" button)
- ☐ Smart field UI indicators (+ icon, override, recompute)
- ☐ Async smart field computation (non-blocking API response)
- ☐ Semantic search: pgvector setup, embedding computation, hybrid search
- ☐ Semantic search UI component with score display
- ☐ Workflow AI nodes: ai_classify, ai_extract, ai_decide, ai_generate
- ☐ Workflow AI node palette entries and properties panels
- ☐ AI rules: content_moderation, similarity_check, ai_validation, ai_enrichment
- ☐ AI rule forms in Rules Editor
- ☐ AI-assisted UI components: SmartFormField, InlineAssistant
- ☐ NaturalLanguageQuery component (text → SQL → results → summary)
- ☐ SmartFilterBar component (natural language → structured filters)
- ☐ DataInsightsPanel component (auto-generated insights)
- ☐ Scheduled AI tasks via workflow engine
- ☐ AI configuration and cost tracking (ai-config.ts, ai-usage.ts)
- ☐ AI features overview panel in project settings
- ☐ Planner agent: auto-detect AI requirements from user descriptions
- ☐ Code Generator: generate smart field configs and AI components

Deliverable: Generated apps can include AI-powered features like auto-categorization, semantic search, content generation, intelligent workflow decisions, and natural language data queries – all configured visually and running on the app's own Anthropic API key.

Phase 10: Multi-App Portal & Polish (Weeks 28-30)

Goal: Production readiness

Tasks:

- ☐ Export: ZIP download with README
- ☐ Export: git push to user's repository
- ☐ Export: Dockerfile generation for production deployment
- ☐ Error handling: graceful agent failures with retry
- ☐ Error handling: preview server crash recovery
- ☐ Performance: agent response caching for common patterns
- ☐ Performance: incremental AppModel updates (not full re-index)
- ☐ UX: keyboard shortcuts for common actions
- ☐ UX: command palette (Cmd+K)
- ☐ UX: onboarding tutorial for new users
- ☐ UX: template apps (task manager, CRM, inventory – pre-built plans)
- ☐ Testing: integration tests for agent pipelines
- ☐ Testing: E2E tests for visual editors
- ☐ Documentation: user guide
- ☐ Monitoring: agent cost tracking per project
- ☐ Monitoring: error logging and alerting

Deliverable: Production-ready platform with multi-app portal, SSO across apps, and enterprise-grade deployment options.

Tasks (Portal):

- Multi-app portal generation (auto-generated per org)
- App grid with role-based visibility
- Unified task inbox (pending approvals/tasks across all apps)
- Cross-app notifications and activity feed
- Portal API endpoints in generated apps (/api/portal/tasks, badges, activity)
- SSO token sharing across apps in same org
- Quick search across all apps

Tasks (Polish):

- Export: ZIP download with README
- Export: git push to user's repository
- Export: Dockerfile generation for production deployment
- Error handling: graceful agent failures with retry
- Error handling: preview server crash recovery
- Performance: agent response caching for common patterns
- Performance: incremental AppModel updates (not full re-index)
- UX: keyboard shortcuts for common actions
- UX: command palette (Cmd+K)
- UX: onboarding tutorial for new users
- UX: template apps (task manager, CRM, inventory – pre-built plans)
- Testing: integration tests for agent pipelines
- Testing: E2E tests for visual editors
- Documentation: user guide
- Monitoring: agent cost tracking per project
- Monitoring: error logging and alerting

Timeline Summary

Phase 0:	Org Foundation	Weeks 1–2	(org structure, people, roles, chart editor)
Phase 1:	Foundation	Weeks 3–5	(core generation + preview + templates)
Phase 2:	Refinement & Chat	Weeks 6–7	(conversational editing + discovery agent)
Phase 3:	Data Model Editor	Weeks 8–9	(visual ERD + DB browser)
Phase 4:	Rules & Access Control	Weeks 10–12	(rules + field/record RBAC)
Phase 5:	Workflow Editor	Weeks 13–15	(workflows + assignments/approvals)
Phase 6:	UI Editor	Weeks 16–19	(visual UI editing)
Phase 7:	Navigation & Modules	Weeks 20–21	(multi-module apps)
Phase 8:	AI Agent Builder	Weeks 22–24	(visual agent builder + runtime)
Phase 9:	AI-Powered Features	Weeks 25–27	(smart fields, semantic search, AI workflows)
Phase 10:	Portal & Polish	Weeks 28–30	(multi-app portal + deployment + UX)
Total: ~30 weeks (7.5 months) for a small team (2–3 developers)			

Team Composition

Developer 1 (Backend/AI):

- All agent prompts and orchestration (including Agent Builder, Discovery agents)
- Discovery Agent system prompt and multi-turn conversation logic
- Template library curation and seed data
- Org-aware suggestion engine
- Backend API endpoints (org, RBAC, projects, templates, discovery)
- Org structure APIs and CSV import
- RBAC policy engine (field-level, record-level, workflow assignment)
- Preview manager + Docker integration
- Org-to-app identity sync system
- Workflow runtime (including assignment/approval/escalation)

- Agent runtime templates (for generated apps)
- Agent guardrails and memory system

Developer 2 (Frontend/Visual):

- All visual editors (React Flow, GrapesJS)
- Org chart visual editor (React Flow)
- Field access matrix editor
- Agent Builder visual editor (React Flow canvas)
- Template gallery page and template detail/customization views
- Discovery conversation UI
- Org dashboard suggested apps section
- "New App" creation flow (describe / template / discover / figma)
- Frontend pages and components (org dashboard, app workspace)
- State management (Zustand stores)
- Instruction builder (including agent + RBAC actions)
- Code editor (Monaco)
- Multi-app portal

Developer 3 (Full-stack/Integration):

- AppModel index system (including agents, orgAware sections)
- Binding and rules propagation
- Module system and cross-module dependencies
- Multi-tenancy middleware (org_id scoping)
- SSO and cross-app auth token system
- Agent tool registry and API-to-tool mapping
- Knowledge base (pgvector integration)
- Export/deployment features
- Testing and DevOps

Appendix A: Environment Variables

```
# Platform
DATABASE_URL=postgresql://appforge:appforge@localhost:5432/appforge
ANTHROPIC_API_KEY=sk-ant-...
JWT_SECRET=your-jwt-secret
FRONTEND_URL=http://localhost:3000

# Ports
BACKEND_PORT=6500
FRONTEND_PORT=3000

# Preview port ranges
PREVIEW_PORT_MIN=3200
PREVIEW_PORT_MAX=3299
DB_PORT_MIN=5500
DB_PORT_MAX=5599

# Agent config
DEFAULT_MODEL=claude-sonnet-4-20250514
UTILITY_MODEL=claude-haiku-4-5-20251001
MAX_GENERATION_TURNS=80
MAX_REFINEMENT_TURNS=30
```

Appendix B: Key Dependencies

```
# Backend (requirements.txt)
fastapi>=0.115.0
uvicorn[standard]>=0.34.0
```

```

claude-agent-sdk>=0.1.0
sqlalchemy[asyncio]>=2.0.0
asynpg>=0.30.0
alembic>=1.14.0
python-dotenv>=1.0.0
sse-starlette>=2.2.0
httpx>=0.28.0
pydantic>=2.10.0
python-jose>=3.3.0      # JWT
passlib>=1.7.4          # password hashing
docker>=7.0.0           # Docker SDK

# Frontend (package.json)
next: ^15.0.0
react: ^19.0.0
typescript: ^5.7.0
tailwindcss: ^4.0.0
@tailwindcss/postcss: ^4.0.0
zustand: ^5.0.0
@tanstack/react-query: ^5.0.0
@xyflow/react: ^12.0.0      # React Flow (for visual editors)
grapesjs: ^0.22.0           # GrapesJS core (UI editor engine)
@grapesjs/react: ^2.0.0     # React wrapper (white-label via <Canvas/>)
@monaco-editor/react: ^4.6.0 # Monaco editor
@dnd-kit/core: ^6.0.0       # drag and drop
lucide-react: ^0.468.0
react-hook-form: ^7.54.0
zod: ^3.24.0
ky: ^1.7.0

# Generated App – AI Dependencies (added when AI features are present)
@anthropic-ai/sdk: ^0.39.0  # Anthropic SDK for agent runtime + smart fields
pgvector: ^0.2.0            # pgvector client (semantic search + knowledge base)
openai: ^4.76.0             # OpenAI SDK for embeddings (text-embedding-3-small)

```

Appendix C: Design Decisions Log

Decision 1: Code as source of truth (not AppModel)

Why: Bidirectional sync between model and code is extremely hard.
The LLM can read code naturally. The AppModel index is just a navigation aid to help the LLM find files faster.

Decision 2: LLM for all code changes (not AST)

Why: Simpler architecture. One pipeline for chat + visual editors.
AST manipulation requires building parsers for every pattern.
LLM handles all patterns naturally. Speed tradeoff is acceptable for an app builder (not a real-time design tool).
Future: Add thin AST layer for high-frequency property changes (color picker, spacing slider) if latency becomes an issue.

Decision 3: PostgreSQL from day one (not SQLite)

Why: Real types, real constraints, real enums. Same database in dev and production. Docker Compose makes it zero-config.
Enterprise apps need Postgres features (JSONB, full-text search, proper transactions, concurrent writes).

Decision 4: Drizzle ORM (not Prisma)

Why: Schema is TypeScript (LLM already knows TypeScript).
No code generation step. No binary engine dependency.
drizzle-kit push for instant schema changes during development.
Easier for the LLM to edit (it's just a .ts file).

Decision 5: Module-based architecture for generated apps

Why: Enterprise apps are too large for single-pass generation.
Modules keep each generation task manageable (10–30 files).
Cross-module dependencies are explicit and tracked.
Modules can be added/removed independently.

Decision 6: Thin workflow runtime + LLM-generated action functions

Why: Runtime handles flow control (simple, deterministic).
LLM handles business logic (complex, needs context).
Visual editor writes workflow JSON (instant, no LLM needed).
Only action function generation needs LLM.
Best of both worlds: visual editing speed + LLM flexibility.

Decision 7: Haiku for utility agents, Sonnet for creative agents

Why: Orchestrator, Indexer, and Validator don't need creative problem-solving – they classify, extract, and verify.
Haiku is 10x cheaper and faster for these tasks.
Code Generator, Refiner, and Planner need strong reasoning and code generation capability – Sonnet is appropriate.

Decision 8: Agent runtime in generated app calls Anthropic API directly

Why: Agents run inside the generated app, not through our platform.
The generated app owns its own Anthropic API key.
This means deployed apps are independent – they don't need our platform running to serve agent requests.
The platform only creates the agent code; it doesn't proxy requests.
This architecture means generated apps are fully self-contained.

Decision 9: Agent tools use the app's own API routes (not direct DB)

Why: Reuses existing validation, auth, and business logic in API routes.
No duplicate code. Access control is already enforced at the API layer.
Tools are auto-discovered from the AppModel – no manual mapping.
If the API changes, the tool automatically gets the new behavior.
Only custom tools (knowledge base search, external APIs) need dedicated handler files.

Decision 10: pgvector for knowledge base (not external vector DB)

Why: The app already has PostgreSQL. pgvector is a simple extension.
No additional infrastructure (no Pinecone, Weaviate, etc.).
Keeps the generated app self-contained and easy to deploy.
HNSW indexes provide fast enough search for most use cases.
For very large knowledge bases (>1M chunks), can switch to dedicated vector DB – but that's an edge case for app builders.

Decision 11: Haiku for smart fields and AI workflow nodes

Why: Smart fields run on every record create/update – they must be fast and cheap. Haiku handles classification, extraction, and summarization well enough. Only complex generation tasks (e.g., content generation, nuanced decisions) use Sonnet.
Typical cost: ~\$0.001 per smart field computation.
At 1000 records/day with 3 smart fields each = ~\$3/day.

Decision 12: Async smart field computation (non-blocking)

Why: LLM calls take 200–2000ms. Blocking the API response would make the UI feel slow. Instead, the API returns immediately and smart fields populate asynchronously. The UI shows a loading indicator on smart fields until they're computed.
For use cases requiring synchronous computation (e.g., AI validation that must block), a sync mode is available.

Decision 13: Planner auto-detects AI requirements from natural language

Why: Users shouldn't have to explicitly request "AI features."
When someone says "auto-categorize tickets," the Planner should recognize this implies a smart field. This makes the

platform feel intelligent – it understands intent, not just explicit instructions. The mapping table in Section 21.2 covers common patterns.

Decision 14: Organization as root entity (not project)

Why: Enterprise apps don't exist in isolation. An org has multiple apps, shared users, shared roles, and cross-app workflows.

Making organization the root entity enables:

- Single identity across all apps (no duplicate accounts)
- Org-wide RBAC that the Planner can auto-infer
- Cross-app task inbox and portal
- Consistent access control patterns across apps

Decision 15: Org structure in platform DB, synced to generated apps

Why: The org structure is the developer's concern, not the app's.

Storing it in the platform DB means:

- One place to manage people, roles, departments
- Changes propagate to all apps automatically
- Generated apps are simpler (they consume, not define)
- No risk of org data drifting between apps

Sync is lightweight: just users, roles, and policies.

Decision 16: Planner auto-infers RBAC from org structure

Why: The developer should NOT have to manually specify "who can see what field." The Planner knows the org has departments and roles – it can infer sensible defaults:

- Sensitive fields (cost, salary) restricted to relevant dept
- Record scope based on department ownership
- Approval chains follow reporting lines

The developer reviews and adjusts, not builds from scratch.

Decision 17: Three-layer RBAC (org → app → field/record)

Why: Single-layer RBAC (just roles) is too coarse for enterprise.

Field-level access is critical: a sales manager should see deal amounts but not employee salaries. Record-level scoping is critical: a regional manager should only see their region. Three layers give maximum flexibility with minimum config.

Each layer has a sensible default (visible to all) so the developer only configures what needs restricting.

Decision 18: Discovery Agent produces a structured brief, not code

Why: The Discovery Agent's output is a requirements document (brief), not a technical plan. This keeps the pipeline clean:

Discovery → Brief → Planner → Plan → Code Generator → Code.

The Planner already knows how to turn requirements into plans.

Making Discovery produce plans would duplicate Planner logic.

The brief format is simple enough for the user to review and approve before passing to the Planner.

Decision 19: Template library with Planner-format plans (not generated code)

Why: Templates store structured plans (entities, pages, workflows), not pre-generated source code. This means:

- Templates work with any future code generation improvements
- Templates can be customized before generation
- The same template produces different code for different orgs (because the Code Generator adapts to org structure and RBAC)
- No maintenance burden of keeping template code up to date
- Templates are small (JSON) and easy to version

Decision 20: GrapesJS over Craft.js for UI editor

Why: GrapesJS provides 6–12 months of built-in features we'd have to build from scratch with Craft.js:

- Style Manager (visual CSS editor, 50+ properties, per-breakpoint)
- Layer Manager (component tree)

- Device Manager (responsive preview)
- Undo/Redo Manager (reliable, keyboard shortcuts)
- Block Manager, Selector Manager, Asset Manager

Craft.js is headless – zero built-in UI, everything from scratch. Craft.js also has 225 open issues, last npm publish >1 year ago, and the maintainer is focused on Reka.js rewrite. GrapesJS: 25K stars, 110K weekly downloads, actively maintained. Using @grapesjs/react with <Canvas/> we disable all default UI and build our own panels with shadcn/ui – fully white-labeled. The canvas renders in an iframe (style isolation is a feature, not a bug – generated app CSS can't break our editor chrome). Neither library generates React source code, but our architecture doesn't need that – the editor produces instructions, the LLM generates the code. GrapesJS just needs to be a great editor.

Decision 21: No separate Form Builder – forms are UI editor components

Why: A form is just a section of a page. Having a separate Form Builder and Page Builder forces users to choose "which editor?" – unnecessary friction. Instead, the UI Editor's component palette includes a rich Form category with 25+ field types (text, select, date, file, etc.). Each form component has traits for validation, data binding, conditional visibility, and submit actions. A multi-step wizard is just form groups inside a FormStepper component. This is what modern tools (Retool, Webflow) do – forms are components, not a separate editor.

Decision 22: Clean Architecture for generated apps

Why: The original flat structure (API routes with inline DB queries, business logic mixed with HTTP handling, grab-bag lib/ folder) made generated apps hard to test, extend, and reason about.

Clean Architecture enforces strict layer separation:

- domain/ – pure business logic, zero framework dependencies
- application/ – services orchestrating use cases
- infrastructure/ – DB repositories, auth, AI, email adapters
- app/ – thin HTTP handlers delegating to services
- components/ – presentation only

This structure gives the LLM agents clear placement rules for every type of code. Import direction is enforced: domain has zero dependencies, services coordinate domain + infrastructure, route handlers never touch the database directly. Generated apps become testable (domain logic is pure functions), maintainable (each layer can change independently), and production-ready. The cost is slightly more files, but the benefit is that both humans and LLMs always know where to put new code.

Decision 23: Server Components by default, Client Components only for interactivity

Why: Generated apps are enterprise CRUD tools – 80% of pages are read-heavy lists and detail views. Server Components eliminate client-side JS for these pages, giving instant first paint. Server Components can call services directly (no API fetch roundtrip for reads), which pairs naturally with Clean Architecture's service layer. Client Components are reserved for forms with state, real-time UI, and interactive tables. API routes remain the primary mutation pattern (not Server Actions) because they're more portable, testable, and the agents already know how to generate them. Server Actions may be used as thin wrappers calling the same services when convenient, but they're optional.

Decision 24: Socket.IO for real-time push, SSE for agent streaming

Why: Generated apps need real-time for specific features: smart field completion (async compute → push result), workflow task assignments (instant notification), live data refresh (another user updates a record), and collaborative editing (future). Socket.IO over raw WebSockets because:

- Automatic reconnection + fallback to long-polling
- Room-based broadcasting (per-org, per-user, per-record)
- Works behind corporate proxies that block WS upgrades
- Next.js lacks native WS support; Socket.IO's standalone server attaches to the same HTTP server cleanly

Real-time is NOT added to every app – only when the Planner detects features that need it (agents, workflows with assignments, smart fields, collaboration). This keeps simple apps simple. SSE stays for agent chat streaming because it's simpler, one-directional, and already works well. Using both SSE and Socket.IO is intentional: right tool for each job. Real-time files live in `src/infrastructure/realtime/` – isolated from business logic per Clean Architecture.